

الجمهورية الشعبية الديمقراطية الجزائرية
People's Democratic Republic of Algeria
وزارة التعليم العالي و البحث العلمي
Ministry of Higher Education and Scientific Research
المدرسة العليا للإعلام الآلي 8 ماي 1945 - سيدي بلعباس
Higher School of Computer Science
8 Mai 1945 - Sidi Bel Abbès



Graduation Thesis

To obtain the diploma of **Engineering Degree**

Field of Study: **Computer Science**

Specialization: **Information Systems and Web (SIW)**

Theme

A Knowledge Graph-Based Question Answering System for a Connected Smart Healthcare Assistant for Cardiovascular Patient

Presented by
NASSANE Marwa

Defended on: **25 September 2025**

In front of the jury composed of

Dr.MAHAMMED Nadir
Pr.MALKI Mimoun
Dr.MALKI Abdelhamid
Dr.BELMEKKI Ghizelane Amira
Dr.KECHAR Mohamed

President of the Jury
Thesis Supervisor
Co-Supervisor
Examiner
Representative of the incubator, Examiner

Academic Year: 2024/2025

Acknowledgement

First and foremost, I thank Almighty Allah for granting me the strength, patience, and guidance to complete this thesis successfully.

I am deeply grateful to my family, especially my beloved mother, whose prayers, support, and encouragement have been my greatest source of motivation throughout this journey.

I would like to express my deep gratitude to my supervisors: **Pr. MALKI Mimoun** and **Dr. MALKI Abdelhamid**, for their patient guidance, enthusiastic encouragement, and useful critiques of this research work. I would also like to express my sincere thanks to **Dr. KECHAR Mohamed** for his valuable advice, constant encouragement, and support during the realization of this project.

I would also like to acknowledge the president of the jury as well as the members of the jury for their interest in my research by agreeing to examine it and enrich it with their proposals.

My heartfelt thanks also go to my friends, who stood by me during this time and provided both academic and moral support, and to the ESI-SBA High School administration and all the staff members for their dedication, assistance, and efforts in creating an environment that fosters learning and growth.

Abstract

Medical decision support systems play a critical role in modern healthcare, as they assist doctors in delivering faster, more accurate, and evidence-based decisions. Knowledge Graphs are particularly important in this context, as they provide a structured and unified representation of complex medical data, facilitating integration, reasoning, and knowledge discovery across diverse sources

Question Answering systems built on KGs further enhance this process by allowing doctors to interact with medical knowledge in natural language, making access to information more intuitive and efficient.

Moreover, Electronic Medical Records which gives doctors direct access to patient history and clinical information, enabling more precise and personalized treatments. In addition, the use of medical sensors to continuously capture patients' vital signs is of great importance, as it not only supports real-time monitoring but also ensures constant and reliable communication between patients and doctors, thereby enhancing the quality of care and promoting proactive medical interventions.

This research aims to develop an intelligent decision support system that incorporates a Knowledge Graph-based Question Answering component, integrates seamlessly with Electronic Medical Records , and enables doctors to monitor patients in real time through medical sensors.

Key words: Medical Decision Support Systems, Knowledge Graphs, Question Answering Systems, Electronic Medical Records, Medical Sensors

Résumé

Les systèmes d'aide à la décision médicale jouent un rôle essentiel dans les soins de santé modernes, car ils assistent les médecins dans la prise de décisions plus rapides, plus précises et fondées sur des preuves. Les graphes de connaissances sont particulièrement importants dans ce contexte, car ils offrent une représentation structurée et unifiée des données médicales complexes, facilitant ainsi l'intégration, le raisonnement et la découverte de connaissances à partir de sources variées.

Les systèmes de questions-réponses basés sur les graphes de connaissances renforcent davantage ce processus en permettant aux médecins d'interagir avec le savoir médical en langage naturel, rendant l'accès à l'information plus intuitif et efficace.

De plus, les dossiers médicaux électroniques offrent aux médecins un accès direct à l'historique et aux informations cliniques des patients, permettant des traitements plus précis et personnalisés. L'utilisation de capteurs médicaux pour surveiller en continu les signes vitaux des patients revêt également une grande importance, car elle permet non seulement un suivi en temps réel, mais garantit aussi une communication constante et fiable entre patients et médecins, améliorant ainsi la qualité des soins et favorisant des interventions médicales proactives.

Cette recherche vise à développer un système intelligent d'aide à la décision médicale intégrant un module de questions-réponses basé sur un graphe de connaissances, s'intégrant de manière fluide aux dossiers médicaux électroniques, et permettant aux médecins de suivre les patients en temps réel grâce aux capteurs médicaux.

Mots-clés : systèmes d'aide à la décision médicale, graphes de connaissances, systèmes de questions-réponses, dossiers médicaux électroniques, capteurs médicaux

الملخص

تلعب أنظمة دعم القرار الطبي دوراً حاسماً في الرعاية الصحية الحديثة، إذ تساعد الأطباء على اتخاذ قرارات أسرع وأكثر دقة تستند إلى الأدلة. وتُعدّ الرسوم البيانية المعرفية مهمة بشكل خاص في هذا السياق، لأنها توفر تمثيلاً منظماً وموحداً للبيانات الطبية المعقدة، مما يسهل التكامل والاستدلال واكتشاف المعرفة من مصادر متنوعة.

كما أن أنظمة الإجابة عن الأسئلة المبنية على الرسوم البيانية المعرفية تعزز هذه العملية من خلال تمكين الأطباء من التفاعل مع المعرفة الطبية بلغة طبيعية، مما يجعل الوصول إلى المعلومات أكثر سهولة وفعالية.

علاوة على ذلك، فإن السجلات الطبية الإلكترونية تتيح للأطباء الوصول المباشر إلى التاريخ المرضي والمعلومات السريرية للمريض، مما يساعد على توفير علاجات أكثر دقة وشخصية. وبالإضافة إلى ذلك، فإن استخدام الحساسات الطبية لمتابعة المؤشرات الحيوية للمريض بشكل مستمر يعد أمراً بالغ الأهمية، لأنه لا يقتصر على دعم المراقبة في الوقت الحقيقي فحسب، بل يضمن أيضاً التواصل المستمر والموثوق بين المرضى والأطباء، مما يعزز جودة الرعاية الطبية ويدعم التدخلات الاستباقية.

يهدف هذا البحث إلى تطوير نظام ذكي لدعم القرار الطبي يدمج مكوناً للإجابة عن الأسئلة معتمداً على الرسوم البيانية المعرفية، ويتكامل بسلاسة مع السجلات الطبية الإلكترونية، ويسمح للأطباء بمتابعة المرضى في الوقت الحقيقي عبر الحساسات الطبية.

الكلمات المفتاحية: أنظمة دعم القرار الطبي، الرسوم البيانية المعرفية، أنظمة الإجابة عن الأسئلة، السجلات الطبية الإلكترونية، الحساسات الطبية

I	Introduction	14
1	INTRODUCTION	15
1.1	Context	15
1.2	Problem Statement	16
1.3	Objective	16
1.4	Thesis outline	16
II	Background	18
2	Ontology	19
2.1	What is Ontology	19
2.1.1	Definition 1	19
2.1.2	Definition 2	19
2.2	Components of Ontology	19
2.2.1	Classes	19
2.2.2	Properties	19
2.2.3	Individuals	19
2.2.4	Axioms and Constraints	20
2.3	Types of Ontologies	20
2.3.1	Top-level ontologies	20
2.3.2	Core ontologies	20
2.3.3	Domain ontologies	20
2.4	Ontology Languages and Standards	20
2.4.1	RDF – Resource Description Framework	20
2.4.2	RDFS – RDF Schema	21
2.4.3	OWL – Web Ontology Language	21
2.5	Conclusion	22
3	Knowledge Graph	23
3.1	What is Knowledge Graph	23
3.1.1	Definition 1	23
3.1.2	Definition 2	23
3.2	KG construction	24
3.2.1	Domain Definition and Scope Setting	24

3.2.2	Data Collection & Preprocessing	24
3.2.3	Entity & Relationship Extraction	25
3.2.4	Schema Design	25
3.3	Storing & Querying KGs	25
3.3.1	Graph Databases	25
3.3.2	Query Languages	26
3.4	Reasoning & Inferencing in KGs	26
3.4.1	Embedding based reasoning	26
3.4.2	Symbolic based reasoning	27
3.4.3	Neural network based reasoning	27
3.4.4	Mixed reasoning	27
3.5	KGs embedding	28
3.5.1	What is KGs embedding	28
3.5.2	Application KGE	28
3.5.2.1	Link prediction	28
3.5.2.2	Triple classification	29
3.5.2.3	Recommendation Systems	29
3.5.3	Models of KGs Embeddings	29
3.5.3.1	Translation-Based Models	30
3.5.3.2	Bilinear and Tensor factorization models	31
3.5.3.3	Neural Network-Based Models	33
3.5.3.4	Graph Neural Networks Based Models	35
4	Question Answering system	39
4.1	What is a Question Answering (Q/A) System?	39
4.2	Classification of Q/A System	39
4.2.1	domain based classification	39
4.2.2	Classification Based on Question Type	40
4.2.3	Data source	40
4.2.4	Language Paradigm	40
4.2.5	Approaches/Techniques Used	41
4.3	KG Based-QA Systems	41
4.3.1	Definition	41
4.3.2	Architecture of KG-QA Systems	41
4.3.3	Techniques in KG-QA	42
4.4	Conclusion	44
5	IoT	45
5.1	Introduction	45
5.2	IoT in Healthcare	45
5.3	Medical IoT Sensors and Data Acquisition	46
5.3.1	Healthcare Monitoring Devices	46
5.3.2	Data Collection Mechanism	46
5.3.3	Data Collection Frequency	47
5.4	Challenges and Considerations	47

III State of Art 48

6 State of Art	49
6.1 Introduction	49
6.2 Hybrid Approaches: KG + Machine Learning/Deep Learning	49
6.2.1 Diagnosis Method of Thyroid Disease Combining Knowledge Graph and Deep Learning	49
6.2.2 Biomedical knowledge graph embeddings for personalized medicine: Predicting disease-gene associations	52
6.2.3 Decision-Making System for the Diagnosis of Syndrome Based on Traditional Chinese Medicine Knowledge Graph	55
6.2.4 Discussion	57
6.2.5 Comparative analysis	59
6.3 Q/A Systems Based Only on Knowledge Graphs	61
6.3.1 A knowledge graph based question answering method for medical domain	61
6.3.2 Research on Medical Question Answering System Based on Knowledge Graph	63
6.3.3 Question Answering System Based on Knowledge Graph in Traditional Chinese Medicine Diagnosis and Treatment of Viral Hepatitis B	66
6.3.4 Research on a traditional Chinese medicine case-based question-answering system integrating large language models and knowledge graphs	68
6.3.5 A New Approach of Question Answering based on Knowledge Graph in Traditional Medicine	71
6.3.6 Building an intelligent diabetes Q&A system with knowledge graphs and large language models	74
6.3.7 Discussion	77
6.3.8 Comparative analysis	79
6.4 Conclusion	82

IV Q&A System Development and KGs Construction 83

7 KGs Construction and Representation Learning	84
7.1 Introduction	84
7.2 KG Construction	84
7.2.1 Data Source Selection and Acquisition	84
7.2.2 Ontology Schema Design and Modeling	85
7.2.3 Data Preprocessing and Entity Linking	85
7.2.4 Visualization and Exploration	87
7.3 Embedding and Representation Learning	87
7.3.1 Embedding Models	88
7.3.2 Experimental Setup and Training	88
7.3.3 Evaluation and Comparison	92
7.3.4 Discussion of Results	93
7.4 Conclusion	93
8 Q/A System Design and Workflow	94
8.1 Introduction	94
8.2 Natural Language Processing (NLP) Pipeline	94
8.2.1 Multilingual Question Handling	95

8.3	Question Understanding Pipeline	95
8.3.1	Entity Recognition & Extraction	95
8.3.2	Handling Out-of-Context Questions	96
8.3.3	Question Classification & Relationship Identification	96
8.4	Embedding-Based Retrieval & KG Completion	97
8.5	Answer Generatios	98
8.6	Conclusion	98
V	Platform Development	100
9	Requirements and System Specification	101
9.1	Introduction	101
9.2	Functional Requirements	101
9.2.1	Doctor	101
9.2.2	Hospital	102
9.2.3	Patient	102
9.3	Non-Functional Requirements	102
9.4	Use Cases	102
9.4.1	Use Case Descriptions	102
9.4.2	Use Case Diagram	103
9.4.2.1	Authentication & Authorization Module	105
9.4.2.2	Patient Monitoring Module	106
9.4.2.3	Doctor Tracking Module	107
9.5	Conclusion	107
10	System and Architecture Design	108
10.1	Introduction	108
10.2	Class Diagrams	108
10.3	Sequence Diagrams	109
10.3.1	Doctor Registration	109
10.3.2	Patient Registration	109
10.3.3	Doctor–Patient Communication	110
10.3.4	Doctor’s Use of the Q/A System for Diagnosis	111
10.3.5	Patient Vital Signs Recording and Monitoring	112
10.4	Security Considerations	113
10.4.1	Authentication and Authorization	113
10.4.2	User Input Validation and Sanitization	113
10.4.3	API Security	113
10.5	Conclusion	114
11	Implementation	115
11.1	Introduction	115
11.2	Hardware Overview	115
11.2.1	Core Hardware Platforms	115
11.2.2	Medical Sensor Integration and Specifications	117
11.3	Technologies and Tools Used	120
11.3.1	Development Environment	120
11.3.2	Frontend Technologies	121
11.3.3	Backend Technologies	122

11.3.4	Database and Storage	122
11.4	API & Models Integrations	123
11.4.1	APIs Used	123
11.4.1.1	Cloudinary API	123
11.4.1.2	OpenRouter	123
11.4.2	Pre-Trained Models Utilized	124
11.4.2.1	spaCy	124
11.4.2.2	SentenceTransformer	124
11.4.2.3	SapBERT	124
11.5	User Interface Preview	124
11.5.1	Web Application Screens	124
11.5.1.1	Registration Request Page	125
11.5.1.2	Account Activation Page	126
11.5.1.3	Login Page	127
11.5.1.4	Doctor Dashboard	127
11.5.1.5	EMR Filtering and Search Page	127
11.5.1.6	Q/A Consultation Page	129
11.5.1.7	Vital Sign Tracking Page	130
11.5.1.8	Doctor Communication Page	131
11.5.1.9	Hospital Dashboard	133
11.5.2	Mobile Application Screens	134
11.5.2.1	Patient Registration Screen	134
11.5.2.2	Patient Login Screen	135
11.5.2.3	Patient Home Screen	136
11.5.2.4	Inbox Screen	138
11.5.2.5	Schedule Appointment Screen	139
11.5.2.6	Contact Doctor Screen	140
11.5.2.7	Patient Profile Screen	141
11.6	Conclusion	142

VI Conclusion 143

12 Conclusion and perspectives 144

12.1	Conclusion	144
12.2	Perspectives	145

VII Bibliography 146

13 Bibliography 147

LIST OF FIGURES

3.1	knowledge graph example [35]	24
3.2	General illustration of the knowledge graph embedding process. [30]	28
3.3	Illustration of TransE [12]	30
3.4	Illustration of TransH [12]	31
3.5	Illustration of TransR [12]	31
3.6	A tensor model of a knowledge graph [6]	32
3.7	Illustration of RESCAL [12]	32
3.8	Illustration of DistMult [12]	33
3.9	Illustration of how GNNs work. [49]	37
4.1	Illustration of a KG-based QA pipeline [50]	41
5.1	IoT in Healthcare: Data flow from patients through sensors to the cloud and doctors. [64]	45
6.1	An illustration showing types of relationship [51]	50
6.2	An illustration showing the conceptual layer [15]	50
6.3	Mapping table example. [15]	51
6.4	The first experimental results [51]	52
6.5	Comparison of Disease Diagnosis Performance of Different Classifiers [51] . . .	52
6.6	Metagraph representing all the node and relationship types in the KG.. [52] .	54
6.7	Performance metrics for each method [52]	54
6.8	DSDS model architecture based on KG-EMR-TCM [53]	55
6.9	Performance comparison of different models. [53]	56
6.10	The medical knowledge graph in our approach. [54]	61
6.11	The pseudo-code of the intent recognition method. [54]	62
6.12	The performance evaluation of the proposed method and the other state-of-art methods for disease diagnosis. [54]	63
6.13	Query processing module diagram. [55]	65
6.14	Query processing module diagram. [55]	66
6.15	workflow of the question answering system. [57]	66
6.16	Partial display of hepatitis B knowledge graph. [57]	68
6.17	This diagram shows the workflow of the question-answering system. [56] . . .	69
6.18	This diagram shows the workflow of the workflow involved in the construction of a knowledge graph. [56]	70

6.19	Query Process and Response Flow of the Traditional Chinese Medicine Question-Answering System. [56]	70
6.20	Named entity recognition evaluation results. [56]	71
6.21	Architecture of QA system based on Knowledge Graph diagnosis. [58]	72
6.22	Diagram about Query Processing for finding the best answer for the question of disease diagnosis. [58]	73
6.23	Full System Architecture of the Diabetes Question-Answering System [59]	75
6.24	Diabetes Knowledge Graph: Entities and Relationships. [59]	76
6.25	Workflow diagram of the intelligent question-answering (QA) system for diabetes. [59]	76
8.1	The different steps employed for question handling.	94
8.2	The figure below presents the key components designed and implemented in our work	99
9.1	Global Use Case Diagram of the CardioX System	104
9.2	Authentication Use Case Diagram	105
9.3	Patient Monitoring Use Case Diagram	106
9.4	Doctor tracking Use Case Diagram	107
10.1	Global CClass Diagram of the CardioX System	108
10.2	Sequence diagram of the doctor registration process	109
10.3	Sequence diagram of the patient registration process	110
10.4	Sequence diagram of doctor-patient communication through notifications and appointment scheduling	111
10.5	Sequence diagram of the doctor's use of the Q/A system for diagnosis	112
10.6	Sequence diagram of patient vital signs recording and monitoring	113
11.1	MySignals HW	116
11.2	Arduino Uno R3	117
11.3	Exacon D-S18JK temperature sensor	118
11.4	blood pressure sensor	119
11.5	ECG sensor	119
11.6	SpO ₂ sensor	120
11.7	Doctor registration request screen. Used by private doctors to apply for access to the platform.	125
11.8	Hospital registration request screen. Used by hospitals to to apply for access to the platform.	126
11.9	Registration submission confirmation screen. Guides users through the next steps after submitting their registration request.	126
11.10	Screenshot of the Account Activation Page	127
11.11	Login page. Doctors and cardiology administrative staff use this screen to access the hospital platform.	127
11.12	Screenshot of the Patient List Screen displaying patient details, search, and filter options.	128
11.13	Screenshot of the Add New Patient Screen showing input fields for patient details	128
11.14	Screenshot of the Schedule Appointments Screen	129
11.16	Screenshot of the Q&A Screen where doctors can ask questions	129

11.15	Screenshot of the Patient Detail Screen showing detailed patient information, medical history, and contact details.	130
11.17	Screenshot of the Analyse Medical Images screen where doctors can upload and interpret patient scans for diagnostic support.	130
11.18	Screenshot of the Vital Sign Monitoring Screen showing real-time patient vitals	131
11.19	Screenshot of the Patient Detail Recording screen showing fields for medical history, symptoms, and treatment notes.	131
11.20	Screenshot of the Doctor Notification screen showing system alerts and patient updates for doctors.	132
11.21	Screenshot of the Send Notification to Patient screen with message input and delivery options.	132
11.22	Screenshot of the Patient Notification screen where patients receive updates and alerts from doctors or the hospital.	133
11.23	Screenshot of the Hospital Dashboard showing overall hospital statistics, active doctors, and patient activity.	133
11.24	Screenshot of the Doctor List screen displaying registered doctors with search and filter options.	134
11.25	Screenshot of the Add Doctor Screen with input fields for doctor details, specialization, and hospital assignment.	134
11.26	Patient registration screen. Used by patients already registered in the EMR system to activate their account and connect with their doctor through the application.	135
11.27	Patient login screen. Provides patients secure access to the platform to stay connected with their cardiology doctor.	136
11.28	Patient home screen showing recent vital signs, health indicators, and notifications.	137
11.29	Add vital signs screen. Allows manual entry of vital data such as heart rate, temperature, and blood pressure.	138
11.30	Inbox screen. Patients can view recent messages and appointments, interact with them, and access their full communication history with the doctor. . . .	139
11.31	Schedule Appointment screen. Patients can request an appointment by selecting a date, time, type, and specifying the agenda based on the doctor's availability.	140
11.32	Contact Doctor screen. Patients can select a message type, compose a message, and attach files or images to communicate with their doctor.	141
11.33	Patient Profile screen. Patients can view their medical information, including diagnoses, treatments, and personal details, as well as manage their account settings such as logging out or deactivating the account.	142

LIST OF TABLES

6.1	Comparative Overview of Medical KG-based Diagnostic Systems	60
6.2	Comparative Overview of Medical KG-based QA Systems	80
6.3	Comparative Overview of Medical KG-based QA Systems	81
7.1	Relationships between medical concepts and their definitions	86
7.2	Connected components detected in the KG	87
7.3	Mapping of original relations to their inverse relations.	88
7.4	Comparison of different models on link prediction task using Hits@ k , Mean Rank (MR), and Mean Reciprocal Rank (MRR).	92

Part I

Introduction

1.1 Context

Medical decision support systems are becoming increasingly important in modern health-care, as they help doctors analyze patient information, recommend treatments, and detect potential risks. This information is generated from multiple sources, including electronic medical records (EMRs), biomedical literature, clinical guidelines, and medical IoT sensors that continuously monitor vital signs such as heart rate, blood pressure, oxygen saturation, and body temperature. While these data sources enable real-time monitoring and support personalized care, their massive volume, heterogeneity, and unstructured nature make them difficult to manage effectively with traditional databases and information systems. To address this challenge, Knowledge Graphs have emerged as a powerful solution. They represent medical knowledge in a structured way, linking entities such as diseases, symptoms, drugs, and treatments into a graph format. This structure enables better integration of data, supports reasoning over medical knowledge, and facilitates knowledge discovery.

Among the various applications of KGs, one of the most significant and impactful is the development of Question Answering (QA) systems. These systems leverage the structured knowledge within KGs to allow users to ask questions in natural language and obtain precise, context-aware answers. In the medical field, such systems are particularly valuable, as they enable doctors and patients to quickly access critical information, support accurate diagnoses, and reduce the risk of medical errors.

Despite their potential, most existing QA systems rely on template matching or classification models to interpret user queries, which limits their flexibility and expressiveness. Other systems depend heavily on database query languages such as Cypher, which often fail when the knowledge graph is incomplete. To overcome these limitations, recent approaches have begun to explore Knowledge Graph Embedding (KGE) models, which capture hidden semantic relationships within the graph and enable more advanced reasoning.

This research aims to propose an approach for a medical QA system that processes natural language questions from doctors and delivers accurate, reliable answers grounded in a medical KG, thereby combining usability with advanced reasoning capabilities

1.2 Problem Statement

Despite their potential, KGs still suffer from major limitations: they are often incomplete, leaving out important medical facts, and their large-scale, highly connected structure makes querying and knowledge extraction computationally expensive and error-prone. These limitations reduce the reliability of KG-based Question Answering (KG-QA) systems, especially when precise medical information is required.

Another major challenge is flexibility: most KG-QA systems are designed to handle only a restricted set of question types or predefined templates. As a result, they struggle to interpret the wide variety of natural language queries that doctors may ask in practice.

In addition, many existing systems remain disconnected from real-time, patient-specific data such as EMRs and monitoring devices. This prevents them from delivering personalized, context-aware answers that are crucial for effective patient care.

Research Question:

How can we design a medical QA system that overcomes the incompleteness and complexity of Knowledge Graphs, integrates real-time patient-specific data, and achieves flexibility in handling diverse types of natural language questions—thereby providing accurate, personalized, and context-aware decision support for healthcare professionals?

1.3 Objective

The objective of this thesis is to develop a Knowledge Graph-based Question Answering system that supports medical decision-making by allowing doctors to query medical knowledge in natural language. The system will integrate with Electronic Medical Records and real-time patient monitoring to provide comprehensive access to patient data, while leveraging Knowledge Graph reasoning and embeddings to improve accuracy even with incomplete knowledge. Ultimately, the goal is to create an intuitive and reliable tool that enhances decision support and improves healthcare delivery.

This objective builds directly on the findings of the state-of-the-art review. Existing research has shown the potential of KGs in the medical domain for both question answering and enhanced reasoning through hybrid approaches that combine embeddings, deep learning, or machine learning models. By uniting these two directions—KG-based QA systems and hybrid KG reasoning—this thesis aims to fill the identified gaps and provide a clinically relevant solution. The proposed method leverages recent advances in deep learning, large language models, and embedding techniques to strengthen KG-based reasoning while ensuring practical clinical applicability.

1.4 Thesis outline

Chapter 01

The first chapter introduces the context of the research, presents the problem statement, and outlines the objectives of the work. It also provides an overview of the thesis structure.

Chapter 02

The second chapter provides the theoretical background on knowledge graphs, including their construction, storage, querying, reasoning techniques, and embeddings. It also reviews their applications and models.

Chapter 03

The third chapter focuses on question answering systems. It defines Q/A systems, describes their classifications, and presents knowledge graph-based Q/A systems with their architectures and techniques.

Chapter 04

The fourth chapter introduces ontologies, describing their definitions, components, types, and the languages and standards used to represent them.

Chapter 05

The fifth chapter explains the process of constructing knowledge graphs for the proposed system, including data acquisition, preprocessing, ontology schema design, and representation learning using embeddings. Evaluation and discussion of results are also provided.

Chapter 06

The sixth chapter describes the design and workflow of the Q/A system. It covers the NLP pipeline, question understanding, entity recognition, embedding-based retrieval, and answer generation.

Chapter 07

The seventh chapter presents the system requirements and specifications. It details the functional and non-functional requirements, use cases, and modules for doctors, hospitals, and patients.

Chapter 08

The eighth chapter outlines the system and architecture design. It includes class diagrams, sequence diagrams, and security considerations for authentication, input validation, and API protection.

Chapter 09

The ninth chapter is dedicated to the implementation of the system. It describes the technologies and tools used, IoT system integration, APIs, pre-trained models, and provides previews of both web and mobile user interfaces.

Chapter 10

The final chapter concludes the thesis by summarizing the main contributions and findings. It also presents perspectives and potential directions for future research.

Part II

Background

2.1 What is Ontology

The term ontology has been defined in various ways across disciplines. Below are some widely cited definitions from the literature:

2.1.1 Definition 1

In AI, an ontology represents an engineering artifact composed of a specific vocabulary used to describe a certain reality, along with a set of implicit assumptions about the meaning of the words in that vocabulary [5]

2.1.2 Definition 2

An ontology is a shared understanding of a domain of interest. [5] An ontology is a formal and explicit specification of a shared conceptualization of a domain. It defines the domain vocabulary including concepts, relationships, and constraints and provide rules for how these elements can be combined, enabling consistent interpretation, reasoning and interoperability

2.2 Components of Ontology

2.2.1 Classes

Abstract groups, sets, or collections of objects, that represent entities in the domain. For example : Person, Disease, or Medication.

2.2.2 Properties

Define relationships between classes or between individuals. In another word it defines the ways in which classes and individuals can be related. For example : hasSymptom, isTreatedBy

2.2.3 Individuals

Concrete example of a class , actual entities in the domain (e.g., "John Doe," "Diabetes Mellitus").

2.2.4 Axioms and Constraints

Logical statements that impose rules ,restrictions or conditions on classes and properties.(e.g., "Every patient must have at least one diagnosis").

2.3 Types of Ontologies

2.3.1 Top-level ontologies

Top ontologies, also known as philosophical or category ontologies, specify very high-level conceptual categories such as object, event, state, or process. They structure high-level knowledge based on philosophical reflections. These ontologies do not contain structures or instances but instead define the most general objects in a domain. Other concepts are derived from them through specialization, not instantiation. [7]

2.3.2 Core ontologies

Core ontologies are generic ontologies often used like a thesaurus. They define a reference vocabulary (e.g., in physics or mathematics), including hierarchies of terms and semantic relationships such as synonymy or composition. These ontologies provide structuring concepts and describe semantic relations between them. For example, in medicine, they might include concepts such as diagnosis, symptom, or anatomical structure, along with the relationships among them. [7]

2.3.3 Domain ontologies

Domain ontologies are descriptive ontologies tailored to a specific professional field. They define the concepts and relations relevant to that domain and provide a semantically rich specification. These ontologies are often built by specializing more general ontologies (such as core ontologies) and are aligned with how domain experts perceive and manipulate concepts in their field. [7]

2.4 Ontology Languages and Standards

Several formal languages have been standardized to build and communicate ontologies, especially within the Semantic Web stack.This section provides a detailed overview of the most commonly used ontology languages.

2.4.1 RDF – Resource Description Framework

is the most basic ontology language and forms the core data model of the Semantic Web. RDF is a data modeling language designed to represent information in the form of triples: subject, predicate, and object. Each triple describes a statement about a resource using URIs to identify both the entities and their relationships [8].RDF constructs a directed labeled graph where:

- Nodes represent resources.
- Edges represent properties (relations).

- Values can be literals or other resources

It is mainly used to describe web resources and annotate them using a shared vocabulary (ontology). RDF supports multiple serialization formats like Turtle, RDF/XML, and N-Triples.

2.4.2 RDFS – RDF Schema

RDFS extends RDF by providing a vocabulary description language that defines basic ontology structures such as classes, properties, and hierarchical relationships. RDFS also supports domain and range definitions, providing a lightweight way to model structured knowledge with limited reasoning capabilities [8]

RDFS introduces constructs like [8]

- **rdfs:Class**: Defines a class or type of resource.
- **rdfs:subClassOf**: Specifies that one class is a subclass of another, allowing inheritance of characteristics.
- **rdfs:subPropertyOf**: Declares that one property is a subproperty of another, enabling property inheritance.
- **rdfs:domain**: Indicates the class of the subject in a triple where the property is used. It constrains where a property can be applied.
- **rdfs:range**: Specifies the class of the object in a triple where the property is used.
- **rdf:type**: Used to state that a particular resource is an instance of a class.

RDFS enables basic reasoning and supports type systems similar to object-oriented languages but without methods. It allows one to define a vocabulary, not use one predefined.

2.4.3 OWL – Web Ontology Language

extends RDF and RDFS by adding formal semantics and richer expressiveness based on Description Logics. It introduces formal semantics and allows defining richer relationships between classes and properties [8]

Its key features include [8]:

- **Advanced constructs**: OWL supports constructs such as:
 - *Equivalence*
 - *Disjointness*
 - *Transitivity*
 - *Cardinality constraints*
- **Three standard OWL profiles**:
 - OWL Lite**: A simplified version, suitable for basic taxonomies.
 - OWL DL**: Based on Description Logics, offering a balance between expressiveness and decidability.
 - OWL Full**: The most expressive profile, but with no computational guarantees.

- **Logical constraints and reasoning:** OWL supports various axioms and restrictions, such as:
 - `owl:equivalentClass`
 - `owl:disjointWith`
 - `owl:someValuesFrom`
 - `owl:Restriction`

2.5 Conclusion

Ontologies provide a formal structure and semantic foundation for organizing domain knowledge. They define the core concepts, relationships, and constraints that govern how information is interpreted and used. When this structured vocabulary is combined with real-world data, it gives rise to a knowledge graph.

In this context, the ontology acts as the schema, while the knowledge graph represents the instantiated data. This combination enables intelligent reasoning, advanced querying, and seamless integration across heterogeneous information sources.

3.1 What is Knowledge Graph

3.1.1 Definition 1

The concept of KGs was first introduced by google in 2012, defined as a large scale of knowledge, composed of numerous entities and relationships between them. Recently KGs, as form of semantic networks, have been applied in many fields such as natural language processing, question answering system and intelligent recommendation system [1].

A multitude of extensive knowledge graphs have been developed, including: YAGO, Freebase, DBpedia and Wikidata [2].

3.1.2 Definition 2

A KGs is a structured representation of real-world data that encompasses entities, concepts, and the relationships between them, stored in a graph database and visualized as a graph structure, where nodes represent entities and edges represent relationships between entities, which help both human and machine to understand the semantic meaning of the data. As shown in the figure below:

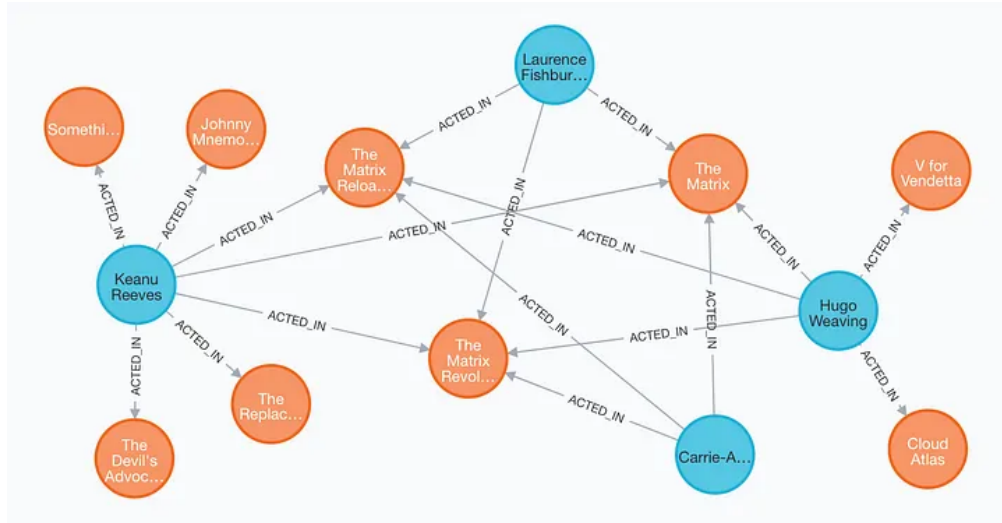


Figure 3.1: knowledge graph example [35]

The KG represent actors and movies, where entities like *Keanu Reeves* or *The Matrix* are connected by semantic relations such as `ACTED_IN`

3.2 KG construction

KGs construction is the process of transforming raw data into structure ,interconnected representation of knowledge.This process involves several steps which are:

3.2.1 Domain Definition and Scope Setting

Before building the KG, it is important to first determine its goal and define the knowledge domain. The goal is to identify the KG's objectives and use cases, as well as to specify the relevant entities and relationships. [40]

- **Set Clear Objectives:** Identify the primary use case—e.g., improving search, enabling intelligent chatbots, enhancing analytics, or connecting siloed data.
- **Align with Strategic Needs:** Ensure the knowledge graph supports specific organizational needs such as decision-making, risk monitoring, or data integration.
- **Determine the Knowledge Domain:** Choose the domain (e.g., healthcare, e-commerce, legal) and define the key entities, relationships, and properties relevant to that context.

3.2.2 Data Collection & Preprocessing

Data Collection involves gathering data from various sources,both structured (eg.,database ,API) and unstructured (eg.,web scraping ,text, document ,social media ...) .

Preprocessing is essential to ensure the quality and consistency of the data. It includes tasks such as cleaning duplicates, removing inconsistencies, standardizing formats, and verifying data accuracy. A well-curated dataset forms the backbone of a reliable and meaningful knowledge graph. [40]

3.2.3 Entity & Relationship Extraction

This step involves identifying and categorizing entities (such as people, organizations, and locations) and determining the relationships between them based on the collected data. This section outlines both traditional and modern techniques used for this purpose.

- **Named Entity Recognition (NER)** Named Entity Recognition (NER) is a subtask of natural language processing that identifies and classifies key entities—such as people, places, organizations, dates, and more—within unstructured text. By converting text into structured information, NER supports tasks like text summarization, knowledge graph construction, and question answering [41]. In KG construction it's used to extract entities from text sources, forming the foundational nodes of the graph and enabling accurate representation of real-world concepts.
- **Rule-based relationship extraction** Rule-Based Relationship Extraction uses pre-defined linguistic patterns or syntactic rules to identify relationships between entities in text. It is effective in domains with consistent language but can struggle with complex or ambiguous data. Despite its limitations, it remains useful, especially when combined with machine learning techniques. [40]
- **Large Language Models (LLMs)** Large Language Models (LLMs) are deep learning models trained on massive text corpora to understand and generate human language. They capture semantic, syntactic, and contextual patterns across various domains [42]. In the context of knowledge graph construction, LLMs automate the extraction of entities and relationships from unstructured text. By leveraging their language understanding capabilities, LLMs can identify structured facts (e.g., subject–predicate–object triples) with high accuracy and minimal manual effort. This significantly accelerates the KG building process and allows for efficient analysis of large-scale textual data [40]
- **Machine learning-based graph embeddings** Machine learning-based graph embedding techniques map entities and relationships into a continuous vector space, preserving semantic structure. These embeddings facilitate efficient querying and analysis and are particularly effective in applications such as recommendation systems and anomaly detection, enhancing both scalability and performance in graph-based systems. [40]

3.2.4 Schema Design

Once entities and relationships have been extracted, the next step is to define the conceptual structure of the knowledge graph. This involves designing an ontology or schema. This structural foundation enhances the graph's semantic richness and ensures its usability across various applications such as search, recommendations, and intelligent agents.

3.3 Storing & Querying KGs

3.3.1 Graph Databases

This section introduces the most widely used knowledge graph storage systems

- **Neo4j:** one of the most popular and widely used open-source graph databases, with over 12,000 stars on github. as it implement a graph model right to the storage level It provides runtime failover, cluster support, and ACID transactions. It utilize Cypher ,a

a graph-optimized query language with a syntax similar to SQL, it also provides a driver for languages like JavaScript, Python, and .NET [3]

- **ArangoDB:** high performance, scalable, open source graph database, with over 13,000 GitHub stars. With only one query language, it mixes an integrated search engine with native graphs, JSON support, and multiple data access patterns [3]
- **Dgraph:** Dgraph is a native graph database that supports native GraphQL and has over 19,000 stars on GitHub. It is a fast, scalable, distributed, and highly available open-source database. It offers built-in support for GraphQL, enabling efficient graph navigation and information retrieval [3]

3.3.2 Query Languages

Query languages are essential for interacting with knowledge graphs stored in graph databases, enabling users to retrieve, manipulate, and analyze data. This section provides an overview of widely used query languages, each associated with a specific database system

- **Cypher:** Cypher is a declarative query language designed for graph databases, primarily used with Neo4j. Known for its intuitive and SQL-like syntax, Cypher enables users to retrieve, manipulate, and analyze graph data through pattern-based matching. It supports visual and expressive querying of nodes and relationships. Cypher also integrates with Object Graph Mappers (OGMs) such as Neo4j-OGM for Java and GQLAlchemy for Python, allowing seamless mapping between domain objects and graph structures. [36]
- **SPARQL:** SPARQL (SPARQL Protocol and RDF Query Language) is a standardized query language used to retrieve and manipulate data stored in RDF (Resource Description Framework) format. It enables querying of triple-based data using four main query forms: SELECT, ASK, CONSTRUCT, and DESCRIBE. Each query operates over data represented as triples — subject, predicate, and object — making SPARQL essential for semantic web and linked data applications [36]

3.4 Reasoning & Inferencing in KGs

KG reasoning refers to the process of inferring new knowledge — particularly new entities and relationships — from existing data in a KG. Its goal is to mimic human-like reasoning, enabling machines to make intelligent decisions

KGR is divided into four main approaches: embedding-based reasoning, symbolic-based reasoning, neural-network-based reasoning, and mixed reasoning. [38] In this section we will provide a detailed overview of each one

3.4.1 Embedding based reasoning

Embedding-based reasoning refers to the process of using KGEs models to transform entities and relations into a low-dimensional vector space. This transformation reduces the complexity of symbolic representations and enables the discovery of implicit associations between entities and relations. Once embedded as vectors, reasoning tasks such as link prediction, node classification, and knowledge graph completion can be performed through mathematical operations, making them faster and more computationally efficient. [38]

3.4.2 Symbolic based reasoning

Symbolic-based reasoning refers to methods that use explicit logical rules, primarily grounded in first-order predicate logic or description logic, to infer new facts (e.g., new relationships) in a knowledge graph. [38]

The key techniques used are:

- Inductive Logic Programming (ILP)
- AMIE (Association Rule Mining under Incomplete Evidence)
- ProPPR (Programming with Personalized PageRank)
- TensorLog
- SDType & SDValidate
- Triplet Quality Assessment

A major strength of symbolic-based reasoning lies in its similarity to human logical thinking and its high level of explainability. However, these methods tend to be computationally expensive, particularly when applied to large-scale knowledge graphs.

3.4.3 Neural network based reasoning

Neural network based reasoning uses neural architectures to infer relationships between entities by learning patterns from data, offering strong expressiveness and performance in tasks like link prediction and entity classification.. [38] Key Techniques & Models used are:

- NTN (Neural Tensor Network)
- R-GCN (Relational Graph Convolutional Network)
- IRN (Implicit ReasonNets)

This type of reasoning is highly expressive and well-suited to a variety of reasoning tasks. It also supports multi-step inference and enables common-sense reasoning capabilities.

3.4.4 Mixed reasoning

Mixed reasoning combines the strengths of symbolic-based, embedding-based, and neural network-based reasoning to provide complementary advantages. This hybrid approach improves both the accuracy and interpretability of inference in knowledge graphs, making it a prominent trend in the field. [38]

One notable example is Neural Logic Programming (Neural-LP) [39], a differentiable framework that integrates relational representation learning, rule learning, and recurrent neural networks. It leverages LSTM units to model implicit variables and attention mechanisms across multi-step reasoning paths.

3.5 KGs embedding

3.5.1 What is KGs embedding

KGs embeddings are representation learning techniques that transform the structured information in a KGs into low-dimensional vector spaces. These embeddings encode both entities and relations as dense vectors, while preserving the semantic relationships among them. This allows for efficient computation of entity similarity and other inference tasks within the graph [10] .

To illustrate the embedding process, consider a knowledge graph triple (h, r, t) , where h and t denote entities and r represents the relation linking them. The goal of KG embedding is to project these components into a continuous vector space while preserving their semantic and structural meaning, as depicted in the figure below.

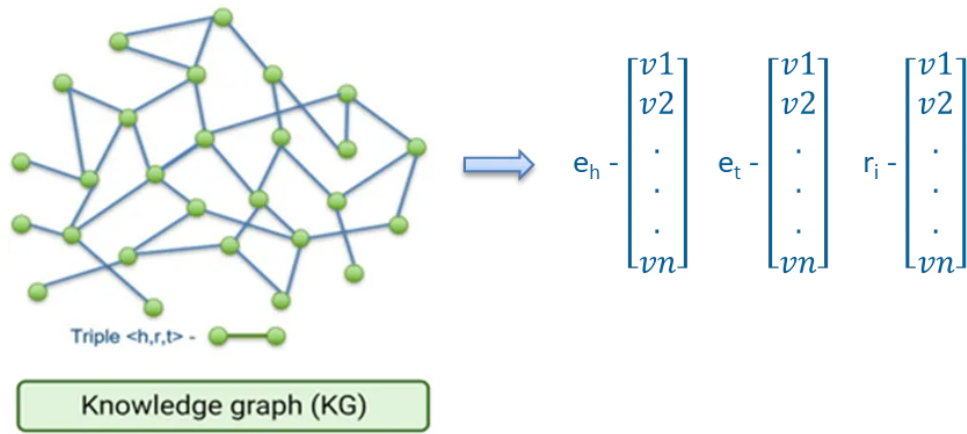


Figure 3.2: General illustration of the knowledge graph embedding process. [30]

3.5.2 Application KGE

KGE has a wide range of applications across various domains. This section outlines some of the most commonly used applications, emphasizing how embedding representations enhance reasoning, prediction, and information retrieval tasks.

3.5.2.1 Link prediction

It involves identifying missing entities in a triple to complete a valid fact. Given an incomplete triple, the task is to predict the missing head or tail entity. This is commonly formulated in two ways:

- Predicting the head entity h given the relation and tail: $(?, r, t)$
- Predicting the tail entity t given the head and relation: $(h, r, ?)$

During prediction the model assign a score to each candidate triple by filling the missing entity. These candidate will be rank based on their score, the entity with heights ranking will be selected as the predicted answer

During the training the KGE model learn vector representation of entities and relationship, by

processing know triple and generate false tripl as negative samples.Using a scoring function the modle assigns higher scores to valid tripl and lower score to invalide triple. Through iterative optimization, typically with gradient descent and a suitable loss function, the embeddings are adjusted to better distinguish true facts from false ones. This process continues until the model converges, resulting in embeddings that capture the graph’s semantic and structural patterns, enabling accurate link prediction. [32]

3.5.2.2 Triple classification

triple classification is a binary classification task , that’s aims to determine whether a given triple (h, r,t) is valid or not [6] .It’s the ability of a model to differentiate between true fact from false ones, within a KG by prediction weather a triple should exist or not

For example, the statement ”New Delhi is the capital of India” can be represented as the triple (New Delhi, isCapitalOf, India). A scoring function is used to assign a confidence score to the triple, similar to methods used in link prediction. If the score exceeds a certain threshold, the triple is classified as correct; otherwise, it is considered incorrect. [31]

3.5.2.3 Recommendation Systems

Recommender systems help users navigate large sets of options by ranking and suggesting relevant items based on user behavior and item similarity. This approach, known as collaborative filtering (CF), often suffers from data sparsity and the cold start problem—situations where limited or no user-item interactions are available . [33]

KGE-based recommendation systems typically include two components: a graph embedding module that learns entity representations, and a recommendation module that predicts user-item interactions. Depending on how these modules interact, three learning strategies are commonly used [34]

- **Two-stage learning:** Embeddings are learned first using a knowledge graph embedding method, and then used as input features for the recommendation model.
- **Joint learning:** The graph embedding and recommendation modules are trained simultaneously in an end-to-end manner, allowing the recommendation task to directly influence the embedding process.
- **Multi-task learning:** The recommendation module is trained alongside an auxiliary KG-related task, such as link prediction, leveraging structural similarities between the recommendation graph and the knowledge graph.

KGEs techniques, beyond their core use cases, have shown potential in a variety of application domains. Notably, they are increasingly applied in **question answering (QA) systems**, where both the question and elements of the knowledge graph are embedded into a shared vector space to enable accurate answer retrieval [6]

3.5.3 Models of KGs Embeddings

This section introduces three categories of embedding models: Translation-Based Models, Tensor Factorization Models, and Neural Network-Based Models

3.5.3.1 Translation-Based Models

Translation-Based Models is a class of knowledge graph embedding technique, that represents relationship between entities as translation in continuous vector space. Inspired by the translation invariance property observed in word embeddings model word2vec [6]

- **TransE:** TransE is a translation-based embedding model for KGs that represents relations as linear translations in the embedding space. The main idea is that for a valid triple (s, r, o) , the embedding of the subject plus the embedding of the relation should be close to the embedding of the object [12]. For a triplet (h, r, t) , it enforces:

$$h + r \approx t$$

- **Scoring function:**

$$f(h, r, t) = \|\mathbf{h} + \mathbf{r} - \mathbf{t}\|_{1/2}$$

- **Strengths:** Simple, computationally efficient, and effective for one-to-one relations.
- **Limitation:** Struggles with complex relations such as 1-to-N, N-to-1, and N-to-N [6].

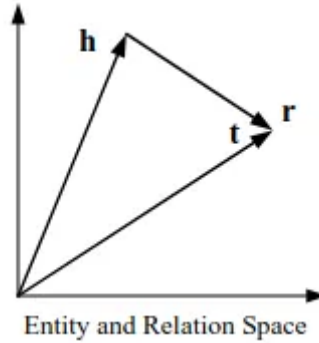


Figure 3.3: Illustration of TransE [12]

- **TransH:** TransH improves upon TransE by allowing entities to have different embedding vectors per relation. It introduces a relation-specific hyperplane [6], meaning that the same entity can have different semantics in different relational contexts. Entities are projected onto the hyperplane via:

$$\mathbf{h}_{\perp} = \mathbf{h} - (\mathbf{w}_r^{\top} \mathbf{h}) \mathbf{w}_r, \quad \mathbf{t}_{\perp} = \mathbf{t} - (\mathbf{w}_r^{\top} \mathbf{t}) \mathbf{w}_r$$

- **Scoring function:**

$$f(h, r, t) = \|\mathbf{h}_{\perp} + \mathbf{r} - \mathbf{t}_{\perp}\|$$

- **Advantages:** Handles complex relation types (1-to-N, N-to-1, N-to-N) and is scalable without significantly increasing model complexity.

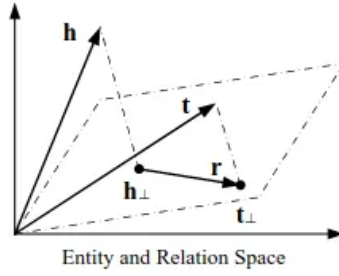


Figure 3.4: Illustration of TransH [12]

- **TransR:** TransR extends the concept of relation-specific hyperplanes from TransH by introducing relation-specific spaces. Entities (h, t) are embedded in an entity space $\mathbb{R}^d$, and each relation r is associated with its own space $\mathbb{R}^k$, where relations are modeled as translation vectors. To evaluate a triple, entities are projected to the relation-specific space using a projection matrix [12].
 - **Limitation:** TransR uses a projection matrix, which significantly increases computational resource requirements.

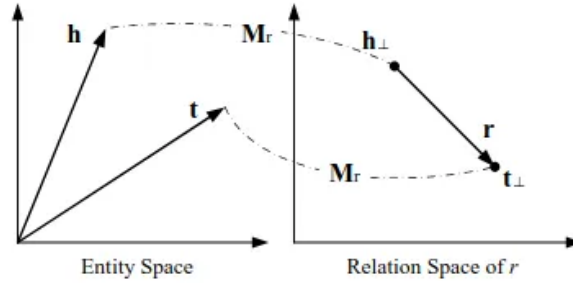


Figure 3.5: Illustration of TransR [12]

3.5.3.2 Bilinear and Tensor factorization models

Tensor factorization models are a class of methods for knowledge graph representation learning. They begin by transforming KG triples into a 3D binary tensor $\mathcal{X} \in \mathbb{R}^{n \times n \times m}$, where n is the number of entities and m the number of relations. Each slice $\mathcal{X}_k$ corresponds to a relation R_k , with $\mathcal{X}_{ijk} = 1$ indicating that the triple (e_i, r_k, e_j) exists, and $\mathcal{X}_{ijk} = 0$ otherwise. This tensor is then factorized into embedding matrices for entities and relations, producing low-dimensional representations that capture the semantic structure of the knowledge graph. [6]

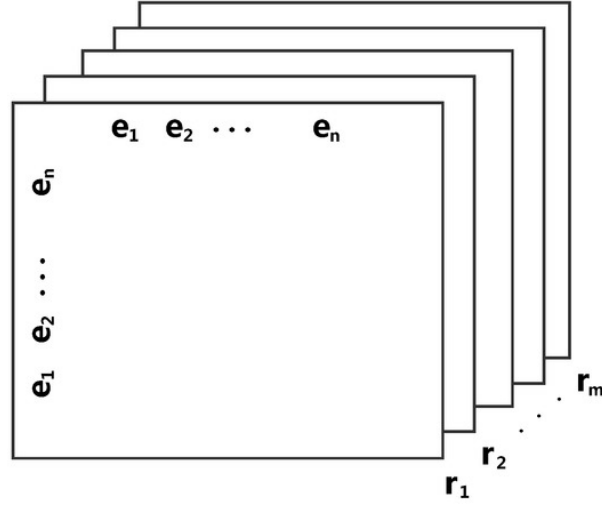


Figure 3.6: A tensor model of a knowledge graph [6]

- **RESCAL:** RESCAL is a tensor factorization-based embedding model that associates each entity with a vector to capture its latent semantics and represents each relation as a full matrix to model interactions between latent factors [12].

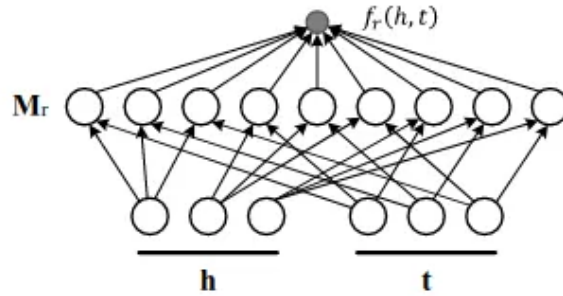


Figure 3.7: Illustration of RESCAL [12]

- **Scoring function:**

$$f(h, r, t) = \mathbf{h}^\top \mathbf{M}_r \mathbf{t}$$

where $\mathbf{h}, \mathbf{t} \in \mathbb{R}^d$ are the entity embeddings and $\mathbf{M}_r \in \mathbb{R}^{d \times d}$ is the relation matrix.

- **Strengths:** Effectively captures complex and dense relationships by modeling full interactions between entity dimensions.
- **Limitations:** Requires $O(d^2)$ parameters per relation, which is computationally expensive and less scalable for large KGs [12].
- **DistMult:** DistMult simplifies RESCAL by restricting each relation matrix $\mathbf{M}_r$ to a diagonal matrix. For a given relation r , it introduces a vector embedding $\mathbf{r} \in \mathbb{R}^d$ such that $\mathbf{M}_r = \text{diag}(\mathbf{r})$ [12].

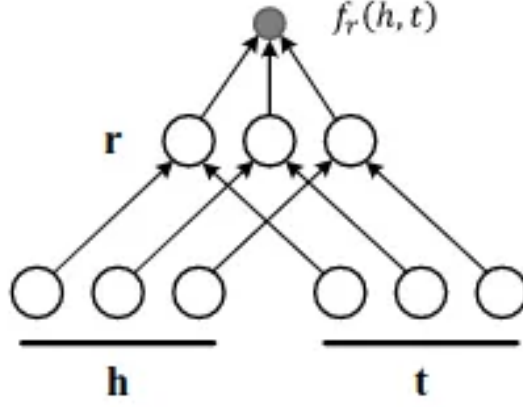


Figure 3.8: Illustration of DistMult [12]

– **Scoring function:**

$$f(h, r, t) = \langle \mathbf{h}, \mathbf{r}, \mathbf{t} \rangle = \sum_{i=1}^d h_i \cdot r_i \cdot t_i$$

- **Strengths:** Computationally simple and efficient due to the use of diagonal relation matrices [12].
- **Limitations:** Can only model symmetric relations, limiting its ability to handle asymmetric or more complex relational structures [12].

- **Complex:** ComplEx extends DistMult by using complex-valued embeddings to model asymmetric relations [12]. Entity and relation embeddings $\mathbf{h}, \mathbf{r}, \mathbf{t}$ are defined in the complex space $\mathbb{C}^d$.

– **Scoring function:**

$$f(h, r, t) = \text{Re}(\langle \mathbf{h}, \mathbf{r}, \bar{\mathbf{t}} \rangle) = \text{Re} \left(\sum_{i=1}^d h_i \cdot r_i \cdot \bar{t}_i \right)$$

– **Notations:**

- * $\text{Re}(\cdot)$: Real part of a complex number.
- * $\bar{t}_i$: Complex conjugate of t_i .
- * $\langle \cdot, \cdot, \cdot \rangle$: Tri-linear dot product over complex vectors.
- **Strengths:** Effectively models both symmetric and asymmetric relations using complex-valued embeddings.
- **Limitations:** Requires handling of complex arithmetic, which may reduce interpretability and increase implementation complexity.

3.5.3.3 Neural Network-Based Models

Neural Network-Based Models are another class of KGE techniques, that leverage deep learning to capture complex, non-linear relationships between entities and relations. These models enable the representation and generalization of complex data, especially for modeling complicated nonlinear relationships, where the output doesn't change in a simple straight-line fashion with the input. [6]

- **ConvE:** ConvE is a multi-layer convolutional neural network designed for link prediction in knowledge graphs. Unlike traditional fully connected layers, CNNs are more parameter-efficient and capable of capturing complex, non-linear relationships. ConvE enhances modeling interactions by establishing local connections between entities and relations across multiple dimensions [12].

– **Process:**

- * The subject entity vector and the relation vector are reshaped into 2D matrices.
- * The reshaped matrices are concatenated using a ‘concat’ operation.
- * The concatenated matrix undergoes a 2D convolution.
- * The convolution output is passed through a non-linear transformation and then compared with the object entity vector to compute a matching score.

– **Scoring function:**

$$f(s, r, o) = (\sigma(\text{vec}(\text{conv}(\text{concat}(\text{Reshape}(e_s), \text{Reshape}(e_r)))) W) \cdot e_o)$$

– **Notations:**

- * e_s, e_r, e_o : embeddings of the subject, relation, and object.
- * $\text{Reshape}(\cdot)$: converts 1D vectors to 2D matrices.
- * $\text{concat}(\cdot, \cdot)$: concatenates reshaped inputs.
- * $\text{conv}(\cdot)$: 2D convolution operation.
- * $\text{vec}(\cdot)$: flattens the convolution result.
- * W : trainable projection matrix.
- * σ : activation function (e.g., ReLU).
- * $\cdot$: dot product.

- **ConvKB:** ConvKB is a CNN-based embedding model for knowledge base completion [26], [27]. Each triple (h, r, t) is represented as a $K \times 3$ matrix, where K is the embedding dimension. The three columns correspond to the embeddings of the head, relation, and tail.

– **Process:**

- * The triple matrix is passed through 1D convolutional filters of shape 1×3 .
- * These filters slide over the rows to extract relational patterns.
- * Output feature maps are concatenated into a single vector.
- * A dot product with a shared weight vector produces the final score.

– **Scoring function:**

$$f(h, r, t) = \text{concat}(g([\mathbf{v}_h, \mathbf{v}_r, \mathbf{v}_t] * \Omega)) \cdot \mathbf{w}$$

– **Notations:**

- * Ω : set of convolution filters.
- * $*$: convolution operator.
- * g : non-linear activation (e.g., ReLU).
- * concat : flattens and joins output maps.

* **w**: trainable weight vector.

* $\cdot$: dot product.

- **Capsule Networks (CapsNet)**: CapsNet is a deep neural network model designed to preserve spatial and hierarchical relationships better than traditional CNNs [28], [29]. Instead of scalar outputs, it uses vector outputs called capsules. The **length** of the capsule indicates the presence probability, and the **orientation** encodes instantiation parameters like pose, rotation, or scale.

– **Architecture:**

* **Encoder:**

1. **Conv1**: 256 filters (9×9, stride 1, ReLU); extracts low-level features.
2. **PrimaryCaps**: 32 channels of 8D capsules (9×9, stride 2); captures part-level info.
3. **DigitCaps**: 16D capsule per class; uses dynamic routing and affine transforms to build class-specific representations.

* **Decoder:**

- Fully connected network to reconstruct the input image using only the active capsule.
- Adds reconstruction loss to improve generalization.
- Layers:
 - 512 neurons
 - 1024 neurons
 - 784 neurons (reshaped to 28×28 pixels for MNIST)

– **Loss Function (Margin Loss):**

$$L_k = T_k \cdot \max(0, m^+ - \|\mathbf{v}_k\|)^2 + \lambda \cdot (1 - T_k) \cdot \max(0, \|\mathbf{v}_k\| - m^-)^2$$

– **Notations:**

- * $T_k = 1$ if class k is present, else 0
- * $m^+ = 0.9$: margin for positive examples
- * $m^- = 0.1$: margin for negative examples
- * λ : down-weighting factor for absent classes
- * $\|\mathbf{v}_k\|$: length of the capsule output vector

3.5.3.4 Graph Neural Networks Based Models

GNNs are a class of neural networks, designed to operate graph data structure. Unlike conventional neural networks that work on fixed-size input as images or sequences, GNN can process graphs with varying sizes and complex structures. They achieve this by repeatedly aggregating and transforming information from each node's neighboring nodes, enabling them to capture both local and global graph structures [20]

key concept of GNNs

1. **Message passing**: In GNNs, each node updates its representation by exchanging information with their neighbors, through a process called message passing. [21]

2. **Node embedding:**The outcome of message passing is an embedding (vector representation) for each node. These embeddings capture complex structural patterns and dependencies in the graph [21]. They are low-dimensional vector representations of nodes that encode their features and their relationships within the graph
3. **Graph-Level Representations:**while node embedding focuses on individual nodes, graph-level captures information about the entire graph. These are useful for tasks such as graph classification, or overall graph analysis. [21]

How do GNNs work

1. **Initialization:** In this step each node in the graph is associated with a feature vector, this vector is used to represent node features. Features represent node characteristics or properties
2. **Message passing:** Across multiple iterations or layers, each node exchanges information with their neighbors and aggregates information from them to update its own representation [21]
3. **Aggregation:** when the transformed messages from neighboring nodes are obtained, they must be aggregated to update the target node representation. For a node v , $N(v)$ is the set of neighbors of v , the features or representations of these neighbors are combined using an aggregation function, including **sum**, **mean**, **max**, or **min** [22] If we denote the aggregation function as G , the aggregated message for a node i can be written as:

$$\bar{m}_i = G(\{W_j \cdot x_j : j \in \mathcal{N}_i\})$$

Here, $\mathcal{N}_i$ represents the set of neighboring nodes of node i , x_j is the feature vector of neighbor j , and W_j is a learnable weight matrix.

4. **Update:** After aggregating messages from a node's neighbors, each node updates its own feature vector. The node features are updated by combining these messages with its own features. This ensures the node representation reflects both local and neighbor information. There are two common update methods
 - (a) **Addition:** One common approach is to add the aggregated message to the node's original feature vector. The result is then passed through a non-linear transformation to produce the updated representation. [22]

$$h_i^{(l+1)} = \sigma \left(H \left(h_i^{(l)} + \bar{m}_i \right) \right)$$

Where:

- $h_i^{(l)}$: current representation of node i at layer l
- $\bar{m}_i$: aggregated message from neighbors
- H : a neural network (e.g., MLP or linear transformation)
- σ : activation function (e.g., ReLU, ELU, Tanh)

- (b) **Concatenation:** Alternatively, the node's features and the aggregated messages can be concatenated and then transformed: [22]

$$h_i^{(l+1)} = \sigma \left(H \left([h_i^{(l)} \parallel \bar{m}_i] \right) \right)$$

Where:

- $[\cdot \parallel \cdot]$: denotes vector concatenation

The following figure provides a summary of the steps described above:

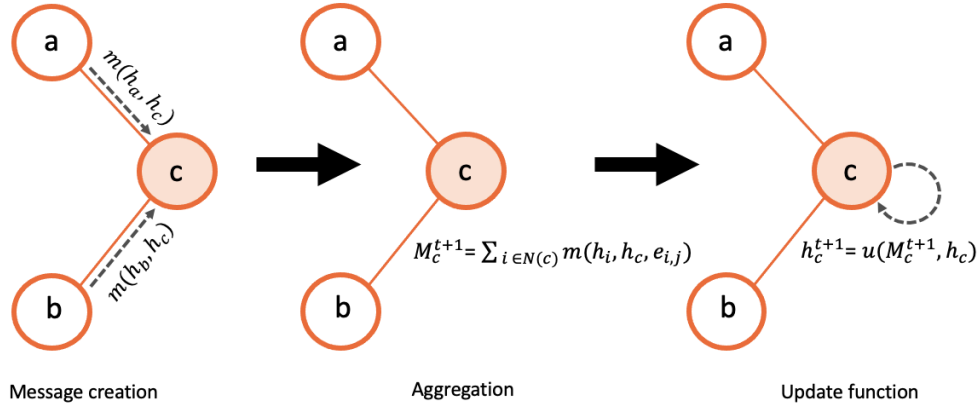


Figure 3.9: Illustration of how GNNs work. [49]

These steps are repeated across multiple layers, enabling each node to progressively gather and refine information from a broader neighborhood within the graph.

GNN types

- **Graph Convolutional Networks (GCNs):** GCNs are a type of neural network designed to operate on graph-structured data. They consist of stacked graph convolutional layers. Each layer takes as input the node feature vectors from the previous layer (or the input features in the first layer) and produces updated node representations by aggregating information from neighboring nodes using a normalized adjacency matrix [23].

– Layer Update Equation:

$$H^{(l+1)} = \sigma \left(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} H^{(l)} W^{(l)} \right)$$

– Where:

- * $\tilde{A} = A + I$: Adjacency matrix with self-loops
- * $\tilde{D}$: Degree matrix of $\tilde{A}$
- * $W^{(l)}$: Trainable weight matrix at layer l
- * σ : Activation function (e.g., ReLU)

- **Graph Attention Networks (GATs):** GATs enhance GCNs by incorporating an attention mechanism that assigns different importance levels to neighboring nodes during message passing. This allows the model to focus more on influential nodes, improving performance on tasks like social network analysis and citation networks [22].

– Layer Update Equation:

$$h_i^{(l+1)} = \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(l)} W^{(l)} h_j^{(l)} \right)$$

– **Where:**

- * $h_i^{(l)}$: Feature vector of node i at layer l
- * $W^{(l)}$: Shared weight matrix
- * $\alpha_{ij}^{(l)}$: Attention coefficient for edge (i, j)
- * σ : Activation function

– **Attention Coefficients:**

$$\alpha_{ij} = \frac{\exp\left(\text{LeakyReLU}\left(a^\top [Wh_i \parallel Wh_j]\right)\right)}{\sum_{k \in \mathcal{N}(i)} \exp\left(\text{LeakyReLU}\left(a^\top [Wh_i \parallel Wh_k]\right)\right)}$$

Where a is a learnable attention vector and $[\cdot \parallel \cdot]$ denotes vector concatenation.

- **Relational Graph Convolutional Networks (R-GCNs):** R-GCNs are designed for graphs with highly multi-relational data, such as knowledge graphs. They extend GCNs by introducing relation-specific transformations, where each relation type has its own weight matrix [24]. To avoid overfitting from too many parameters, techniques like basis decomposition or block-diagonal parameter sharing are applied [25].

– **Applications:** R-GCNs are particularly effective in KG tasks such as entity classification, link prediction, and knowledge graph completion.

– **Layer Update Equation:**

$$h_i^{(l+1)} = \sigma \left(\sum_{r \in \mathcal{R}} \sum_{j \in \mathcal{N}_i^r} \frac{1}{c_{i,r}} W_r^{(l)} h_j^{(l)} + W_0^{(l)} h_i^{(l)} \right)$$

– **Where:**

- * $h_i^{(l)}$: Hidden state of node i at layer l
- * $\mathcal{R}$: Set of relation types
- * $\mathcal{N}_i^r$: Neighbors of node i under relation r
- * $W_r^{(l)}$: Relation-specific transformation matrix
- * $W_0^{(l)}$: Self-loop transformation matrix
- * $c_{i,r}$: Normalization constant
- * σ : Activation function (e.g., ReLU)

4.1 What is a Question Answering (Q/A) System?

Q/A system is a branch of AI within natural language processing NLP and information retrieval, its focus on developing system capable of understating question possessed by human in natural language and generating accurate, human-readable responses. These system analyse the question to determine the context, identify and extract relevant information from vast data source, and presented back to the user in a concise and readable way [13]

4.2 Classification of Q/A System

Q/A systems can be categorized into different types, based on their Domain, Types of Questions, Data Source, answers generated by QA system, language paradigm and approaches [14]. This section provide a detail about each classification :

4.2.1 domain based classification

Q/A system can be categorized into two type based on the scope of knowledge they cover : open domain and close domain

- **Open-Domain QA Systems:** are designed to respond on a wide range of topics, without being limited to a specific subject area or domain, it provides answers for any question on any topic without restrictions.
some open domain QA systems : START system, Google Knowledge Graph, Answers.com, YodaQA. This system are based on large, general-purpose repositories of queries. They leverage general structured texts and world knowledge to generate answers. the answer in this type of system may vary (often not very high), as they rely on large-scale generic data. [14]
- **Closed-Domain QA Systems:** are specialized to operate within a specific domain, filed or subject area such as medicine, law, or engineering. They train on domain-specific knowledge and terminologies, to provide precise and contextually relevant answers within their area of expertise [13]
These systems require linguistic capabilities to understand natural language and deliver

accurate responses. By Limiting the domain of queries and reducing the size of database their development becomes more efficient , leading to improved system accuracy and higher answer quality. [14]

4.2.2 Classification Based on Question Type

QA systems are influenced by the nature of the questions they process: [14]

- **Factoid Questions:** Require specific factual answers (e.g., "Who is the president of Nigeria?").
- **List Questions:** Expect multiple factual answers (e.g., "Which countries are in the EU?").
- **Definition Questions:** Ask for explanations or definitions (e.g., "What is a noun?").
- **Hypothetical Questions:** Pose imagined scenarios (e.g., "What would happen if...?").
- **Causal Questions:** Ask for reasons or explanations (e.g., "Why does it rain?").
- **Confirmation Questions:** Expect Yes/No answers (e.g., "Is Paris the capital of France?").

4.2.3 Data source

Data source refers to the origin or repository of information from which the system retrieves answers to user questions.It's can be classified as [14]:

- **IR-Based Systems:** Extracts answers from large text corpora or the web using search and retrieval methods.
- **Knowledge-Based Systems:**Answers questions by mapping them to queries over structured knowledge bases (e.g., ontologies, databases).
- **Hybrid Systems:**Combine both structured (knowledge bases) and unstructured (text) sources for answer retrieval.

4.2.4 Language Paradigm

Language Paradigm refers to the linguistic scope and capabilities of a QA system, specifically regarding the languages it can process for questions and provide answers in.It's can be classified as [14]:

1. **Monolingual QA Systems:** Same language for query, data, and answer.
2. **Cross-Lingual QA Systems:** Accept queries in one language and retrieve answers from sources in another
3. **Multilingual QA Systems:** Support for multiple languages with query and answer in the same language.

4.2.5 Approaches/Techniques Used

efers to the underlying methods, strategies, or algorithms that a QA system employs to process questions and find or generate answers t's can be classified as [14]:

- **Linguistic Approach:** Uses linguistic analysis (syntax, semantics, parsing, NLP techniques) to process queries and retrieve answers.
- **Statistical Approach:** Utilizes statistical models and machine learning to process and classify questions and answers.
- **Pattern Matching Approach:** Relies on matching text patterns to extract answers.
 - Surface Pattern Matching: Uses simple templates and regular expressions.
 - Template-Based: Predefined question/answer structures
- **Hybrid Approach:** Combines multiple techniques (linguistic, statistical, pattern-based) for improved performance.

4.3 KG Based-QA Systems

4.3.1 Definition

A KG-based Question Answering (KGQA) system is a system that allows users to ask questions in natural language, using their own terminology, and returns concise answers by querying a knowledge graph. It serves as an interface that abstracts away the complexity of formal query languages (e.g., SPARQL, SQL) and the underlying structure of the KG, making the rich, structured information in the KG accessible to non-expert users. [48]

4.3.2 Architecture of KG-QA Systems

This system relies on four main components, as summarized in the figure:

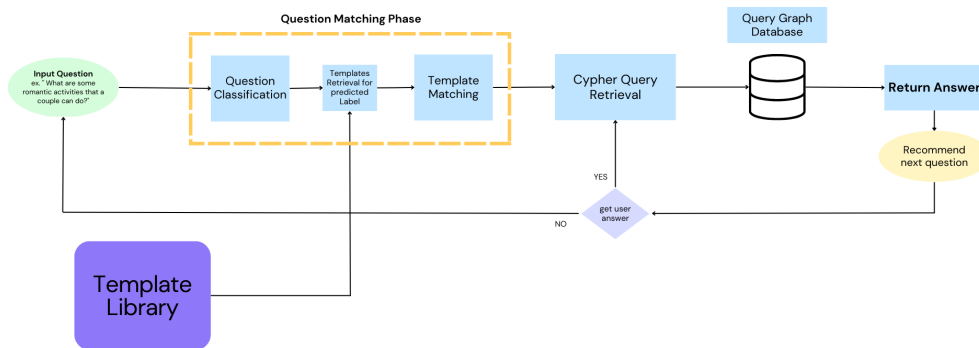


Figure 4.1: Illustration of a KG-based QA pipeline [50]

- **Question Understanding:**this is the first step where the system process the natural language question to extract it core components that include:entities mentioned in the question, determine relationships of the entity and understand the goal of the question
- **KG Mapping:**At this step the extracted entities and relationship are linked to their corresponding elements in the KG
- **Query Generation:**Construct a formal query (e.g., SPARQL, Cypher) that can be executed over the KG to retrieve a relevant answer.
- **Answer Retrieval and Ranking:**The final step involves executing the query and retrieving the answer,and converting the raw KG result into human-readable output.

4.3.3 Techniques in KG-QA

A KG-QA system combines multiple NLP and machine learning components. The following outlines the key techniques used across its major stages:

Natural Language Understanding (NLU):

- **Named Entity Recognition (NER):** Identifies entities and sometimes relations mentioned in the question. Common tools include: **spaCy, BERT-NER, Stanford NER, LLMs**.
- **Relation/Intent Classification:** Determines the intent of the user and classifies the question into predefined categories. Techniques and models used include: **SVMs, CNNs, LSTMs, Transformers, LLMs**.

Entity & Relation Linking: This step involves mapping words or phrases in the question to specific entities and relations in the knowledge graph. Techniques commonly used include:

- **Embedding similarity**
- **TF-IDF with BM25**
- **Neural attention models**
- **Semantic similarity and word embeddings**
- **Deep neural ranking models**

Semantic Parsing & Query Generation: Once entities and relations are identified, the system translates the question into a formal query language (e.g., SPARQL or Cypher):

- **Template-Based Parsing:** Relies on predefined query templates based on specific question patterns (e.g., "Who is the president of X?").
- **Semantic Parsing:** Automatically converts natural language into logical forms or queries. Techniques include: **Sequence-to-sequence models, Transformer architectures, Grammar-based parsing**.

Answer Retrieval: Retrieving answers from the knowledge graph typically involves two complementary approaches:

- **Query Execution:** Executes the generated query (e.g., SPARQL or Cypher) directly over the KG to retrieve exact matches.
- **Embedding-Based Retrieval:** Applied when symbolic querying is insufficient due to data sparsity or ambiguity. This includes:
 - **Link Prediction:** Infers missing triples.
 - **Answer Ranking:** Scores and ranks candidate answers using embedding similarity.
 - **Multi-hop Reasoning:** Explores indirect paths across entities for complex questions.

Techniques and models used: **TransE, TransH, ComplEx, R-GCN, GAT**.

Answer Generation: This stage transforms retrieved results into a final response. Depending on the system, it may involve:

- **Template-based NLG:** Uses predefined templates for fixed question types.
- **Pretrained Language Models:** Employs models like **T5**, **GPT**, **BART** fine-tuned on QA datasets to generate fluent, context-aware answers.

4.4 Conclusion

This chapter provided an overview of Question Answering (Q/A) systems, their core functions, and classification methods based on domain, question type, data sources, and language paradigms. We also examined the various techniques used to build these systems

5.1 Introduction

The Internet of Things (IoT) refers to a network of physical objects devices, vehicles, appliances, and other items embedded with sensors, software, and other technologies that enable them to connect and exchange data over the internet or other communication networks. This interconnectivity allows for the automation of processes, real-time data collection, and enhanced decision-making across various sectors. [60].

5.2 IoT in Healthcare

In healthcare, the Internet of Things is commonly referred to as the Internet of Medical Things (IoMT). It encompasses a network of connected medical devices and applications that collect, analyze, and transmit health data over networks. IoMT facilitates remote patient monitoring, telemedicine, and real-time clinical automation, thereby improving patient care, operational efficiency, and diagnostics. [61].

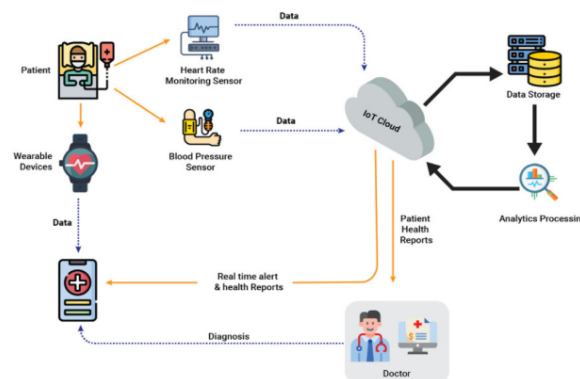


Figure 5.1: IoT in Healthcare: Data flow from patients through sensors to the cloud and doctors. [64]

5.3 Medical IoT Sensors and Data Acquisition

IoT devices provide healthcare professionals with opportunities to monitor patients continuously, while also enabling patients to track their own health. Wearable IoT devices bring significant benefits, including real-time monitoring, remote healthcare access, and better clinical decision-making.

5.3.1 Healthcare Monitoring Devices

- **Remote Patient Monitoring:** Continuous tracking of vital signs (heart rate, blood pressure, temperature) outside clinical settings. Data is sent to software platforms for analysis, with alerts or recommendations generated automatically.[63].
- **Glucose Monitoring:** IoT-enabled sensors monitor glucose levels, eliminating manual tracking and providing real-time alerts.[63].
- **Heart Rate Monitoring:** Wearable heart rate monitors provide continuous cardiac tracking with high accuracy, enabling timely interventions.[63].
- **Hand Hygiene Monitoring:** Devices remind staff and patients to maintain hygiene, reducing hospital infection rates by over 60%.[63].

5.3.2 Data Collection Mechanism

Medical IoT devices collect diverse patient data for improved healthcare delivery and predictive analysis. Key types of data include [63]:

- **Vital Signs:** Heart rate, blood pressure, oxygen saturation, body temperature.
- **Behavioral Data:** Activity tracking, sleep patterns, calories burned.
- **Medication Adherence:** Smart pillboxes or patches to monitor intake and send reminders.
- **Environmental Data:** Room temperature, humidity, and other context for patient comfort.
- **Specialized Medical Data:** Sensor-specific data for chronic conditions such as glucose and ECG.

Sensor Types and Connectivity:

- Motion sensors (accelerometers, gyroscopes) for movement and fall detection.
- Biometric sensors (ECG, SpO₂, temperature) for physiological monitoring.
- Thermal sensors for detecting variations in body temperature.

Data is transmitted via Wi-Fi, Bluetooth, or mobile networks to centralized healthcare systems for remote monitoring and decision support.

5.3.3 Data Collection Frequency

- **Continuous Monitoring:** Real-time collection from critical metrics like glucose and heart rate [63].
- **Interval-Based Collection:** Periodic collection from devices like fitness trackers or smart pill dispensers [63].

5.4 Challenges and Considerations

Despite their numerous benefits, the integration of IoT devices in healthcare faces several challenges [63]:

- **Data Security and Privacy:** IoT devices continuously collect sensitive patient information, making them targets for cyberattacks. Ensuring robust security measures—such as encryption, secure transmission protocols, and multi-factor authentication—is crucial. Compliance with regulations like HIPAA and GDPR is mandatory, and human oversight is essential to protect patient privacy.
- **Data Accuracy:** Technical malfunctions, battery issues, signal interference, or incorrect device usage can compromise the reliability of collected data. Expert validation and cross-checking with clinical observations are necessary to ensure data accuracy for medical decision-making.
- **Data Overload and Management:** IoT devices generate large volumes of structured and unstructured data. Efficient processing, organization, and auditing are required to avoid redundancy, incompleteness, or outdated information, which can negatively impact clinical insights and forecasting.
- **Data Ownership:** Ambiguities around who owns IoT-collected data—the patient, healthcare provider, or device manufacturer—pose legal and ethical concerns. Clear consent procedures, transparent data policies, and adherence to legal frameworks are essential to ensure proper data usage and patient control.

Part III

State of Art

6.1 Introduction

Several approaches have been proposed to leverage KGs in the medical domain, either to build QA systems or to enhance reasoning and prediction over large-scale clinical data. By structuring complex biomedical information into machine-interpretable knowledge, KGs provide a foundation for more accurate, explainable, and efficient decision support.

In this section, we review recent advances that exploit KGs in two main directions

- **Hybrid Approaches:** Methods that combine KGs with embedding models, deep learning, or machine learning techniques to improve reasoning, prediction, and knowledge retrieval.
- **KG-based QA Systems:** Approaches that design and implement QA systems in the medical domain by leveraging KGs. These methods focus on mapping natural language queries to graph representations, traversing KGs to extract relevant information, and delivering accurate, interpretable answers.

For each category, we outline key methodologies, highlight their advantages and limitations, and summarize representative contributions in a comparative table.

6.2 Hybrid Approaches: KG + Machine Learning/Deep Learning

6.2.1 Diagnosis Method of Thyroid Disease Combining Knowledge Graph and Deep Learning

In the paper [51] presents an intelligent diagnostic method for thyroid disease, that integrates Knowledge graph and Deep learning models. The primary methodological framework

of this research encompasses the following components: the construction of a knowledge graph, the embedding of the knowledge graph, and the training of the Bidirectional Long Short-Term Memory (BLSTM) model.

1. Construction of THYROID KGs:

- **Data collection:**The thyroid KG is built using structure data from a third-class hospital’s database,which include clean and well organized patient information, drug use information and diagnosis results.
- **Conceptual design:** the graph captures rich semantic relationship between various medical entitis ,allow users to visually observed and analyse how medical entities are connected The data obtained from the database are classified and the key medical entities extracted are: patient entity ,thyroid medication,diagnosis result ,basic information. the relationship between entities are defined as Thyroid factual relationship:that represent the relationship between thyroid medical entities such as :<patient, hospitalized diagnosis, goiter> , both “patient” and “goiter” represent medical entities while “hospitalized diagnosis” serve as factual relationship
the various types of relationship illustrated in Figure

relationship	Explanation
X has Y	Entity Y belongs to Entity X
X attribute_of Y	Entity Y has an attribute entity X
X use drug Y	Illness entity X Drug entity Y
X diagnosis Y	The diagnosis result of inspection report entity Y is entity X, and there is a diagnosis relationship between entities.

Figure 6.1: An illustration showing types of relationship [51]

A concept classification tree is used to organize the knowledge graph,it show the medical entities and the connection between these entities

The conceptual layer of the thyroid knowledge graph. illustrated in Figure

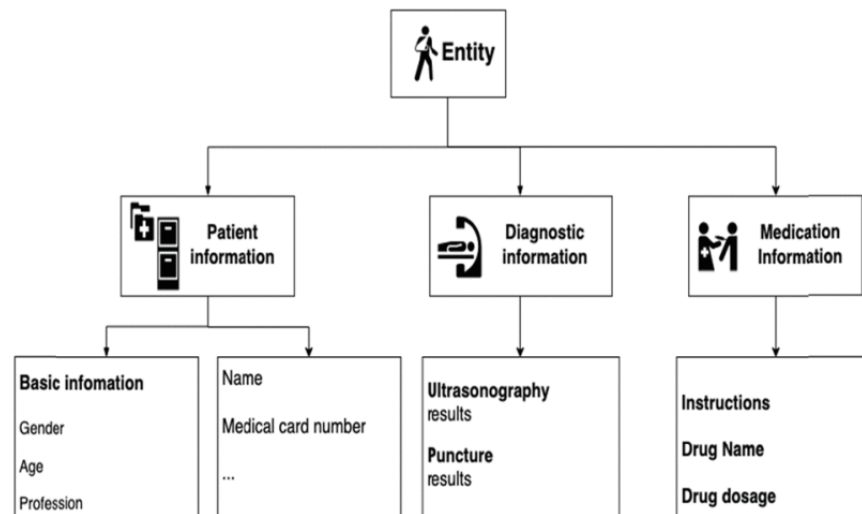


Figure 6.2: An illustration showing the conceptual layer [15]

- **Physical layer filling** This step involves mapping the concept defined layer to the actual database entities,using a method named entity mapping .The process

uses the Breadth-First Search (BFS) to traverse and populate the concept classification tree ,resulting the construction of the thyroid knowledge graph. This step included:

- (a) **Creating a mapping table:** that associates database entities with their corresponding concepts in the conceptual layer tree

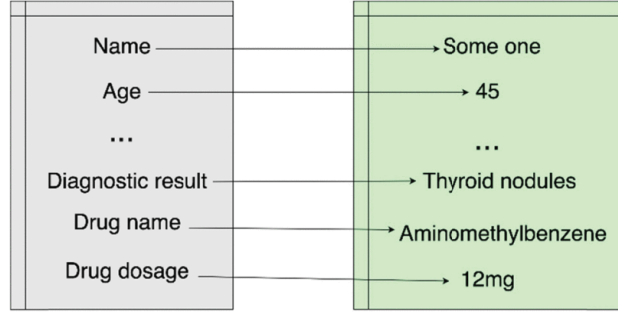


Figure 6.3: Mapping table example. [15]

- (b) **Filling entities** :the BFS algorithm is used to fill the concept tree with entities,placing each entity into its corresponding sub-node and associating it with its attributes and and their respective values.
- (c) **Extracting entities and relationships**:to generate structured triples that form the knowledge graph

As a result the knowledge graph represented as :

$$\text{GKG} = (E \cup \varphi(E), R \cup \varphi(R)) \quad (6.1)$$

where:

- E is the set of entities,
- R is the set of relationships, and
- φ maps entities and relationships to semantic types (T).

2. **Deep Learning Model**The goal of this model is to determine the probability that a specific pathology (initial conduction) lead to thyroid diseases.This is done by analysing a path in the knowledge graph ,which is a sequence of entities and their relationships,each path is represented as $\pi_l = e_0 r_0 e_1 r_1 \dots e_{l-1} r_{l-1}$.

Experiment and and analyse:

The experimental data used in this study consists of 1,200 cases for each of the following diseases: thyroid disease, diabetes, hypertension, and coronary heart disease. These data were obtained from the hospital database

To validate the feasibility and effectiveness of the proposed method,servel machine learning algorithms such as SVM,and neural network based model such like Propagation Neural Network (BPNN), Recurrent Neural Network (RNN), and Long Short-Term Memory (LSTM).The evaluation metric used:Accuracy, Precision, Recall, and F1-score.

The first set of experiments focus on comparing different feature extraction methods LDA

(Latent Dirichlet Allocation), VSM (Vector Space Model), and a medical knowledge graph, using SVM classifier and five-fold cross-validation. The result shows that features from the medical KG outperformed the other methods by achieving over 80

The experimental results are shown in the Figure

Index\Method	Knowledge Graph	LDA	VSM
Accuracy	83.24	70.03	74.25
Precision	83.18	66.80	72.49
Recall	81.15	70.81	78.85
F1	81.76	68.72	77.75

Figure 6.4: The first experimental results [51]

Next using the KG features various classifiers were tested (. SVM, BPNN, RNN, LSTM, and BLSTM.) The results show that deep learning model outperform the traditional machine learning algorithms, specifically the BLSTM delivered the best results across all the evaluation metric

The second experimental results are shown in the Figure

Index\Method	SVM	BPNN	RNN	LSTM	Our method
Accuracy	83.24	81.36	85.68	86.19	88.86
Precision	83.18	80.52	84.27	86.02	87.45
Recall	81.15	78.90	82.62	83.66	89.82
F1	81.76	78.95	81.89	83.10	89.40

Figure 6.5: Comparison of Disease Diagnosis Performance of Different Classifiers [51]

6.2.2 Biomedical knowledge graph embeddings for personalized medicine: Predicting disease-gene associations

The paper [52] addresses the challenge of extracting clinically valuable insights from the vast and fragmented biomedical knowledge available today, particularly for personalized medicine. It focuses on predicting disease-gene associations by integrating data from multiple biomedical databases into a unified Knowledge Graph (KG) and applying Knowledge Graph Embedding (KGE) methods. The methodology followed in this paper is as follows:

- **Data Collection:** The data used in the study to construct the large-scale biomedical knowledge graph (KG) was collected from three main sources:
 1. Gene Ontology (GO): Structured ontologies for gene and gene product characterization.

2. DisGeNET: A database of gene-disease and variant-disease associations.

3. Ensembl: A resource for genome annotation and gene information.

- **KG construction:** In this step, the authors use the data collected from various sources to construct the graph. They first extracted relationships from the GO OBO file, preserving original semantics and using annotation qualifiers to define relationship types. Annotation files for human gene products (proteins and RNA) were included, and gene-disease associations from DisGeNET v7 were merged. All gene symbols were standardized to Ensembl Gene IDs using the Ensembl Biomart via the pybiomart Python client. The final KG includes five main entity types: genes, diseases, molecular functions, cellular components, and biological processes.

- **Knowledge Graph Embedding:** The study applied Knowledge Graph Embedding (KGE) methods to convert KG entities into vector representations. Three algorithms were tested using the DGL-KE package: ComplEx, DistMult, and TransE. Training involved negative sampling, where incorrect triples were generated by corrupting valid ones, and filtered sampling was used to discard any negative samples already present in the KG.

Training The dataset was split 60/20/20 into training, validation, and test sets, ensuring all genes and diseases were fairly represented across sets. Only gene-disease associations were included in the validation and test sets, while GO-related triples were added to training only. Optuna, an optimization framework, was used to tune hyperparameters like max steps, learning rate, batch size, hidden dimensions, and regularization.

To evaluate each of the algorithms, the study used standard ranking metrics provided by DGL-KE: Mean Rank (MR), Mean Reciprocal Rank (MRR), and Hits@1, Hits@3, and Hits@10

- **Prediction of Disease-Gene Associations:** In this step, the study formulates disease-gene association prediction as a link prediction task on the knowledge graph. A scoring function $f(h, r, t)$, specific to the embedding method used, ranks potential associations between genes (head entities) and diseases (tail entities) through the "gene-disease association" relation. Using the fully trained KG and the best hyperparameters identified during optimization, the model ranks genes for each disease based on their predicted strength of association.
- **Analysis of Predicted Disease-Gene Associations (Autism Case Study):** The study focused on autism-related predictions by selecting top-ranked gene-disease associations. These genes were used to build a PPI network with STRING and analyzed using the Leiden algorithm to detect functional gene communities. Pathway enrichment was then performed with Reactome using: Profiler to uncover potential biological functions.

Result:

– **Knowledge Graph Construction** The resulting KG contains:

- * 93,657 nodes (entities): genes, diseases, molecular functions, cellular components, biological processes.
- * 1,705,585 relationships of 59 types, including gene-disease associations, gene-GO annotations, and GO ontology relationships.

The figure below represents different entities and the relationships between them:

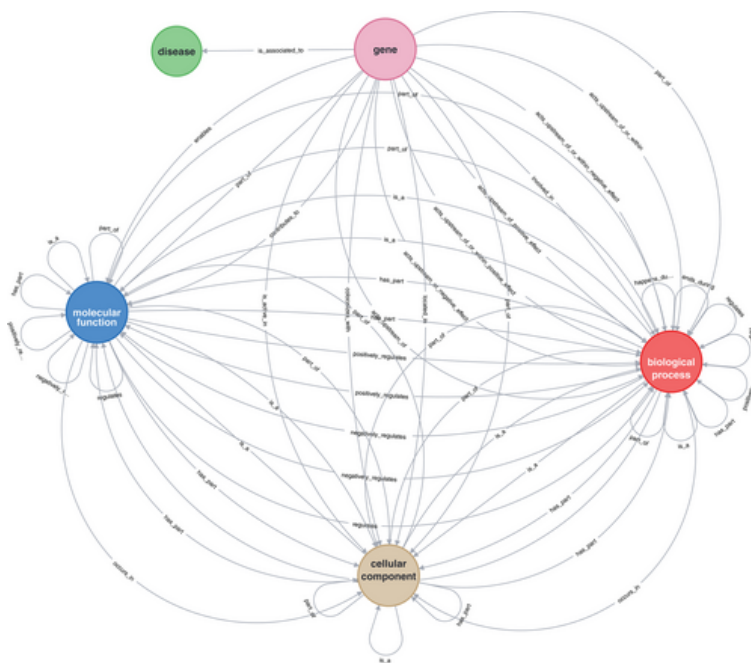


Figure 6.6: Metagraph representing all the node and relationship types in the KG.. [52]

- **KG Embedding** The performance metrics were calculated on the validation set using the optimal hyperparameter configuration for each algorithm. All algorithms demonstrated strong performance. The figure below presents the results obtained for each algorithm:

Method	MRR	MR	HITS@1	HITS@3	HITS@10
DistMult	0.93	1.24	0.88	0.98	0.99
ComplEx	0.95	1.17	0.92	0.98	0.99
TransE (l1)	0.95	1.17	0.91	0.99	0.99
TransE (l2)	0.96	1.13	0.93	0.99	0.99

Figure 6.7: Performance metrics for each method [52]

The TransE algorithm using L2 regularization achieving the best results

– Gene-Disease Association Predictions

- * Generated a ranked list of gene candidates for ASD, identifying 3,389 novel candidate genes in the top decile not previously associated with ASD in the KG.
- * Demonstrated the ability to predict biologically plausible novel gene-disease links.

– Biological Community Detection and Enrichment

Six major biological communities were identified in the ASD gene interaction network. Enrichment analysis showed involvement in key pathways such as:

- * Metabolism
- * GPCR-mediated synaptic transmission
- * Cytokine signaling
- * Gene expression regulation
- * Nervous system development
- * Wnt signaling

All of these are relevant to ASD and potential treatment strategies.

– Implications for Personalized Medicine

This approach supports the discovery of novel gene-disease associations and enables patient stratification based on shared biological mechanisms. It offers a scalable, machine learning-driven framework for integrating complex biomedical data to advance precision medicine.

6.2.3 Decision-Making System for the Diagnosis of Syndrome Based on Traditional Chinese Medicine Knowledge Graph

The paper [53] introduces a Decision-Making System for the Diagnosis of Syndrome (DSDS) based on a Traditional Chinese Medicine (TCM) Knowledge Graph. The goal is to improve the accuracy and informatization of TCM syndrome diagnosis by leveraging relationships between symptoms and syndromes extracted from electronic medical records (EMRs).

The proposed framework of DSDS is detailed in the figure below:

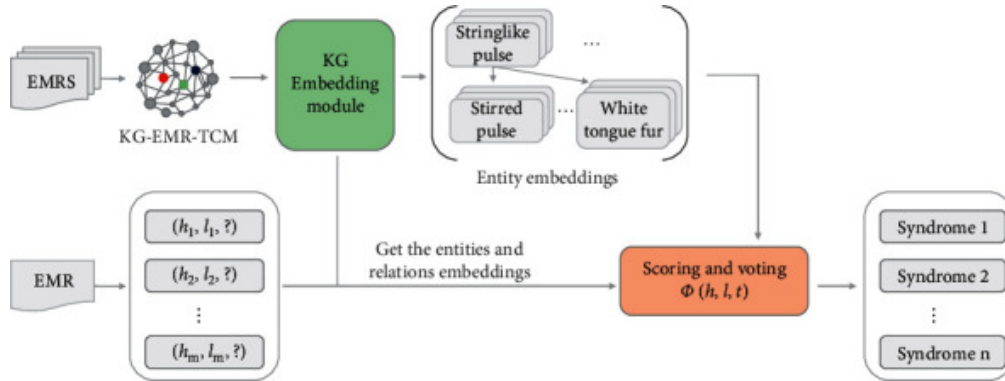


Figure 6.8: DSDS model architecture based on KG-EMR-TCM [53]

The methodology followed in this paper is outlined in the following steps.

- **Knowledge Graph Construction (KG-EMR-TCM)**

In this step, the researchers built a Knowledge Graph (KG) from 108,746 desensitized Traditional Chinese Medicine (TCM) electronic medical records (EMRs). They aggregated information from each EMR such as symptoms, diagnoses, and treatment principles to construct the graph. This allowed them to link symptoms with syndromes. The resulting KG, named KG-EMR-TCM, was stored in Neo4j.

- **Knowledge Graph Embedding**

In this step, the KG-EMR-TCM was embedded using the ComplEx model to represent entities and relationships as low-dimensional vectors, capturing their semantic meaning.

The ComplEx model maps entities and relations into complex vector space to handle both symmetric and antisymmetric relations. A scoring function evaluates the validity of triples, assigning higher scores to correct ones and lower scores to incorrect ones. This helps the model learn meaningful relationships in the KG structure for downstream tasks like reasoning or recommendation.

- **Syndrome Prediction**

The DSDS model is used to predict syndromes from EMRs by completing missing triples of the form $(h, l, ?)$, where the tail entity (syndrome) is unknown. The ComplEx embedding model computes scores for each candidate triple, ranking possible syndromes based on how well they fit with the symptoms.

Since high scores alone can be misleading due to symptom overlap in TCM, the authors proposed a scoring and voting mechanism:

- Generate multiple candidate triples and score them using a combined score function (f).
- Count the frequency of each predicted syndrome using a voting method (G).
- Select the syndrome with the highest overall vote (S_n) as the final recommendation.

This ensures a more robust and symptom-aware prediction.

Experimental Setup & Result:

- **Data & KGs:**

- Dataset: 108,746 anonymized TCM medical records from QIHUANG TCM platform.
- Content: Each record includes symptoms, syndromes, diagnosis, physical exams, and nursing precautions.
- Knowledge Graph (KG): Built from 1,499,457 triples using 202,619 unique entities across categories like pulse taking, tongue inspection, and syndrome.

- **Model Evaluation:** The proposed model was compared to three other approaches using the following evaluation metrics: Accuracy, Weighted Recall, and Weighted F1-Score. The models included Gradient Boosting Classifier (GBC), XML-CNN (a baseline deep learning model), KG-XML-CNN (XML-CNN enhanced with knowledge graph embeddings), The obtained results are summarized in the figure below:

Method	Weight-recall	Weight-F1	Accuracy
GBC	—	—	—
XML-CNN	0.525	0.527	0.574
KG-XML-CNN	0.584	0.580	0.626
DSDS	0.618	0.612	0.621

Figure 6.9: Performance comparison of different models. [53]

DSDS outperformed all models in recall and F1, showing its strength in handling overlapping symptoms through embedding and voting mechanisms.

- **Decision-Making Performance** In this part, the authors evaluated how well their system supports clinical diagnosis by providing Top-N syndrome recommendations. Instead of predicting only one syndrome, the model ranks multiple possible syndromes and checks if the correct one is among them.

They measured Top-N accuracy, where:

- Top-1 (only one recommendation): 62.1% accuracy
- Top-3 (top three suggestions): 71.0% accuracy
- Top-5 (top five suggestions): 80.6% accuracy

This shows that by increasing the number of recommended syndromes, the model significantly improves its support for clinical decision-making.

6.2.4 Discussion

In this approach [51], a KG is constructed from various medical databases, encompassing entities related to Thyroid Disease and their associated relationships. These entities and relationships are embedded into low-dimensional vectors using the TransE model, which are then used to train a Bidirectional LSTM (BLSTM) model for diagnostic purposes. By leveraging the KG, the system benefits from the structured and semantically rich connections among medical concepts, enhancing diagnostic reasoning. The TransE embeddings enable the model to represent and utilize the semantic relationships between different entities, while the BLSTM effectively captures complex sequential patterns in patient data, contributing to improved diagnostic accuracy.

Despite these strengths, the KG is built from hospital-specific data, which may limit its applicability to other clinical environments or patient populations. Additionally, the model’s diagnostic performance can be negatively impacted by inaccurate or incomplete symptom inputs provided by users.

This study [53] introduces an end-to-end model called DSDS (Decision-making System for the Diagnosis of Syndrome), designed to support syndrome prediction in Traditional Chinese Medicine (TCM) by leveraging a KG constructed from electronic medical records. The KG captures complex relationships among medical entities, helping to address diagnostic overlaps commonly seen in TCM. Symptoms and syndromes are embedded as vectors using the ComplEx model, and a scoring and voting mechanism is applied to rank the most relevant syndrome predictions. This approach achieved 80.6 accuracy for top-5 recommendations, providing actionable clinical decision support. By integrating KG embeddings with a robust ranking strategy, the model effectively handles symptom ambiguity and outperforms baseline methods such as Gradient Boosting Classifier (GBC) and XML-CNN.

Despite its strengths, the system’s generalizability is limited by the use of a single dataset for KG construction, which may not capture the diversity of TCM practices or populations. Furthermore, the model evaluation relies solely on the ComplEx embedding technique, without comprehensive comparisons to more recent KG embedding models such as RotatE or InteractE.

This paper [52] presents a method for predicting gene-disease associations using KGE techniques applied to a biomedical KG constructed from diverse data sources. The KG integrates information on genes, diseases, and molecular functions from databases such as DisGeNET and Gene Ontology (GO), enabling a rich representation of biological relationships. KGE models—including TransE, DistMult, and ComplEx—are employed to predict novel gene-disease associations, with TransE combined with L2 regularization showing particularly strong performance (e.g., HITS@10 = 99). The model successfully identified 3,389 novel genes associated with autism spectrum disorder (ASD), clustering them into six biologically meaningful pathways such as synaptic transmission and cytokine signaling, which align with known ASD mechanisms.

Despite these promising results, the approach is limited by its dependence on DisGeNET and GO, which may not capture rare diseases or under-researched genes, thereby constraining the prediction scope. Additionally, the high-dimensional nature of KGE embeddings complicates interpretability, making it challenging to understand why specific genes are prioritized without further post-hoc analysis.

6.2.5 Comparative analysis

The comparative table provides an overview of state-of-the-art approaches in developing medical assistance systems that combine knowledge graphs, machine learning, and deep learning models. The comparison is organized according to the following criteria:

- Methods used for knowledge graph construction
- Algorithms employed
- Data sets utilized
- Reported results and performance metrics

Table 6.1: Comparative Overview of Medical KG-based Diagnostic Systems

Paper Title	KG Construction	Algorithm Used	Data Set Used	Results
Diagnosis Method of Thyroid Disease Combining Knowledge Graph and Deep Learning	-Ontology design -Entity mapping using BFS	-BLSTM with forward and backward hidden layers -TransE	-Hospital database with 4 diseases (thyroid, diabetes, hypertension, coronary); 1200 cases/disease	BLSTM: Accuracy = 87.6%, Precision = 86.1%, Recall = 84.7%, F1 = 85.4%
Biomedical Knowledge Graph Embeddings for Personalized Medicine: Predicting Disease-Gene Associations	Heterogeneous multi-graph; Semantic GO preservation; Merged via Ensembl Biomart	-TransE -DistMult -ComplEx	-Gene Ontology (GO), -DisGeNET v7.0 -pybiomart	TransE: MRR = 0.96, MR = 1.13, HITS@1 = 0.93, HITS@3 = 0.99, HITS@10 = 0.99
Decision-Making System for the Diagnosis of Syndrome Based on TCM Knowledge Graph	-Data cleaning, desensitization -word segmentation, entity aggregation -Stored in Neo4J	-ComplEx -TensorBoard	QIHUANG TCM Dataset	Top-1 = 62.1%, Top-3 = 71.0%, Top-5 = 80.6%; Weighted Recall = 0.618; Weighted F1 = 0.612

6.3 Q/A Systems Based Only on Knowledge Graphs

6.3.1 A knowledge graph based question answering method for medical domain

This paper [54] presents a knowledge graph based question answering method for medical domain

The methodology can be summarized in the following key steps:

1. **Medical Knowledge graph construction:** A medical KG was constructed using high-quality medical data sources, include: medical instructions of drugs and diseases, which contain information about Drugs(:Adaptation diseases, adaptation symptoms, taboo populations, etc) and Diseases(Complication symptoms, body parts affected, hospital departments, treatments, prevention methods, etc.) the extracted entities from the medical instructions, including: disease, symptom, drug, sex, population, body part. Additionally certain fields within the instruction such as cause, indications, contraindication, prevention, cause ect are treated as properties of the corresponding entities. To model the semantic structure of the medical knowledge, relationship between entities are assign weighted values. a positive Weight (e.g., 1.0) indicates a positive correlation (e.g., a disease causing a symptom), while negative Weight (e.g., -1.0) indicates a negative correlation. (e.g., a drug being contraindicated for a specific population). Furthermore, the strength of relationship between disease and symptom is differentiated into: strong correlations (weight = 1.0) to represent the main symptom and ,weak correlations (weight=0.5) that represent complication or secondary symptom. To extract entities and relationships from the medical instructions, the system uses semantic templates. These templates are hand-crafted rules that match patterns in the text.

The conceptual layer of the medical knowledge graph. illustrated in Figure

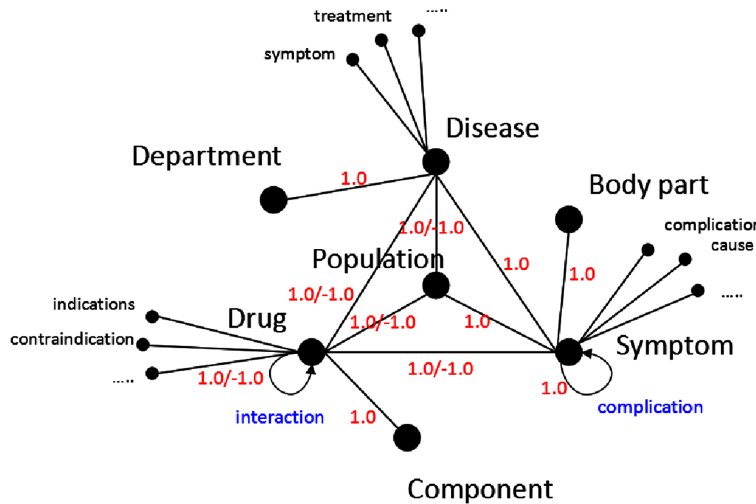


Figure 6.10: The medical knowledge graph in our approach. [54]

2. **Question Type Identification and Classification:** This component is responsible for understanding the user queries by identifying the key entities and determining the question intent. To achieve this the system perform two key task:

- **Named Entity Extraction** The question is first segmented into a sequence of words : $WD = \{wd_1, wd_2, \dots, wd_j, \dots\}$ using Jieba segmentation tool. Common stop words (e.g., "is", "has", "and", "of") are removed. The remaining words are matched against a dictionary of named entities in the knowledge graph. If a word Wdj matches an entity in the knowledge graph, it is added to the set of extracted entities E .
- **Intent Recognition:** An information Gain-Based Classifier build to classify the question into one of nine medical categories (Disease Diagnosis ,Symptom ,Treatment ,Diet ,Cause ,Drug Recommendation,Taboo Population,Indication, Prevention). Questions are represented by generalized entities and keywords, encoded as one-hot vectors. The model computes the Information Gain of each feature to determine the most likely category, selecting the one with the highest total score. The pseudo-code of the intent recognition method presented in the Figure:

Algorithm 1 Information gain based question intention recognition.

Input: A representation of a question: $q_i = \{\{entity\}, \{word\}\}$, where $\{entity\}$ is a set of entities in q_o and $\{word\}$ is a set of the other common words in q_i .

Output: The concept of the question q_i : c_{max} .

- 1: Replace the $\{entity\}$ in q_i to $\{concept\}$ which the $\{entity\}$ belongs to.
- 2: Calculate the information gain for each element x_j in q_i for each concept c_k according to $IG(x_j, c_k)$.
- 3: **function** *RecogConcept*(q_i)
- 4: Scan x_j in a given question q_i
- 5: Sum the total value of x_j in the given question q_i for each concept c_k
- 6: Find a concept c_{max} which has a maximum value;
- 7: **return** c_{max} .
- 8: **end function**

Figure 6.11: The pseudo-code of the intent recognition method. [54]

3. **Answer Retrieval Strategy:** use the KG to answer questions by traversing from question entities to answer entities. each question is represented as set of entities and concept $q_i = \{e_1, e_2, \dots, e_n, c_k\}$ where e_j are entities and c_k represent the intent of the question. The objective is to infer the most appropriate entity in the graph as an answer

Implementation and evaluation:

The evaluation was conducted on a Linux Ubuntu 16.04 system equipped with an Intel i5-3470 CPU @3.20 GHz and 16 GB RAM. The proposed KBQA system was developed in Java. The medical KG was constructed using 182,638 standard medical instructions from a medical company (YiFeng Pharmacy), the graph contain containe 34,788 nodes ,601,475 relationships and was imported into Neo4J for the visualization. And 149,998 question-answer pairs to test the system

The proposed method was evaluated using metrics such as : training accuracy, decision accuracy, training time, and decision time. The result showed a training accuracy of 94.94 and decision accuracy of 93.89 which indicate a high accuracy . The training time was 201.08 seconds for all training samples, and decision was less than 0.001 seconds per query, indicating high efficiency.

For the question answering two methods were applied ,for category like "symptom", "treatment", "drug", answer were retrieve by directly searching properties for entities. For disease diagnosis a a weighted path-ranking inference method was applied

The proposed method was compared with two traditional approaches: Bayesian Networks, Case-Based Reasoning, the results show that the proposed method outperforms the traditional

method ,in both accuracy and Disease converge, by achieving more than 80 in decision accuracy
the obtained results are as shown in figure :

Table 5 The performance evaluation of our method and the other state-of-art methods for disease diagnosis.

Method	Decision accuracy (%)	Disease coverage (%)
Our approach (KGQA)	86.4	95.7
Vembandasamy et al. (Bayesian)	78.1	92.4
Abdel-Badeeh et al. (Case based)	75.2	90.3

Figure 6.12: The performance evaluation of the proposed method and the other state-of-art methods for disease diagnosis. [54]

6.3.2 Research on Medical Question Answering System Based on Knowledge Graph

In this paper [54] the authors aim to address a critical challenge in the medical question answering system (e.g., resource imbalance, unqualified online consultations). by proposing an approach that integrates the knowledge graph and natural language processing (NLP) for accurate,automated responses.The methodology can be summarized in the following key steps:

1. **Knowledge graph construction:**the construction of the medical KG involves the key steps:
 - **Data Acquisition:**The data used to build the MKG was extracted from two primary sources.Professional databases containing structure medical data from specialized industries such as hospital records and pharmaceutical databases.The second source comprises unstructured web data including medical articles,HTML pages,which are collected through web scraping technique.However the unstructured data require a significant preceding before it can be it can be transform into usable knowledge
 - **Information Extraction** this process involves automatically identifying and extracting structure knowledge including entities ,relationships and attribute.structure data requires only minimal preprocessing before use in it.while semi-structured or unstructured require advanced NLP technique to convert it into usable knowledge.Key technique used in the step :**Entity Extraction (NER)** to identify main concept in the KG,**Relation Extraction** to extract connection between entities and **Attribute Extraction** to capture descriptive detail.
 - **Knowledge Fusion** this phase integrate the extracted data to resolve problem like:duplicate or conflicting form different sources,as well as variations in terminology (e.g., "flu" versus "influenza").to resolve theses the authors apply **Entity Disambiguation** through an improved d K-means algorithm to cluster ambiguous term by context.**Entity Linking** where they use Jaccard similarity to map extracted entities to the knowledge graph by measuring semantic overlap.the refined data is then stored in triples and stored in Neo4j
2. **Question Answering System:**The medical Q/A system employs a semantic analysis to process natural language question through three key steps :

- **Question classification:** this step is about classify medical question based on their characteristics, noting that this approach limits the query scope to predefined classes. To handle this the system uses template-based keyword matching with predefined question
- **Question analyse** this process begins by extracting medical entities from user question, using the Aho-Corasick (AC) multi-pattern matching algorithm, in case exact matches fail, Jaccard similarity (0.5 in sample comparison), cosine similarity, and TF-IDF are used. Finally the system classifies the user question intent using Naive Bayes model trained on manually labeled data and keyword-based features
- **Query processing:** The developed system is Retrieval-Based system not generated, because medical domain demands high precision and generative models can hallucinate incorrect facts. Two databases are used graph database and relational database. The core module responsible for question understanding and retrieving the right answer includes two sub-modules. Knowledge Base Retrieval Sub-Module which works in this way: After identifying the user intent and the medical entities in the question, the system generates Cypher queries based on the recognized entities and question type which then sent to the graph database to fetch answers. The second one is question and Answer (QA) Retrieval Sub-Module which achieved in case no intent type is found. This module employs a problem classifier that predicts the specific type of the question and extracts domain-specific key words and entities type, and narrows the scope of the search accordingly. It then queries the Q/A database which might include frequently asked questions or historical QA archives and generate a query to retrieve the most similar question from the database

The query processing module represented in the figure:

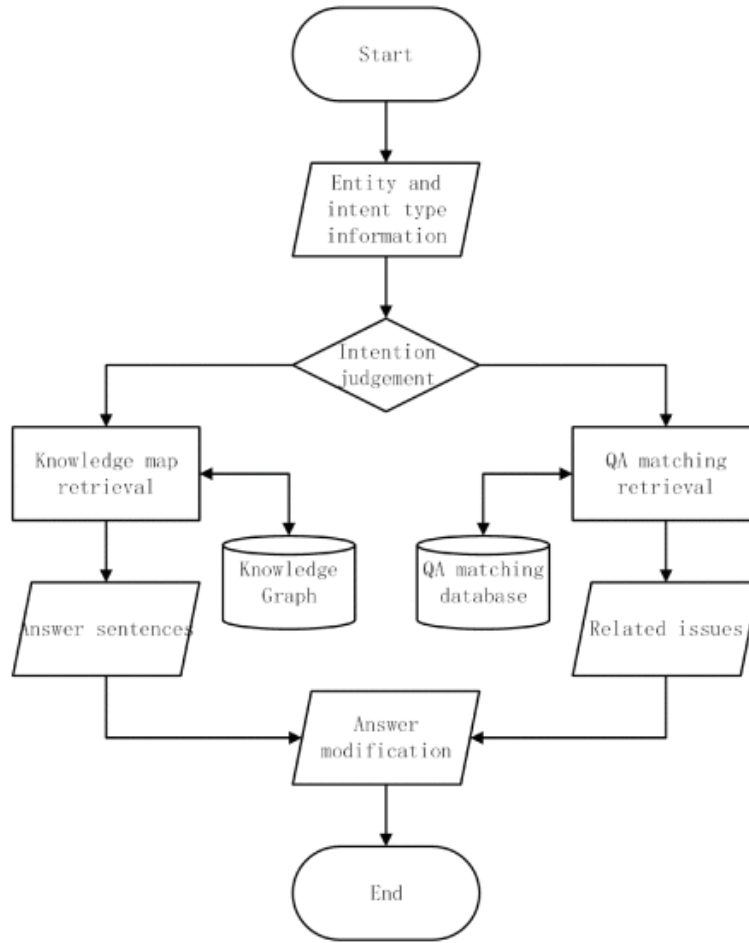


Figure 6.13: Query processing module diagram. [55]

Experimental and Results:

The data used to build the medical knowledge graph was collected from vertical medical websites (e.g., 39 Health Network, Drug Search, Medical Search). As a result a knowledge graph with 44,000 knowledge entities across 7 types and 300,000 relationships across 11 types of entities. stored in the Neo4J database. The evaluation of the proposed method focuses on measuring its performance in responding to questions related to diseases, drugs, and symptoms. Two retrieval methods were compared: knowledge graph-based retrieval and question-and-answer (QA) pair-based retrieval. The evaluation metric user are Precision, Recall, F1 Score

The results show that Knowledge Graph Retrieval: achieve Higher recall, lower accuracy. In contrast, QA pair retrieval demonstrate higher accuracy as it reused previously verified answers and lower recall.

the results are illustrated in the following Figure

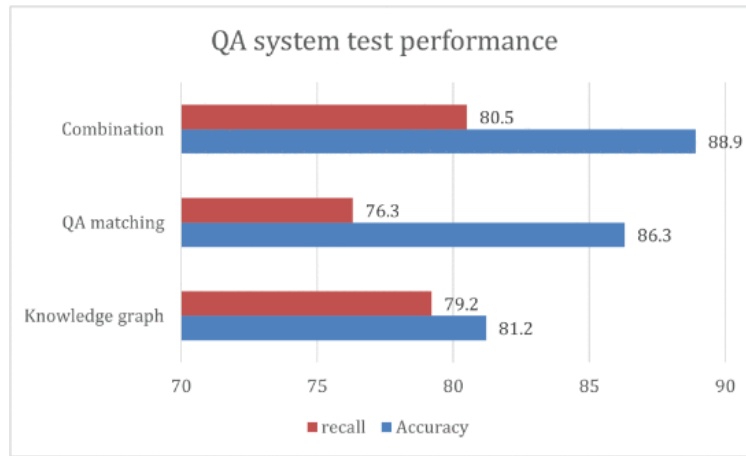


Figure 6.14: Query processing module diagram. [55]

The study concluded that combining both retrieval approaches can balance this trade-off effectively: the knowledge graph improves coverage, while QA pair matching increases precision. However, the final accuracy of the system remained suboptimal, primarily due to the quality of the data sources.

6.3.3 Question Answering System Based on Knowledge Graph in Traditional Chinese Medicine Diagnosis and Treatment of Viral Hepatitis B

In this paper [57] authors presents a question-answering (QA) system for Traditional Chinese Medicine (TCM) diagnosis and treatment of viral hepatitis B, leveraging a knowledge graph (KG) and deep learning techniques

The figure below presents the workflow of the question-answering system, that relies on there main steps:

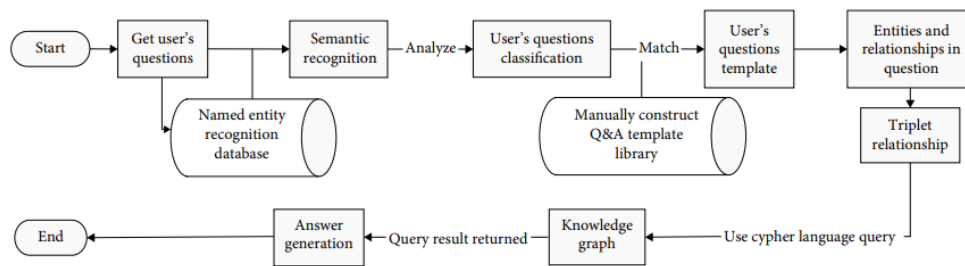


Figure 6.15: workflow of the question answering system. [57]

Data collection The system is built on two primary data source Medical Records: Collected from 8,544 patients (2009–2011) across 20 Grade A hospitals in China, filtered to 2,573 first-time patients treated solely with TCM. Included 25,549 clinical records with details like symptoms, diagnosis, prescriptions, and lab indicators Supplementary Knowledge: Structured data crawled from Xunyiwenyao (<https://www.xywy.com/>), a medical website, focusing on hepatitis B self-recuperation (e.g., diet, suitable/taboo foods).

KG construction:The KG was constructed through

- **Ontology Design:** A structured ontology layer was created based on medical record structure and hepatitis B-specific knowledge, referencing standards like "*Diagnostics of TCM*" and "*The Traditional Chinese Medicine Pharmacopoeia*."
- **Knowledge Extraction:** A joint model with a multihead mechanism extracted entities and relationships from medical records, manually proofread by TCM experts for accuracy. Web data was parsed using Python's `urllib` and `XPath`, stored temporarily in MongoDB.
- **Knowledge Representation:** In this study, knowledge is represented using triples in the form of (entity1, relation, entity2), where the knowledge graph is defined as $G = (E, R, S)$. Here, E is the set of entities, R is the set of relations, and S is the set of all valid triples.
- **KG Storage:** The knowledge graph was stored in a Neo4j graph database, using Cypher for data import and querying.

Q/A system development : The workflow of processing user question is as follow:

- **Entity Extraction:** This study used a Bi-LSTM + CRF model to extract both continuous and non-continuous entities at a fine granularity. It addressed the challenge of medical entity recognition in Chinese, which is more complex than in English due to the need for word segmentation and irregular naming conventions in the medical domain.
- **Question Template Matching:** This study uses question template matching to understand user queries. A set of common question templates was designed, and Word2Vec was used to calculate text similarity between user queries and these templates.
- **Query Execution:** When a user submits a query—such as about hepatitis B—the system first conducts semantic analysis and word segmentation. It then matches the query to predefined templates and performs entity recognition. Using the matched template and identified medical entities, the system generates Cypher queries with Neo4j to retrieve relevant relationships from the knowledge graph.

Experimental Evaluation: Setup: Experiments were conducted on a system with an Intel Core i5-4210M (2.60 GHz), 4 GB RAM, and Windows 7, using Python in PyCharm 2017.3.3. The environment supported web crawling, entity/relationship extraction, and Neo4j integration.

- **Knowledge Graph:** Based on real-world doctors' medical records and hepatitis B supplemental care data crawled from web pages, a partial section of the constructed knowledge graph is shown in the figure below:



Figure 6.16: Partial display of hepatitis B knowledge graph. [57]

- **NER Experiment:** The Bi-LSTM + CRF model was evaluated for NER performance on medical record data. Metrics were:
 - **Precision:** 0.8234
 - **Recall:** 0.7241
 - **F1 Score:** 0.7710
- **Q/A Evaluation:** The system was tested with 300 manually designed TCM hepatitis B-related questions, semantically aligned with the template library. Evaluation compared system outputs to original medical records and web data, with correctness defined by accurate entity recognition, semantic template matching, and valid KG query results. Results showed:
 - Accuracy:** 81.67% of questions were answered correctly, demonstrating robust performance in recognizing entities and retrieving relevant answers.
- **Template Matching Effectiveness:** The Word2Vec-based template matching achieved high semantic alignment for common queries but struggled with questions outside the template library, highlighting limitations in intent analysis flexibility.

6.3.4 Research on a traditional Chinese medicine case-based question-answering system integrating large language models and knowledge graphs

This study [56] proposes a novel approach to enhancing Traditional Chinese Medicine (TCM) research by developing a case-based Q&A system that combines LLMs with KGs. By leveraging these advanced AI techniques, the system aims to overcome key challenges in

analyzing TCM case records, including high data volume, complex semantic content, and inefficient information retrieval.

The development of this system relies on two main components: knowledge graph (KG) construction and question-answering (QA) system development, as shown in the figure below:

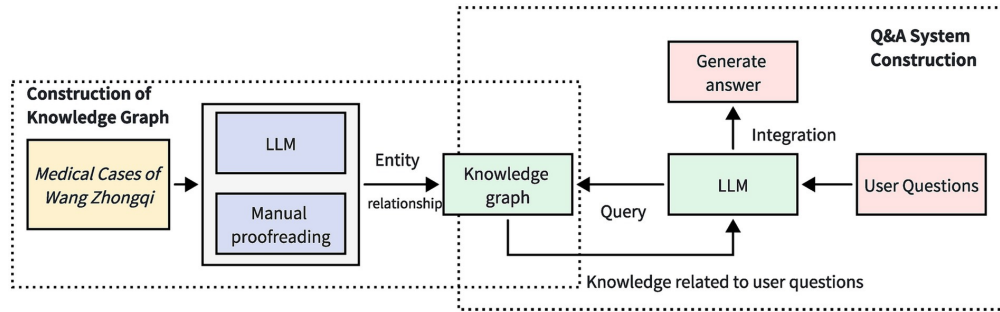


Figure 6.17: This diagram shows the workflow of the question-answering system. [56]

KG construction

- **Data Source:** The data used to construct the knowledge graph consists of 679 TCM case records from Medical Cases of Wang Zhongqi (1923–1935), covering 41 diseases. These cases were selected from an original set of 1,573 based on the completeness of their entity information, including disease names, symptoms, pathogenesis, treatment principles, and medications.
- **Schema layer construction:** The schema layer of the knowledge graph is built using a combination of top-down and bottom-up approaches. It starts with a standardized TCM semantic framework to define entities and relationships, ensuring consistency. Then, it is refined based on the structure of the Medical Cases of Wang Zhongqi to accurately reflect the content of the case records.
- **Knowledge extraction:** This process involves extracting entities and the relationships between them from unstructured data. It comprises two main steps:
 - Entity extraction: The study uses Named Entity Recognition (NER) to extract key entities—symptoms, pathogenesis, and treatments—from TCM case texts. To overcome limited annotated data, it employs GLM-3-Turbo with few-shot learning, using well-crafted prompts. These include examples with clear symptoms, no symptoms, and complex expressions to enhance accuracy and reduce false positives. This approach enables precise and reliable entity extraction for building the knowledge graph
 - Relationship extraction: The study applies relation extraction to identify and structure connections between entities—such as symptoms, diseases, and treatments—in TCM texts. Using the results from NER, entities are linked into TCM-specific relational triples (e , r, e) based on predefined schema relationships. The extracted data is organized in Excel and processed using the pandas library to generate structured triples for the knowledge graph

The constructed knowledge graph was stored and visualized using the Neo4j graph database. The figure below summarizes all the previous steps.

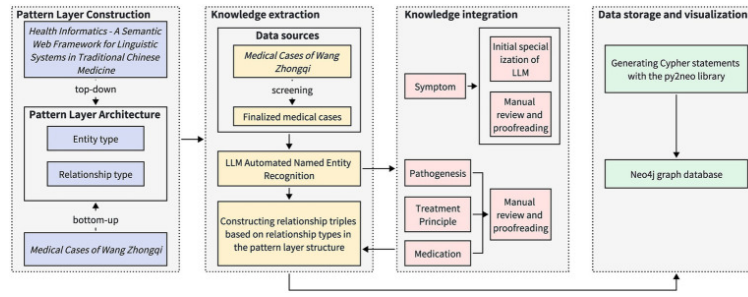


Figure 6.18: This diagram shows the workflow of the workflow involved in the construction of a knowledge graph. [56]

Question answering system development the developed Q/A system support natural language queries. The system involve four main steps :

- **Extracting entities:** In this step, the LLM analyzes the user's query to identify key entities such as diseases, symptoms, or medicines using its natural language processing capabilities.
- **Generating cypher statements:** Based on the extracted entities, the LLM generates Cypher queries using a predefined structure to maintain consistency and accuracy, minimizing the risk of unstable output.
- **Querying Nodes** The generated Cypher queries are executed on the Neo4j knowledge graph, retrieving the relevant entity nodes and the relationships between them.

Generating Answers Finally, the system uses the returned results from the knowledge graph to generate a clear and accurate natural language response, which is then delivered to the user.

The figure below summarizes all the previous steps:

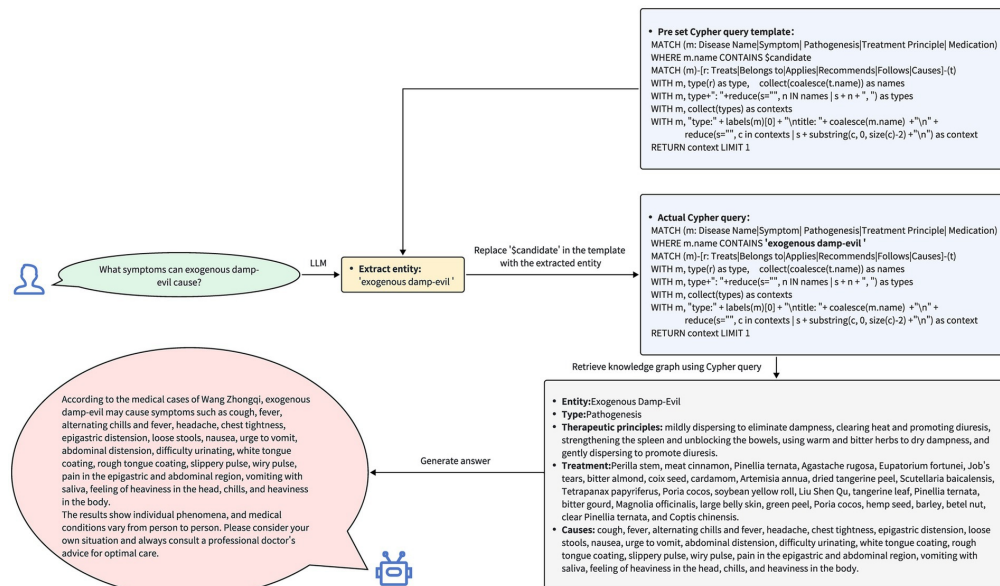


Figure 6.19: Query Process and Response Flow of the Traditional Chinese Medicine Question-Answering System. [56]

Evaluation:

- **NER Comparative Experiment:** Evaluate different LLMs for their ability to extract key TCM entities (symptoms, pathogenesis, treatment principles) from case records. The compared models and evaluation metrics are summarized in the figure below:

	Precision	Recall	F1 scores
GPT-3.5-Turbo	0.4834	0.8989	0.6287
GLM-3-Turbo	0.9462	0.9778	0.9617
Spark3.5 Max	0.8173	0.9200	0.7892
Qwen-Turbo	0.6335	0.8883	0.7395
Moonshot-V1-8 K	0.9086	0.8833	0.8958

Bold values indicate superior performance.

Figure 6.20: Named entity recognition evaluation results. [56]

GLM-3-Turbo achieved the top scores with a precision of 0.9462, recall of 0.9778, and an F1 score of 0.9617, demonstrating exceptional accuracy in extracting TCM entities. **QA System Performance (RAGAS Evaluation)** Evaluate the accuracy and reliability of the QA system that integrates LLMs with KG. The study employed the RAGAS framework designed for evaluating Retrieval-Augmented Generation systems.

GPT-3.5-Turbo delivered the best performance, scoring 0.9375 in faithfulness, 0.9686 in answer relevance, and 0.9500 in context recall, indicating highly accurate and context-aware responses.

- **Manual (SUS) Evaluation of Safety, Usability, and Smoothness:** A manual evaluation using the SUS framework assessed the system’s safety, usability, and smoothness on a 1–3 scale. the custom QA system combining LLMs with the knowledge graph significantly outperformed all others, with near-perfect scores: 2.868 (Safety), 2.848 (Usability), and 2.980 (Smoothness). These results confirm the system’s superior reliability, fluency, and professional quality in answering TCM-related queries.

6.3.5 A New Approach of Question Answering based on Knowledge Graph in Traditional Medicine

This paper [58] proposes a novel hybrid Question Answering (QA) system based on a Knowledge Graph (KG) to handle the problem of limited reasoning capabilities in medical chatbots, specifically for the domain of Traditional Medicine in Vietnamese. The figure below illustrates the architecture of the KG-based QA system:

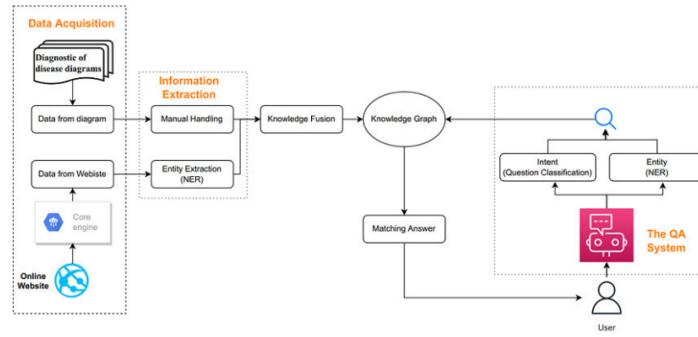


Figure 6.21: Architecture of QA system based on Knowledge Graph diagnosis. [58]

The overall approach is developed in detail through two main phases:

- **Knowledge Graph Construction:** The authors adopted a three-stage pipeline for building the medical knowledge graph:

- **Data Acquisition:** Medical data was collected from two primary sources:

- * **Diagnostic disease diagrams:** These are highly reputable but originally available in image format. Therefore, we manually converted the information into structured text.
- * **Medical websites:** Data was also collected from trusted Vietnamese medical platforms such as vinmec.com and nhathuoclongchau.com.

This combination ensures both reliability (from diagnostic diagrams) and breadth of coverage (from online medical resources).

- **Information Extraction:** To extract structured knowledge, the authors focused on entity recognition (disease and symptom names), relation extraction, and attribute identification. A manually annotated dataset was created, consisting of 2,104 sentences and 26,106 entity labels. Preprocessing steps included removal of HTML tags, Unicode normalization for Vietnamese, and cleaning of noisy symbols. Annotation quality was validated by three independent annotators, yielding a 90.5 accuracy rate, 96 per 400 sentences, and a Cohen’s Kappa score of 0.81, indicating strong inter-annotator agreement.
- **Knowledge Fusion:** Given the heterogeneity of the sources, the authors applied entity disambiguation, linking, and similarity matching to resolve redundancies and ensure consistency. The finalized graph was implemented in Neo4j, a graph database.

- * **Nodes:** 13,311, representing *diseases*, *symptoms*, *typical symptoms*, and *diagrams*.
- * **Relationships:** 16,638, including *HAS_SYMPTOM*, *HAS_TYPICAL_SYMPTOM*, *YES*, *NO*, *HAS*, *HAS_NO*.
- * **Attributes:** Disease nodes were enriched with *overview*, *diagnostic measures*, *causes*, *preventive measures*, and *treatments*.

* **Distribution:** The graph confirms its diagnostic orientation, with *symptom nodes* (10,520) and *HAS_SYMPTOM* relationships (15,247) being dominant.

- **QA System Development:** To develop the system the author uses Rasa Framework, an open-source machine learning platform for building text- and voice-based conversational agents. The . The development process involves:
 - **Intent Classification & NER:** User queries were categorized into six types: disease diagnosis, disease overview, causes, treatment, prevention, and symptoms. Leveraging Rasa’s NLU, the system was tailored for Vietnamese by incorporating *ConveRTTokenizer* and *ConveRTFeaturizer* in the processing pipeline. Each intent was trained with 10–20 sample questions
 - **Query Processing:** The system uses two data sources: Disease diagrams (highly reputable, prioritized first). Medical websites (used if the symptom is not in the diagrams) The system first checks if the user’s symptoms match the typical symptom of any disease diagram. If yes, it follows the diagram’s sequential logic (YES/NO questions) to a diagnosis. If the symptoms are not in a diagram, the system queries the general KG for matching diseases. Crucially, it uses a node similarity algorithm to find symptoms that frequently co-occur with the user’s input. It then asks the user about these related symptoms to refine the diagnosis iteratively. This mimics a doctor’s reasoning.
 - **Answer Formulation:** Once the question type (intent), entities, and attributes are identified, the system uses predefined templates to generate answers. This Answers are matched with the appropriate template and returned in a structured, user-friendly format. The figure below summarizes the workflow used to find the best answer:

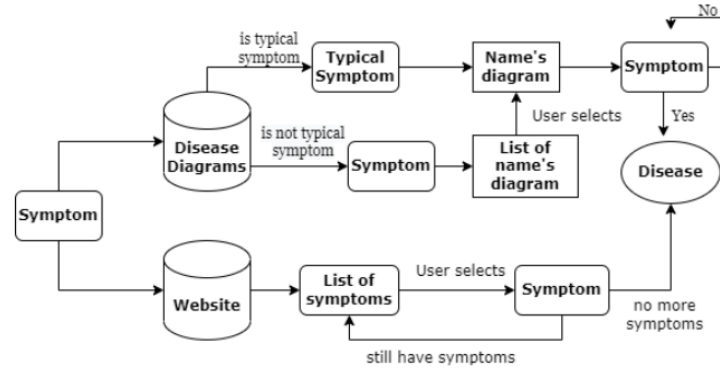


Figure 6.22: Diagram about Query Processing for finding the best answer for the question of disease diagnosis. [58]

Experiment and result:

- **NER Model Performance:** Five state-of-the-art models were benchmarked for Vietnamese medical NER:

- ACE (XLM-R embedding): 65.7% F1
- ACE (GloVe embedding): 70.3% F1
- Fine-tuned BERT: 66.4% F1
- Fine-tuned XLM-R-large: 66.7% F1
- **Fine-tuned PhoBERT: 76.3% F1 (best result)**

As expected, PhoBERT, trained specifically for Vietnamese, outperformed multilingual and English-based models, and was adopted for NER in the QA pipeline.

- **Knowledge Graph Reliability:** The trustworthiness of the graph was validated via link prediction. Several machine learning models were used:
 - Logistic Regression: 74.46% AUC-ROC
 - Random Forest: 92.89% AUC-ROC
 - **LightGBM: 98.26% AUC-ROC (best result)**

These results confirmed the graph’s strong internal consistency.

- **QA System Demonstration:** The complete system, integrating the knowledge graph and Rasa framework, was deployed and tested through a prototype dialogue system. A sample conversation on disease investigation demonstrated the system’s ability to recognize intents, extract entities, and provide accurate responses.

6.3.6 Building an intelligent diabetes Q&A system with knowledge graphs and large language models

In this paper [59], the authors build an intelligent diabetes question-answering (QA) system by integrating structured knowledge graphs with large language models (LLMs), aiming to deliver personalized medical information to diabetic patients. The motivation stems from the rising global prevalence of diabetes and the need for accurate, tailored healthcare advice that existing platforms can’t adequately provide, especially for complex medical queries. The developed system follows a multi-component architecture, as illustrated in the figure below:

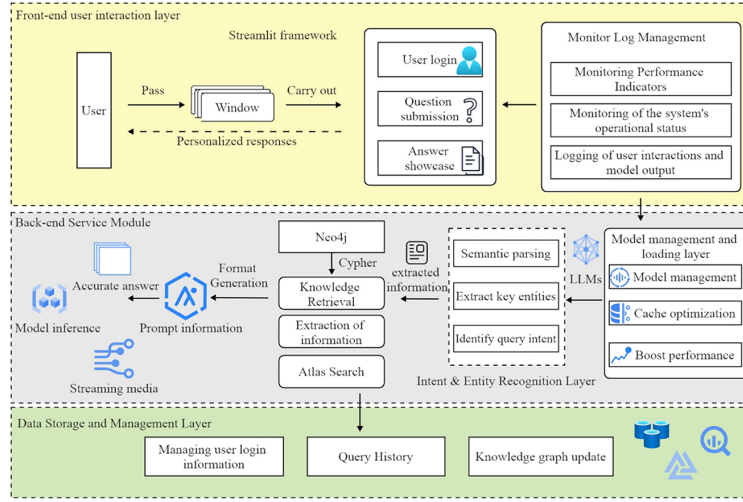


Figure 6.23: Full System Architecture of the Diabetes Question-Answering System [59]

- **Intent Recognition (IR):** To classify the user’s question into one or more medically relevant intent categories, such as: Disease overview Causes Symptoms Treatment methods Recommended foods, restricted foods, required medications, necessary examinations, concurrent diseases the authors uses the Qwen2.5-7B large language model operated via prompt-based learning. Prompts are crafted such that the model can perform intent recognition directly, without extra fine-tuning.
- **Named Entity Recognition (NER):** the purpose of this step is to extract key medical entities from the user query—such as diseases, symptoms, treatments, medications, body parts, examination indicators, etc. The study proposed a hybrid NER framework by integrating Prompt-based Learning with Low-Rank Adaptation (LoRA) on the Baichuan2-13B model.

- **Prompt Template Design:** Explicit task guidance was provided through carefully crafted prompts. The system instructed the model to extract predefined medical entities such as *medications*, *symptoms*, *treatment methods*, *causes*, *pathogenesis*, *examination indicators*, *body parts*, and *clinical manifestations*.

Example instruction:

“You are an expert in diabetes entity extraction. Extract entities such as ‘symptoms’, ‘treatment methods’, and ‘medications’ from the input and return results in JSON format.”

The input structure was formalized as the concatenation of prompt and text:

$$X' = [P; X]$$

The training objective was to maximize the conditional probability of correct entity predictions:

$$\hat{y} = \arg \max_y P(y | X', \theta)$$

- **LoRA Fine-tuning:** LoRA was applied to adapt the Baichuan2 model efficiently to the medical domain. Weight matrices were decomposed into low-rank factors:

$$W' = W + \Delta W, \quad \Delta W = AB, \quad r \ll \min(d, k)$$

This decomposition reduced the number of trainable parameters, thereby lowering computational and memory costs. Optimization was performed using the cross-entropy loss function:

$$L = - \sum_{i=1}^N t_i \log(y_i)$$

LoRA ensured fast and resource-efficient adaptation to diabetes-specific texts

- **Construction of the Diabetes KG:** The authors constructed a **diabetes knowledge graph** using Neo4j, where nodes represent diseases, symptoms, drugs, and attributes, while edges denote their relationships. Heterogeneous data sources were integrated, and disease nodes were enriched with causes, treatments, and overviews. Neo4j’s capabilities, including the Cypher query language, scalability, and visualization tools, enabled efficient storage, retrieval, and interpretation of medical data. The resulting graph provides a **holistic view of diabetes management**, linking entities such as *Type 2 Diabetes*, *Blood Glucose*, *Metformin*, and related symptoms, thereby supporting both clinical diagnostics and research. The figure below represents the different entities and relationships in the KG:

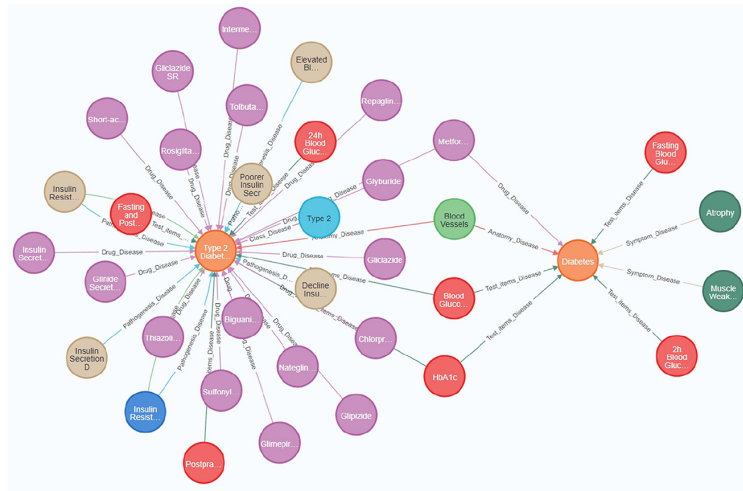


Figure 6.24: Diabetes Knowledge Graph: Entities and Relationships. [59]

Methodology & System Architecture The system’s workflow is a pipeline:

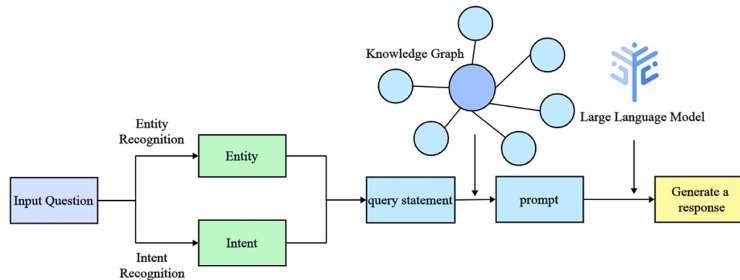


Figure 6.25: Workflow diagram of the intelligent question-answering (QA) system for diabetes. [59]

1. **User Input:** A patient asks a question (e.g., “What are the symptoms of diabetes?”).

2. Semantic Parsing:

[label=)]**Entity Recognition:** A LoRA-fine-tuned Baichuan2-13B model extracts key medical terms (e.g., “diabetes,” “metformin,” “neuropathy”) using carefully designed prompts. **Intent Recognition:** A Prompt-based Qwen2.5-7B model classifies the user’s goal (e.g., query symptoms, query treatment methods).

- (3) **Knowledge Retrieval:** The identified entities and intent are used to form a Cypher query to retrieve precise information from the Neo4j diabetes knowledge graph.
- 4. **Answer Generation:** The retrieved knowledge is fed back into an LLM as a context-rich prompt to generate a natural language, accurate, and personalized response.

Experiments & Results For NER, the authors used the CCKS2021 DiaKG dataset (41 diabetes documents, 22k entities) with GPT-4 data augmentation, and their LoRA-Prompt-Baichuan2 model achieved the best F1-score of 83.17. For intent recognition, they built a custom 9-intent dataset (also augmented with GPT-4), where the Prompt-Qwen2.5 model outperformed baselines with an F1-score of 86.48 without additional training.

6.3.7 Discussion

This study [54] utilizes a KG to structure medical concepts, capturing both positive and negative relationships with weighted edges to support multi-hop reasoning. The KG enables the system to perform intention classification—distinguishing between queries focused on “diagnosis” versus “treatment”—with an accuracy of 93.89, thereby accommodating a wide range of medical question types. Weighted path-ranking was shown to outperform Bayesian and case-based methods, achieving 86.4 accuracy for diagnosis tasks. The KG was constructed from 182,638 medical instructions, and the resulting QA system was evaluated on 149,998 QA pairs, demonstrating strong real-world applicability.

However, the system is limited in handling multi-intent queries, such as those combining causes and treatments (e.g., “What causes X, and how to treat it?”). Additionally, it relies heavily on predefined relationships within the KG; if the graph lacks certain important links, the system fails to generate accurate answers, even when such information exists in real-world clinical knowledge. Furthermore, the KG is built using Chinese medical data, which may restrict its generalizability across different languages and regional medical practices. This paper

[55] presents a medical question answering system built on a KG and employs rule-based classification combined with string-matching algorithms, such as Aho-Corasick, to process natural language queries. The system features a large-scale KG containing seven entity types (e.g., diseases, drugs) and eleven relationship types. The use of the Aho-Corasick algorithm enables fast multi-pattern string matching, while hybrid similarity metrics—including Jaccard, TF-IDF, and cosine similarity enhance the system’s ability to handle partial or noisy queries.

Despite its strengths, the system faces several limitations: it struggles with multi-intent queries and lacks support for multi-hop reasoning, which constrains its ability to handle more complex information needs. The system is also built using Chinese medical data, limiting its applicability to other languages or medical contexts. Furthermore, its reliance on predefined question templates restricts its flexibility, and it often falls back on a QA-pair database for answer precision, a method that does not scale efficiently when confronted with novel or diverse queries.

This research [56] focuses on developing a Traditional Chinese Medicine (TCM) case-based question-answering (QA) system by integrating LLMs with KGs. The system automates the extraction of key entities from TCM case records and supports natural language queries, enabling efficient retrieval and analysis of TCM knowledge. By combining the natural language understanding capabilities of LLMs with the structured reasoning power of KGs, the system enhances answer accuracy and significantly reduces hallucinations. It demonstrated strong performance, achieving high RAGAS scores—such as 0.9375 for faithfulness and 0.9686 for answer relevance and employed models like GLM-3-Turbo for named entity recognition (NER), achieving a high F1 score of 0.9617 and thereby minimizing manual annotation efforts.

However, the knowledge graph was constructed using only 679 cases from a single physician, Wang Zhongqi, which may limit the diversity and generalizability of TCM practices represented. Additionally, entity retrieval encountered challenges: obscure or specialized TCM terms sometimes failed to yield correct responses due to the LLMs’ limited domain-specific training, and semantically equivalent entities (such as symptoms) extracted by the system did not always align exactly with KG entries, leading to failed queries and incomplete answers for users.

This work [57] focuses on constructing a knowledge graph to organize Hepatitis B-related knowledge—covering symptoms, treatments, and dietary recommendations—by integrating real-world Traditional Chinese Medicine (TCM) medical records from 2,573 patients with structured web data from platforms like Xunyiwenyao. To ensure consistency and reliability in entity and relationship mapping, TCM terminology was standardized using authoritative references such as Diagnostics of TCM and the Traditional Chinese Medicine Pharmacopoeia. The resulting knowledge graph supports a question-answering system that allows users to interact via natural language queries. The system employs AI techniques, including a Bi-LSTM combined with CRF for named entity recognition, achieving a respectable F1-score of 0.771. For intent recognition, it uses template matching to classify queries and retrieve answers. The QA system demonstrated solid performance, achieving 81.67 accuracy in responding to Hepatitis B-related questions.

However, its reliance on predefined question templates limits flexibility, especially when handling complex or previously unseen query structures, and requires manual expansion of templates to support broader use case

This This work [58] introduces a hybrid methodology for constructing a medical knowledge graph by combining verified diagnostic diagrams with automatically extracted website data, thereby balancing reliability and coverage. Their system advances beyond keyword matching by using a node similarity algorithm to infer co-occurring symptoms, supports Vietnamese through fine-tuned PhoBERT for NER, and integrates Rasa for interactive, privacy-preserving dialogue management. However, the work lacks end-to-end quantitative evaluation of the QA system, relies heavily on manual diagram conversion which limits scalability, and depends on static knowledge with template-based responses. These limitations highlight the need for broader evaluation, automated graph construction, and more adaptive reasoning models in future research.

In previous studies, the method used to retrieve answers from KGs involved generating Cypher queries. However, this approach relies on exact matches, which presents several limitations. It fails in cases where links are missing due to incomplete edges, or when answers require indirect, multi-hop reasoning across the KG.

Another significant limitation is that the KGs used are static — they do not capture recent updates, emerging events, or newly acquired knowledge. As a result, they cannot be dynamically updated with new information, reducing their effectiveness over time.

This research [59] presents a technically robust proof-of-concept with a key strength in its innovative hybrid architecture. It effectively integrates a structured medical knowledge graph with large language models, using advanced techniques like LoRA fine-tuning and prompt-based learning to achieve high accuracy in understanding user questions while minimizing computational costs. The system delivers impressive quantitative results, outperforming standard models, and is designed with a practical user interface, showcasing strong potential for real-world application. Furthermore, the authors demonstrate proactive ethical consideration by using anonymized and synthetic data to protect patient privacy.

However, the study’s main limitations lie in its lack of real-world validation, as the system remains untested by patients or clinicians in actual clinical settings. Its knowledge graph is static and lacks a mechanism for automatic updates, threatening its long-term accuracy as medical science evolves. Finally, the model’s scope is limited to diabetes and its performance is dependent on carefully designed prompts, raising questions about its adaptability to more complex, multi-morbidity cases and generalizability to other medical domains.

6.3.8 Comparative analysis

The comparative table summarizes various approaches from the state of the art that focus on the development of medical question answering systems based on knowledge graphs, organized according to the following criteria

- Methods used for knowledge graph construction
- Data storage systems utilized
- Workflow for understanding user questions
- Mechanisms for answer inference and generation
- Data sources used to construct the knowledge graph
- Results obtained by each approach

Table 6.2: Comparative Overview of Medical KG-based QA Systems

Paper Title	KG Construction & Data Storage	Question Understanding	Answer Inference Mechanism	Data Set Used	Results
A Knowledge Graph-based Question Answering Method for Medical Domain	-Dictionary-based NER -Semantic template matching -Sentiment analysis(Weight assignment) ;Storage:MySQL, Neo4J	-Dictionary-based NER -Jieba tokenizer -IG-based feature selection -one-hot encoding	-Custom IG-based classifier -Weighted path-rankings (Random Walks + Entity Scoring)	YiFeng Pharmacy (2020): 182,638 medical instructions 149,998 Q&A pairs	diagnosis-accuracy=86.4% disease-coverage=95.7%
Research on Medical Question Answering System Based on Knowledge Graph	-NER,K-Means -Clustering+Jaccard Similarity (Knowledge fusion) -Neo4J	-Aho-Corasick algorithm -TF-IDF -Naive Bayes classifier -Jaccard similarity & Cosine similarity	-Cypher Query Language -Template-based answer generation	-Hospital records -pharma databases -web-scraped data	-Accuracy =81.2% -Recall= 79.2%
KG-based QA in TCM Diagnosis of Hepatitis B	Neo4J	-BLSTM + CRF -DL post-processing	-Word2Vec -Template-to-Cypher query generation -Template-based NLG	8,544 patient records (2009–2011); Web data from Xunyi-wenyao	Accuracy=81.67%
TCM Case-based QA with LLMs and Knowledge Graphs	Ontology design (Top-down schema); GLM-3-Turbo (LLM); Neo4J	-NER:GLM-3-Turbo (LLM)	-Prompted relation extraction -LLM-generated Cypher queries -LLM-generated final answers	679 cases from Wang Zhongqi (1923–1935)	-Answer-Relevancy=96.86% -Context-Recall=95.00% Faithfulness=93.75%

Table 6.3: Comparative Overview of Medical KG-based QA Systems

Paper Title	KG Construction & Data Storage	Question Understanding	Answer Inference Mechanism	Data Set Used	Results
A New Approach of Question Answering based on Knowledge Graph in Traditional Medicine	-Entity & relation extraction, -manual annotation -knowledge fusion -Neo4J	-Rasa framework, 6 intents, -PhoBERT for NER -symptom matching + node similarity	-Rule-based templates -diagram logic -KG query with symptom similarity	-Diagnostic disease diagrams (digitized) -Vietnamese medical websites (vinmec.com, nhathuoclongchau.com)	-NER F1: 76.3% -KG AUC-ROC: 98.26%
Building an intelligent diabetes Q&A system with knowledge graphs and large language models	-Prompt-based NER with LoRA (Baichuan2-13B) -heterogeneous data integration -Neo4J	-Qwen2.5-7B for intent recognition -LoRA-fine-tuned Baichuan2-13B for NER	-Cypher query retrieval -LLM-generated natural language response	-CCKS2021 DiaKG -diabetes documents -heterogeneous medical data	-NER F1: 83.17% -Intent recognition F1: 86.48%

6.4 Conclusion

The state-of-the-art review has shown that KGs hold strong potential for supporting medical decision-making, particularly through QA systems. Existing approaches rely on a variety of techniques for intent and entity recognition as well as information retrieval, including deep learning models, template-matching methods, and more recently, large language models (LLMs). In parallel, hybrid approaches that integrate embeddings with deep learning have demonstrated their value for enhancing KG reasoning and prediction.

Despite this progress, current solutions still face significant challenges, such as incomplete graphs, limited clinical depth in patient-oriented systems, and the lack of interactive and explainable interfaces tailored for clinicians.

Building on these insights, the development process proposed in this thesis is directly informed by the reviewed approaches. It incorporates proven techniques for KG construction, question understanding, and reasoning, while integrating recent advances such as LLMs for improved intent and entity recognition, KG embedding models for enhanced reasoning and knowledge retrieval, and NLP methods for processing unstructured clinical data.

Furthermore, the proposed system will be integrated into a medical platform to assist doctors in clinical decision-making and connected with Electronic Medical Records (EMRs) to enable more personalized and context-aware support. By grounding the methodology in established research and extending it with these novel contributions, this thesis aims to deliver a clinically relevant QA system that is both usable and robust.

Part IV

Q&A System Development and KGs Construction

CHAPTER 7

KGS CONSTRUCTION AND REPRESENTATION LEARNING

7.1 Introduction

This chapter presents the construction of the KG and the application of representation learning techniques. It explains the process of data acquisition, ontology modeling, and preprocessing, followed by embedding models and training strategies. The goal is to build a structured KG and evaluate models for effective link prediction and analysis.

7.2 KG Construction

This section presents the process of constructing a domain specific medical KG focused on cardiology .The objective is to represent cardiology related medical knowledge in a structured,machine-readable format enabling its application in intelligent systems such as question answering and clinical decision support tools.

The construction of the KG involves several key steps,including:**Data collection,Data Pre-processing,Schema Design and storage** Each of these steps played a crucial role in ensuring the accuracy, relevance, and usability of the resulting knowledge graph

7.2.1 Data Source Selection and Acquisition

The first phase of constructing the cardiology-specific KG involved identifying and selecting high-quality, trustworthy data sources that provide comprehensive medical information.To ensure relevance and reliability, I focused on reputable medical websites that offer detailed content on cardiovascular domain. To automate the data collection process I use web scraping technique to extract both structured and unstructured data from these sources.The medical websites used as primary data sources include:

- **Victor Chang Cardiac Research Institute Website:**This website provides comprehensive information on cardiovascular diseases, including detailed descriptions of various conditions, their associated symptoms, risk factors, treatment options, prevention strategies, and diagnostic methods.
- **Drugs.com:**This platform offers detailed, independent information on over 24,000 prescription medications, over-the-counter drugs, and natural products. For this project,

the focus was on drugs used in the treatment of cardiovascular diseases, including information on their indications, usage guidelines, warnings, side effects, and similar or alternative medications.

- **Mayo Clinic Official Website:** This website provides user-friendly guides on hundreds of diseases, symptoms, diagnostic tests, medical procedures, medications, and supplements. In this project, the focus was limited to content related specifically to cardiovascular and heart-related diseases.

For the Web Scaping , two main Python libraries were utilized:

- **Requests:** is a powerful and user-friendly HTTP library that simplifies the process of sending HTTP requests. It was used to send GET requests to the selected medical websites and retrieve their HTML content efficiently.
- **BeautifulSoup:** is a parsing library designed to extract data from HTML and XML files. It enables easy navigation and extraction of specific elements within the HTML structure using tag names, classes, and attributes, making it ideal for web scraping tasks.

7.2.2 Ontology Schema Design and Modeling

The ontology design phase focuses on representing the medical concepts and the relationships between them. The schema was constructed manually based on a detailed and analytical observation of the collected data. By examining the structure and content of the extracted information—such as disease descriptions, symptoms, treatments, and risk factors—we identified the key concepts and their relationships relevant to the cardiology domain.

The schema was defined in a way that ensures semantic consistency and supports future integration with external medical ontologies if needed. For example, the main classes include Disease, Symptom, Treatment, Risk Factor, Complication, and Prevention, with relationships such as hasSymptom, treatedBy, hasRiskFactor, and canPrevent.

The ontology was designed manually rather than using existing medical ontology standards. This choice was motivated by the need for greater accuracy, integration of domain-specific expertise, and variability of the collected data formats. A manual approach allowed us to filter irrelevant information, ensure consistency in terminology, and tailor the ontology to the specific requirements of the cardiology knowledge graph.

As a result, the main medical concepts obtained are :**Disease, Medication, Treatment, Risk Factor, Side Effect, Complication, and Warning.**

The following table presents the main relationships linking these medical concepts, along with the definition of each relationship. The following figure represents the cardiovascular ontology schema obtained, illustrating the main concepts and their relationships within the cardiology domain. Figure X – Cardiovascular ontology schema

7.2.3 Data Preprocessing and Entity Linking

This stage involves preparing the raw information extracted from the medical websites in order to form triples for integration into the KGs and to enable training in the embedding model. Since the data was obtained through web scraping, it contained inconsistencies such as formatting variations, duplicate entries, and irrelevant text. Therefore, the following preprocessing steps were applied:

Table 7.1: Relationships between medical concepts and their definitions

Relationship	Definition
hasSymptom	Links a disease to its related symptoms.
treatedBy	Links a disease to its treatments or medications.
hasRiskFactor	Links a disease to its risk factors.
mayCauseComplication	Links a disease to its possible complications.
hasSideEffect	Links a medication or treatment to its side effects.
hasWarning	Links a medication or treatment to important warnings.
canPrevent	Links a prevention method to a disease.

1. **JSON to CSV transformation:**First, the JSON file containing the extracted medical information was transformed into a CSV file with three columns: Source, Relationship, and Entity. In this transformation, each main object in the JSON file (e.g., "Angina") was considered the source entity. Inside each object, the key–value pairs were reorganized so that:

The key represents the relationship (e.g., "Symptoms", "Causes", "Treatment"). The value represents the target entity or a list of entities, which were separated and stored as individual entries in the CSV file.

This structure allowed the extracted knowledge to be represented as triples in the format:(Source, Relationship, Entity)

2. **Data Cleaning** Since the collected data was in paragraph form rather than separate statements, most of the preprocessing was applied to the *target* column. The key preprocessing steps are as follows:

- **Remove HTML and Special Characters:** Strip HTML tags (e.g., <p>,
) and non-printable characters. Remove symbols such as \n, \t, "•", and unnecessary punctuation.
- **Standardize Text Casing:** Transform all text into lowercase to ensure uniformity.
- **Stopword and Noise Removal:** Remove irrelevant words or phrases such as "read more" or "click here," as well as common stopwords (e.g., *and*, *or*, *that*). Remove non-informative statements such as "This section introduces the main common symptoms..." or other introductory/navigation text. Remove unnecessary parentheses and example statements. Remove abbreviations by replacing them with their full forms (e.g., *electrodiagramme (ECHO)* → *electrodiagramme*).
- **Splitting Compound Entities:** Break multi-entity strings into separate entities (e.g., "fever and fatigue" → "fever", "fatigue").
- **Named Entity Recognition (NER):** Apply a medical-domain Named Entity Recognition process to the cleaned text, using a medical dictionary to identify and extract relevant entities such as diseases,
- **Duplicate Removal:**Remove repeated triples (same source, relationship, and entity).
- **Handling Missing or Incomplete Data:**Remove triples with empty entity fields.

3. **Graph Construction and Connectivity Analysis:** At this stage, the preprocessing steps have produced a set of triples that form the foundation of the KG. Before proceeding further, it was necessary to **analyze the graph’s connectivity**. Using the **NetworkX** Python library, the triples were assembled into a graph structure. The connectivity analysis revealed the presence of **8 disconnected subgraphs**, as summarized in Table 7.2.

Table 7.2: Connected components detected in the KG

Component	Number of Triples
1	5328
2	1
3	1
4	1
5	1
6	1
7	1
8	1

To ensure **coherence and relevance**, only the **largest subgraph** was retained, as the smaller ones contained insufficient or irrelevant information for the intended application.

After preprocessing, we obtained a medical KG containing 5,328 triples, 2,942 unique entities, and 13 unique relationships.

7.2.4 Visualization and Exploration

To visualize the KG, Neo4j was used as a graph database and storage solution, enabling the exploration of entities and their relationships. The figure below illustrates how the different entities are interconnected.

7.3 Embedding and Representation Learning

In order to capture the semantic meaning of entities and relations within the constructed KG, we employ KGEs models. The central idea behind these models is to map each entity and relation into a low-dimensional vector space, where their latent semantic properties can be effectively represented.

Formally, each entity $e \in E$ is represented by a vector $e \in \mathbb{R}^d$, and each relation $r \in R$ is represented either as a vector or as a transformation in the same space. Given a triple (h, r, t) , the embedding model defines a scoring function $f(h, r, t)$ that assigns higher scores to valid triples and lower scores to corrupted (invalid) ones.

Such representations support a wide range of applications, including recommendation systems, KG completion, and link prediction. In this work, our focus is on the link prediction task. Specifically, we begin by constructing a sub-KG from a given set of entities and relationships. The objective is then to predict the missing parts of triples in order to complete the sub-KG and extract relevant answers.

This section outlines the main steps followed and the different models applied in order to identify the most suitable one, based on specific evaluation metrics.

7.3.1 Embedding Models

The constructed medical KG is large and complex, containing one-to-many and many-to-many relationships. Therefore, we focused on models that are capable of handling this type of KG structure. To evaluate their performance, we tested several models from different categories of KGEs approaches. These models include:

- **Translational Distance Models:**TransH, and TransR.
- **Semantic Matching / Bilinear Models:**DistMult
- **Geometric / Rotation-based Models:**RotatE
- **Neural Network-based Models:**GAT,R-GCN and R-GCN+GAT

7.3.2 Experimental Setup and Training

Dataset: For training the embedding models, the constructed KG was not used in its raw form. Instead, we enriched it by adding **inverse triples** for every existing triple. This process involved swapping the head and tail entities while assigning an additional inverse relation to capture the opposite direction of the interaction.

The inclusion of inverse triples is crucial for KGE models, as it allows them to better capture bidirectional semantics, improve link prediction performance, and reduce bias towards one-way relational patterns. By integrating both directions of a relation, the models gain a more comprehensive understanding of the underlying structure of the KG.

As a result of this augmentation, the dataset grew to a total of **13,206 triples** and **27 unique relations**. The mapping between original and inverse relations is summarized in Table 7.3.

Table 7.3: Mapping of original relations to their inverse relations.

Original Relation	Inverse Relation
HAS_Treatment	Treatment_For
HAS_Procedure	Procedure_For
HAS_Medication	Medication_For
HAS_Prevention	Prevents
HAS_Alternative_Medication	Alternative_To
Treats	Treated_By
Contraindicated_With	Contraindicated_For
MAY_Cause	Caused_By
HAS_Symptom	Symptom_Of
Diagnosed_BY	Diagnoses
MAY_Lead_To	Complication_Of
Caused_BY	Cause_Of
HAS_Risk_Factors	Risk_Factor_For
HAS_Type	Type_Of

Training Configuration:

- For model training, the dataset was divided into three subsets following a standard split ratio of 80 training, 10 validation, and 10 testing.
- During training, we focused on optimizing the main hyperparameters that significantly influence model performance, including:
 - Embedding dimension size
 - Learning rate and optimizer
 - Regularization methods
 - Negative sampling strategy
 - Number of training epochs
 - Batch size.
- The implementation leveraged two libraries:
 - **PyKEEN**: which provides a comprehensive collection of state-of-the-art KGE models.
 - **Torch Geometric**: which offers graph neural network (GNN)-based architectures and was used to support experiments involving graph-based embeddings.

Models Training:

- **TransH**
 - **Overview**: TransH extends TransE by projecting entities onto a relation-specific hyperplane before applying translation. This helps capture one-to-many and many-to-one relations more effectively.
 - **Training Configuration**:
 - * Embedding dimension: 100
 - * Optimizer: Adam, learning rate = 0.001
 - * Loss: Margin ranking loss (margin = 1.0)
 - * Batch size: 1024
 - * Negative sampling: 1:5 ratio
 - * Epochs: 500
- **TransR**
 - **Overview**: TransR further extends TransE/TransH by learning separate entity and relation spaces. Entities are projected into the relation space using a projection matrix, allowing better handling of complex relational patterns
 - **Training Configuration**:

- * Embedding dimension: 100
- * Optimizer: Adam, learning rate = 0.005
- * Loss: Margin ranking loss (margin = 1.0)
- * Batch size: 512
- * Negative sampling: 1:10 ratio
- * Epochs: 600

- **DistMult**

- **Overview:** DistMult represents relations as diagonal matrices, enabling efficient training. It works well for symmetric relations but struggles with asymmetric ones.
- **Training Configuration:**
 - * Embedding dimension: 50
 - * Optimizer: Adam, learning rate = 0.005
 - * Loss: Binary cross-entropy
 - * Batch size: 1024
 - * Negative sampling: 1:5 ratio
 - * Epochs: 600

- **RotatE**

- **Overview:** RotatE models relations as rotations in the complex plane, making it capable of representing symmetry, antisymmetry, inversion, and composition.
- **Training Configuration:**
 - * Embedding dimension: 100
 - * Optimizer: Adam, learning rate = 0.005
 - * Loss: Margin ranking loss (margin = 6.0)
 - * Batch size: 512
 - * Negative sampling: 1:10 ratio
 - * Epochs: 600

- **RGCN**

- **Overview:** R-GCN extends standard GCNs to handle multi-relational knowledge graphs. It introduces relation-specific transformations for each edge type, allowing the model to capture heterogeneous connectivity patterns across entities and relations. This makes R-GCN particularly well-suited for knowledge graph embedding tasks such as link prediction and node classification.
- **Training Configuration:**

- * **Embedding dimension:** 50
- * **Number of layers:** 2
- * **Hidden dimension:** 100
- * **Dropout:** 0.3
- * **Optimizer:** Adam, learning rate = 0.001
- * **Batch size:** 256
- * **Number of epochs:** 500
- * **Loss:** Cross-entropy (PyKEEN default for R-GCN)
- * **Negative sampling:** Uniform, ratio = 1:1 (default in PyKEEN)

- **GAT+RGCN with RotatE Scoring**

- **Overview:** The **GAT-RGCN with RotatE** architecture is a hybrid model designed for link prediction in knowledge graphs. It combines GAT to assign attention weights and emphasize the most relevant neighbors, RGCN to capture relation-specific dependencies, and the RotatE scoring function to represent relations as rotations in complex space. This combination leverages both structural features of the graph and semantic properties of relations, producing more expressive and robust entity embeddings.
- **Training Configuration:**
 - * **Embedding dimension (entity):** 200 (internally doubled to 400 for complex embeddings)
 - * **Hidden dimension (GNN layers):** 100
 - * **Dropout:** 0.5 (applied after GNN layers)
 - * **GAT heads (first layer):** 2
 - * **Optimizer:** Adam with learning rate = 0.001 and weight decay = 1e-5
 - * **Loss function:** Margin Ranking Loss with margin = 1.0
 - * **Training epochs:** 500
 - * **Batch size:** 32
 - * **Negative sampling:** 5 negatives per positive triple
 - * **Gradient clipping:** max_norm = 1.0
 - * **Device:** CUDA (if available), otherwise CPU

- **GAT**

- **Overview:** GAT is a graph neural network that updates node embeddings by attending to their neighbors with different importance weights. Unlike simple averaging in GCNs, GAT learns attention scores that highlight the most relevant neighbors during aggregation. In our setup GAT is Used RotatE-style scoring function
- **Training Configuration:**

- * **Entity embeddings:** complex vectors of dimension $2 \times d$.
- * **Relation embeddings:** phase angles in $[-\pi, \pi]$.
- * **Architecture:**
 - First GAT layer: $\text{GATConv}(2d \rightarrow h, \text{heads} = H, \text{dropout} = p)$
 - ReLU activation + dropout
 - Second GAT layer: $\text{GATConv}(hH \rightarrow 2d, \text{heads} = 1, \text{dropout} = p)$
- * **Training strategy:**
 - Self-adversarial negative sampling with $K = 16$ negatives.
 - Optimization with Adam, learning rate 1×10^{-3} .
 - Margin parameter $\gamma = 12.0$.
 - Number of epochs: 900, batch size 256.

Training & Setup:

- Environment: Google Colab notebook with GPU acceleration enabled.
- Hardware: NVIDIA Tesla T4 GPU (16 GB VRAM) and 12 GB system RAM.
- Software: Python 3.10, PyTorch backend.
- Libraries: PyKEEN (for KGE models), Torch Geometric (for GNN-based models).
- Training: Executed in GPU runtime to ensure efficient optimization and reduced training time.

7.3.3 Evaluation and Comparison

For each model, performance was assessed on the link prediction task. Given a test triple, the missing entity was predicted by ranking all possible candidates according to the model’s scoring function. The quality of the rankings was then measured using standard evaluation metrics, including **Mean Reciprocal Rank (MRR)**, **Hits@k** ($k = 1, 3, 5, 10$) and **Mean Rank (MR)**.

The following table presents the different trained models and the results for each evaluation metric.

Model	Hits@1	Hits@3	Hits@5	Hits@10	MR	MRR
TransH	0.30	0.42	0.48	0.56	46	0.39
TransR	0.037	0.097	0.15	0.26	189	0.10
DistMult	0.35	0.49	0.57	0.67	19	0.46
RotatE	0.73	0.85	0.89	0.93	11	0.80
R-GCN	0.28	0.46	0.55	0.67	44	0.42
GAT+R-GCN	0.48	0.66	0.72	0.80	185	0.59
GAT	0.44	0.57	0.62	0.70	31	0.53

Table 7.4: Comparison of different models on link prediction task using Hits@ k , Mean Rank (MR), and Mean Reciprocal Rank (MRR).

7.3.4 Discussion of Results

The experimental results provide insights into the relative strengths of different knowledge graph embedding (KGE) and graph neural network (GNN) models for link prediction:

- **RotatE:** Achieves the best overall performance across all metrics, with the highest Hits@ k scores (Hits@5 = 0.89, Hits@10 = 0.93) and the best ranking metrics (MR = 11, MRR = 0.80). This confirms its strong ability to model complex relation patterns such as symmetry and inversion.
- **DistMult:** Shows competitive performance with moderate Hits@ k values and a relatively low mean rank (MR = 19). However, it struggles with more complex relation types due to its bilinear formulation.
- **TransH vs. TransR:** TransH outperforms TransR significantly (MRR = 0.39 vs. 0.10), indicating that projecting entities onto relation-specific hyperplanes provides more expressive embeddings than TransR’s relation space projection in this dataset.
- **R-GCN:** Provides balanced performance with reasonable Hits@10 (0.67) but suffers from higher MR (44), suggesting it captures relational neighborhood structure but may not fully optimize ranking precision.
- **GAT and GAT+RGCN:** The hybrid GAT+RGCN achieves higher Hits@10 (0.80) and MRR (0.59) compared to standalone GAT (MRR = 0.53). However, its mean rank (MR = 185) is worse, likely due to oversmoothing or training instability. GAT alone shows better MR (31), but slightly weaker overall accuracy.
- **General Observation:** KGE models (e.g., RotatE, DistMult) outperform GNN-based methods in ranking metrics, while GNN-based models (GAT, GAT+RGCN) capture structural information but face challenges in optimizing MR.

Overall, **RotatE emerges as the most robust model** for this task, consistently achieving the best performance across all evaluation metrics. Given its ability to capture complex relation patterns and its superior ranking results, we selected RotatE as the final model for the link prediction task. In contrast, GNN-based approaches (GAT, R-GCN, and GAT+RGCN) show potential but require further optimization or hybrid integration to match the performance of RotatE.

7.4 Conclusion

This chapter described the complete pipeline of KG construction and embedding-based representation learning, covering data preparation, ontology design, model training, and evaluation. The results showed varying strengths and limitations across models, with **RotatE emerging as the most effective model** for the link prediction task. This model is therefore selected as the preferred approach for downstream applications.

8.1 Introduction

This chapter describes the architecture and operational flow of the Q/A system. The design follows a modular approach, where each stage performs a specific function in transforming a user's question into an accurate, context-aware answer. This section will detail each step. This architecture can handle a wide range of user questions, retrieve relevant knowledge, and present results in a clear, reliable manner.

8.2 Natural Language Processing (NLP) Pipeline

The following figure illustrates the different steps employed for question handling.

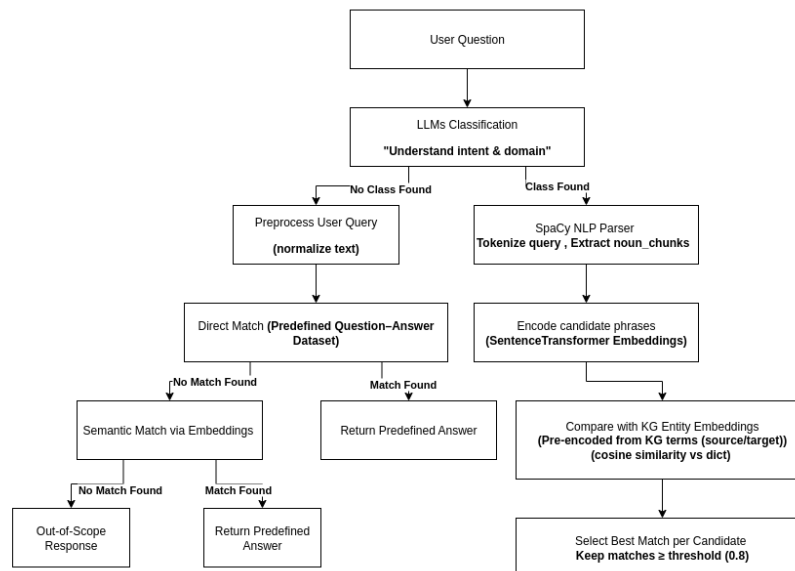


Figure 8.1: The different steps employed for question handling.

8.2.1 Multilingual Question Handling

Our goal is to develop a multilingual Q/A system. Since the collected dataset is primarily in English, we adopt the following process to ensure support for multiple languages:

- **Language Detection** – Identify the language of the user’s query.
- **Translation to English** – Translate the query into English, which serves as the system’s main processing language.
- **Answer Generation** – After retrieving the relevant information, generate the answer in English.
- **Translation to User’s Language** – Translate the final answer back into the user’s original language before displaying it.

To address this, we integrated the `Googletrans` library to manage both language detection and translation. Specifically, the `translator.detect()` method was employed to identify the language of the user’s query and provide a confidence score for the detection. Once the query language was determined, the `translator.translate()` method was applied either to translate the query into English or into the user’s language to display an appropriate answer.

8.3 Question Understanding Pipeline

8.3.1 Entity Recognition & Extraction

The first step, after translating the user’s question into English, is to identify the main medical entities in order to later construct the subgraph. For this purpose, we use embedding-based methods.:

1. **Text Preprocessing & Candidate Phrases:** We first applied `spaCy`’s `en_core_web_sm` model to parse the input question and extract noun chunks (e.g., “chest pain”, “high fever”). These noun chunks serve as candidate medical entities.
2. **Entity Dictionary Representation:** A medical terminology dictionary was prepared from the collected dataset. Each term in the dictionary was embedded into a dense vector space using `SapBERT` (`cambridgelt1/SapBERT-from-PubMedBERT-fulltext`).
3. **Semantic Matching with Sentence Transformers:** For each candidate phrase, we generated embeddings using the `SapBERT` model. We then computed cosine similarity between the candidate embeddings and the dictionary embedding vectors using `SentenceTransformers`’ `util.cos_sim`. Cosine similarity is a metric that measures the angle between two vectors, producing a value between -1 and 1.
4. **Threshold-Based Matching:** If the similarity score exceeded a predefined threshold (0.8 in our experiments), the candidate phrase was accepted as a medical entity and linked to the closest dictionary entry.

This approach allows us to extract medical events even when there is no exact word-level match, since embeddings capture the semantic meaning rather than surface-level similarity. Moreover, the method returns the recognized entities, their matched dictionary term, and the similarity score, ensuring robustness even with paraphrased or misspelled input (e.g., “heart attack” → “myocardial infarction”).

8.3.2 Handling Out-of-Context Questions

In the case of noun medical entity identification, we propose to return an appropriate answer instead of leaving the query unanswered. To achieve this, we designed a dataset containing pairs of questions and answers that a user might ask. This dataset will later serve as a foundation for generating meaningful answers. The overall process can be summarized in the following steps:

1. **Preprocessing:** Each incoming query is normalized by converting it to lowercase and removing punctuation, ensuring consistent text representation.
2. **Embedding In-Domain Questions:** A set of predefined, in-domain questions (collected from the dataset) is embedded into dense vector representations using the `all-MiniLM-L6-v2` model from `SentenceTransformers`.
3. **Exact Match Check:** If the preprocessed query exactly matches one of the predefined questions, the system directly retrieves and returns a stored answer without performing semantic similarity.
4. **Semantic Similarity Matching:** If no exact match is found, an embedding is generated for the user query and compared against the stored embeddings of in-domain questions using cosine similarity. The most similar predefined question is identified as the best match.
5. **Thresholding:** If the similarity score of the best match falls below a predefined threshold (0.5 in our experiments), the system classifies the query as *out-of-context*.
6. **Response Generation:**
 - If the query is recognized as in-domain, the system retrieves and returns an appropriate predefined answer.
 - If the query is out-of-context, the system returns one of several fallback responses, explicitly stating that it can only handle cardiology-related queries (e.g., “I can only answer questions related to CardioX and cardiology. This one seems outside my scope.”).

8.3.3 Question Classification & Relationship Identification

Once the input is identified as a medical question containing cardiovascular-related entities, the next step is to determine the user’s intent. In other words, the system must classify the question to identify the main relationship that will later be used to construct the sub-KG.

To achieve this, we employed a **prompt-based classification** approach using a large language model (LLM). Specifically, we utilized the `qwen/qwen3-coder:free` model, accessed via the OpenRouter API. The model is guided by a carefully designed prompt, instructing it to act as a medical assistant whose sole responsibility is to classify user questions into one or more predefined medical intents.

The supported intent categories are:

- **condition** — questions about the causes of symptoms or conditions,
- **test_used** — questions about diagnostic or screening tests,
- **treatment_option** — questions about treatment strategies for specific conditions,
- **prevention** — questions about prevention of a condition,
- **medication** — questions about a medication,
- **similar_drugs** — questions about alternative or related drugs,
- **none** — questions unrelated to medicine.

The model will return array that contain one or more intent labels (e.g., `["condition", "test_used"]`).

8.4 Embedding-Based Retrieval & KG Completion

This component is responsible for inferring and extracting relevant information from the KG using embedding models. After the medical entities and the underlying intent of the doctor’s query are identified, the embedding model is employed to complete missing information in a triple (*head*, *relation*, *tail*) or to validate whether an existing triple is semantically plausible. The process works as follows:

- **Tail Entity Prediction:** Given a pair consisting of a head entity and a relation ($h, r, ?$), RotatE predicts the most plausible candidate tail entities. Each candidate is assigned a plausibility score, and the top- k results are retained for analysis. *Example:* For the query (Aspirin, treats, ?), the model may infer entities such as *heart attack* or *angina*.
- **Relation Prediction:** For a given pair of head and tail entities ($h, ?, t$), the model estimates the most likely relations connecting them. Candidate relations are scored and ranked, with only those exceeding a predefined threshold considered meaningful. *Example:* For (Hypertension, ?, Beta-blockers), the model may infer the relation *treated_by*.
- **Triple Validation:** A complete triple (h, r, t) is evaluated using the RotatE scoring function. The plausibility score reflects the likelihood that the triple is valid, and a threshold criterion is applied to filter out incorrect or weak associations.

- **Evaluation Mechanism:** All predictions (entities or relations) are associated with model-generated scores. Higher scores correspond to stronger semantic plausibility. Thresholding ensures that only high-confidence predictions are retained, improving both reliability and interpretability of the Q/A system’s outputs.

By leveraging RotatE for embedding-based retrieval and KG completion, the system is able not only to retrieve explicit facts from the KG but also to infer hidden knowledge, thereby enabling richer and more context-aware responses to doctors’ queries.

8.5 Answer Generatios

The final stage of the Q/A pipeline focuses on generating an informative and human-readable response for the doctor. To achieve this, we integrate LLMs provided through the OpenRouter platform. These models are prompted with the doctor’s original query, enriched with the extracted entities and identified intent, to produce a coherent and clinically meaningful answer.

Example: Suppose a doctor submits the query: *“What could be the possible causes of chest pain and shortness of breath in a patient with a history of hypertension?”* After entity recognition (*heart failure, angina, myocardial infarction*) and intent classification (*condition*), the LLM generates:

“Chest pain and shortness of breath in a hypertensive patient may indicate several possible conditions, including angina, early-stage heart failure, or myocardial infarction.”

This approach ensures that the doctor receives not only a relevant but also a context-aware medical response that can directly assist in clinical decision-making.

8.6 Conclusion

This chapter described the full workflow of the Q/A system, from question understanding to answer generation. By combining NLP techniques with Knowledge Graph embeddings, the system is able to process complex medical queries and deliver reliable responses. This design provides a solid basis for implementation and integration into the healthcare platform.

The following figure illustrates the main components discussed in the previous two chapters, highlighting all aspects of our contribution

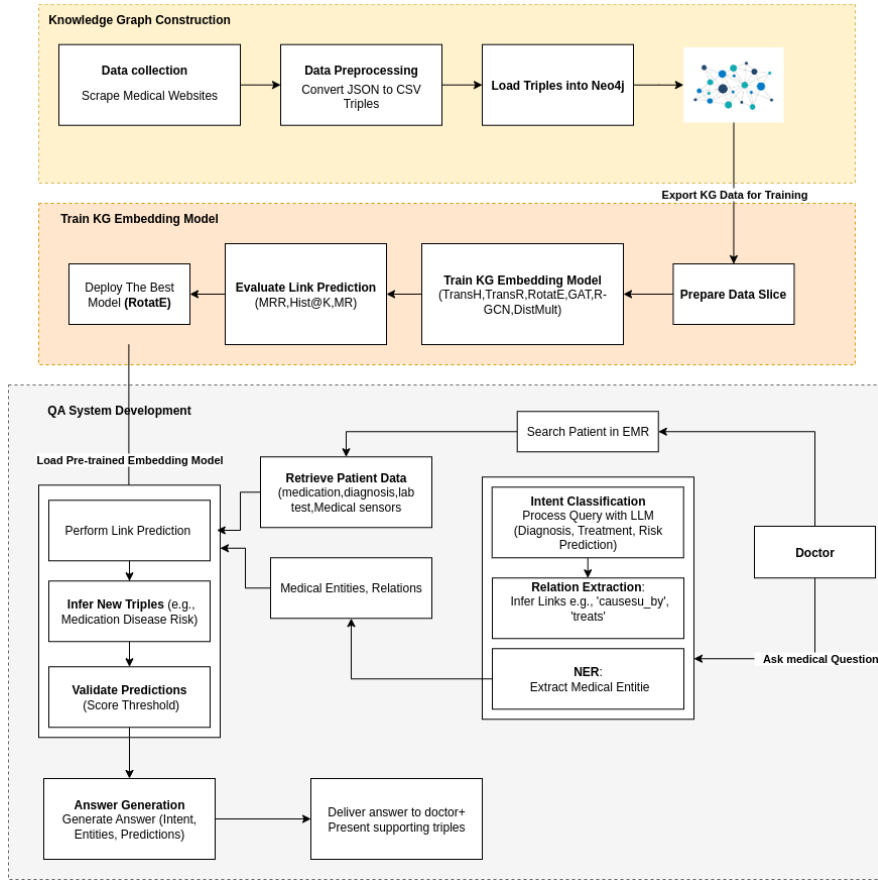


Figure 8.2: The figure below presents the key components designed and implemented in our work

Part V

Platform Development

9.1 Introduction

In this chapter, we define the basic functioning of the system **CardioX**. We begin by detailing the system's functional and non-functional requirements, specifying the services expected by both doctors, Hospital and patients. Next, we analyze the system through a set of scenarios that illustrate how each actor interacts with the platform. These include detailed use case descriptions, the identification of involved actors, and a use case diagram that collectively offer a structured view of the system's functionality.

9.2 Functional Requirements

The system includes several types of users, each with distinct roles and responsibilities. The following functional requirements are organized by actor.

9.2.1 Doctor

- User registration, secure authentication, and authorization.
- Subscribe to the platform and manage subscription payments.
- Query the integrated AI-based Q&A system for decision support.
- View and manage patient EMRs (diagnoses, prescriptions, history).
- Receive real-time patient vital signs from IoT sensors.
- Send alerts, messages, and appointment requests to patients.
- Track and manage appointments.

9.2.2 Hospital

- User registration, secure authentication, and authorization.
- Subscribe to the platform and manage subscription payments.
- Invite and manage doctors within the hospital.
- View analytics and usage reports.

9.2.3 Patient

- User registration and login.
- Connect the mobile app to IoT medical devices.
- View personal health records.
- Receive notifications and messages from the doctor.
- Schedule or confirm appointments

9.3 Non-Functional Requirements

- **Security and Privacy:** All data must be encrypted in transit and at rest. Compliance with health data regulations (e.g., HIPAA/GDPR).
- **Scalability:** The system should support many concurrent users and devices.
- **Availability and Reliability:** Real-time data delivery and system uptime are crucial.
- **Performance:** Low-latency communication between mobile, backend, and web interface.
- **Maintainability:** Modular design for easy updates and improvements.
- **Usability:** Intuitive UI/UX for both doctors and patients.

9.4 Use Cases

9.4.1 Use Case Descriptions

A cardiovascular patient, from the comfort of their home, uses connected IoT medical sensors — such as ECG, SpO₂, and thermometer devices — to continuously record their vital signs. These health metrics are displayed in real time on the patient's mobile application, which also issues alerts when abnormal values are detected.

The recorded data is automatically transmitted to the doctor via the platform. The doctor can remotely monitor and visualize these vital signs, which are analyzed and classified by the system according to severity levels (e.g., high, medium, low alerts). Based on this analysis, the doctor can:

- Send personalized medical advice
- Adjust the patient’s treatment plan.
- Prescribe new medication.
- Request further tests or analyses.
- Schedule or recommend a follow-up appointment

All messages and notifications are received by the patient in their mobile app. The patient is thus able to follow the doctor’s guidance, take appropriate action, and manage their upcoming appointments directly through the app.

When the patient visits the doctor for an in-person consultation, the physician begins with a traditional clinical evaluation. This includes performing a new ECG, checking vital signs, and asking the patient about any new symptoms or changes in their condition.

Following this, the doctor retrieves the patient’s profile and medical history from the Electronic Medical Record (EMR), now enriched with remote monitoring data. To enhance the decision-making process, the doctor may also use the integrated Q&A system to ask targeted questions based on the patient’s case. This includes exploring:

- Possible conditions that align with the patient’s data and symptoms,
- Appropriate treatment options,
- Relevant preventive measures.

The system provides AI-assisted suggestions based on the doctor’s queries and the up-to-date patient data.

Using this combined input clinical observations, remote data, patient dialogue, and Q&A insights the doctor is able to deliver a personalized and informed diagnosis or treatment plan. All new information, diagnoses, and prescriptions are then automatically updated in the patient’s EMR, ensuring comprehensive and continuous care.

9.4.2 Use Case Diagram

This figure presents the complete use case diagram, capturing the core functionalities of the system. To make the system easier to analyze and design, we will later divide it into subsystems and present detailed diagrams for each

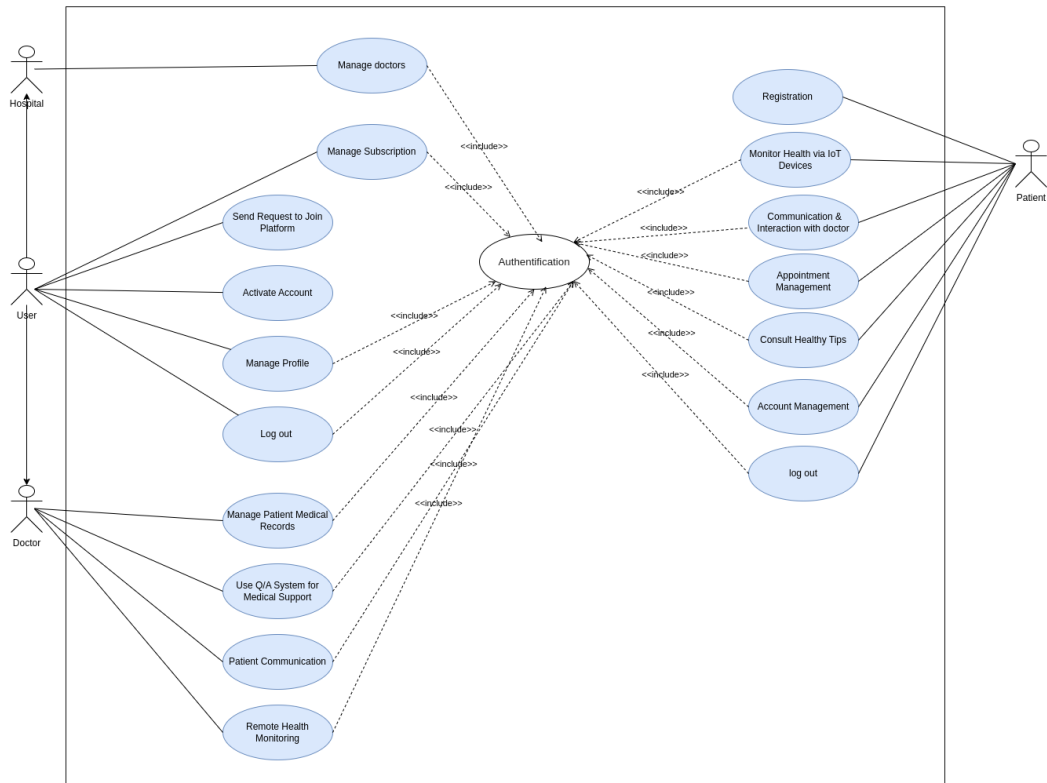


Figure 9.1: Global Use Case Diagram of the CardioX System

Our system is divided into 5 main subsystems:

- Authentication & Authorization Module
- Patient Monitoring Module
- Doctor Tracking Module

9.4.2.1 Authentication & Authorization Module



Figure 9.2: Authentication Use Case Diagram

9.4.2.2 Patient Monitoring Module

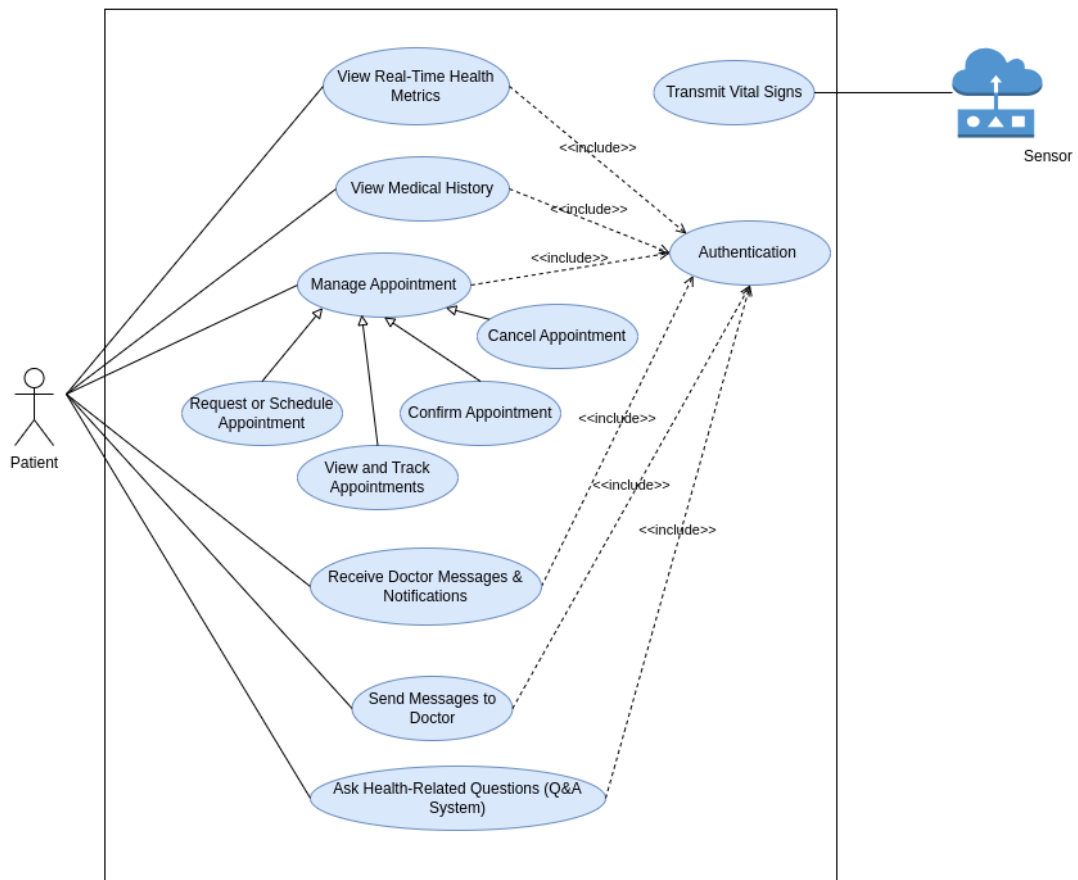


Figure 9.3: Patient Monitoring Use Case Diagram

9.4.2.3 Doctor Tracking Module



Figure 9.4: Doctor tracking Use Case Diagram

9.5 Conclusion

This chapter provided a detailed requirements specification and outlined the main use cases of the system. These serve as the foundation for the subsequent system architecture and design.

CHAPTER 10

SYSTEM AND ARCHITECTURE DESIGN

10.1 Introduction

This chapter defines the system's requirements and shows how they are translated into the architecture and design. It presents class and sequence diagrams to illustrate interactions and highlights security considerations such as authentication, authorization, and data protection.

10.2 Class Diagrams

To better understand the structure and relationships between the main components of the system, the following class diagram illustrates the core entities and how they interact. It provides a high-level view of the system's data model, showing the attributes, operations, and associations between the different classes used in the application.

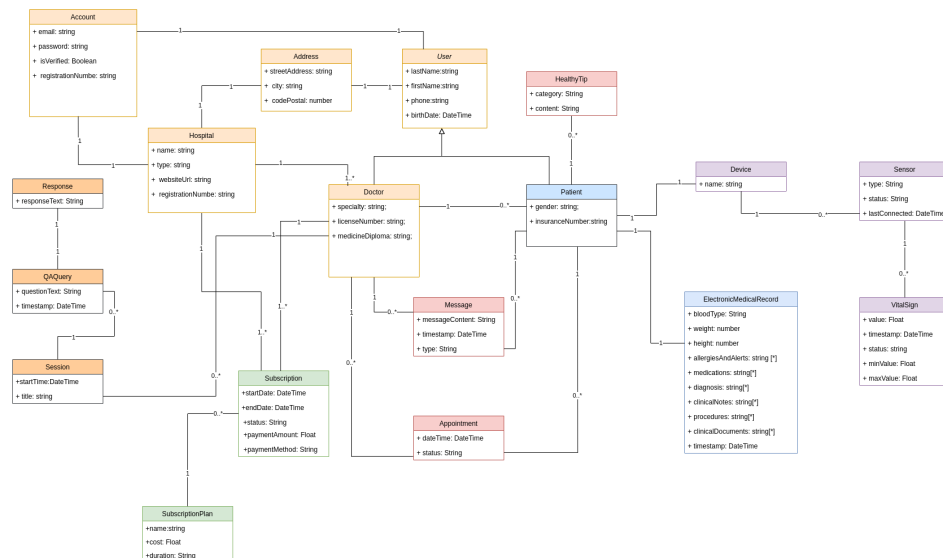


Figure 10.1: Global Class Diagram of the CardioX System

10.3 Sequence Diagrams

This section introduces the sequence diagram of the main functionalities of the system. It illustrates the interactions between different objects and demonstrates how they communicate with one another to complete a specific action.

10.3.1 Doctor Registration

The diagram below represents the registration request process and the subsequent account creation procedure followed by the doctor in order to access the platform's functionalities:

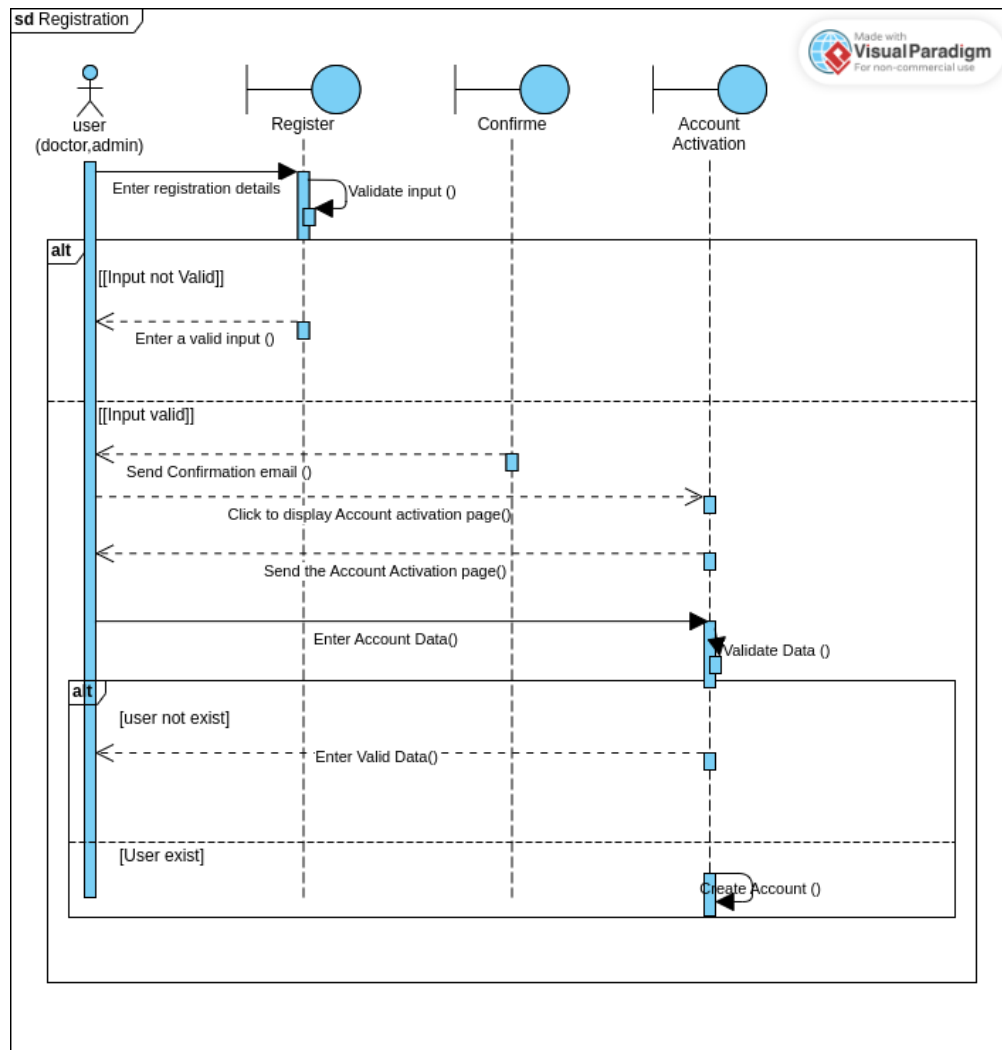


Figure 10.2: Sequence diagram of the doctor registration process

10.3.2 Patient Registration

The diagram below represents the process followed by the patient to create an account in the system in order to communicate with their doctor.

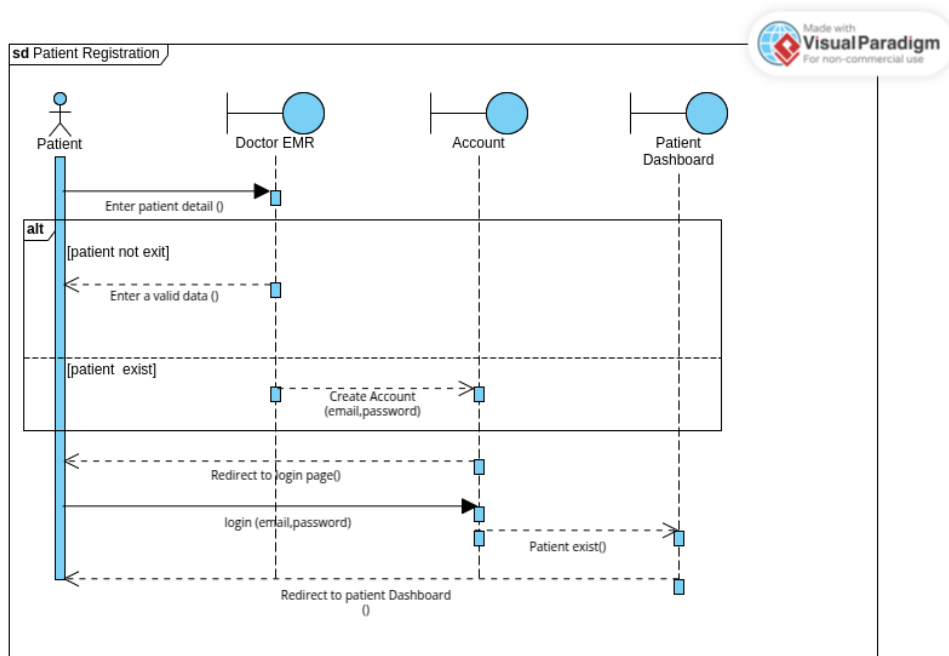


Figure 10.3: Sequence diagram of the patient registration process

10.3.3 Doctor–Patient Communication

The diagram below shows how the doctor and patient communicate through notifications and appointment scheduling. It illustrates how the patient can request an appointment, how the doctor responds, and how the system sends notifications to both sides to support smooth communication.

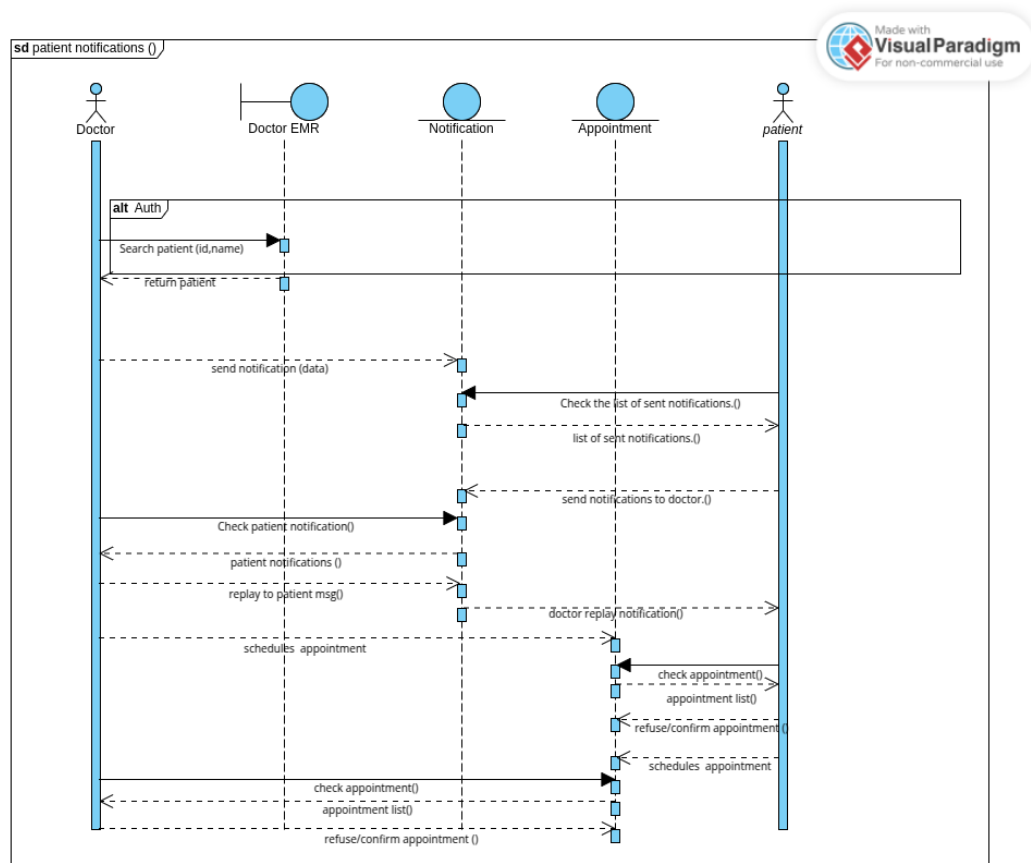


Figure 10.4: Sequence diagram of doctor–patient communication through notifications and appointment scheduling

10.3.4 Doctor’s Use of the Q/A System for Diagnosis

The diagram below illustrates how the doctor uses the Q/A system to support the diagnostic process. It shows the interaction between the doctor and the system, highlighting how questions and responses are exchanged to assist in reaching an accurate diagnosis.

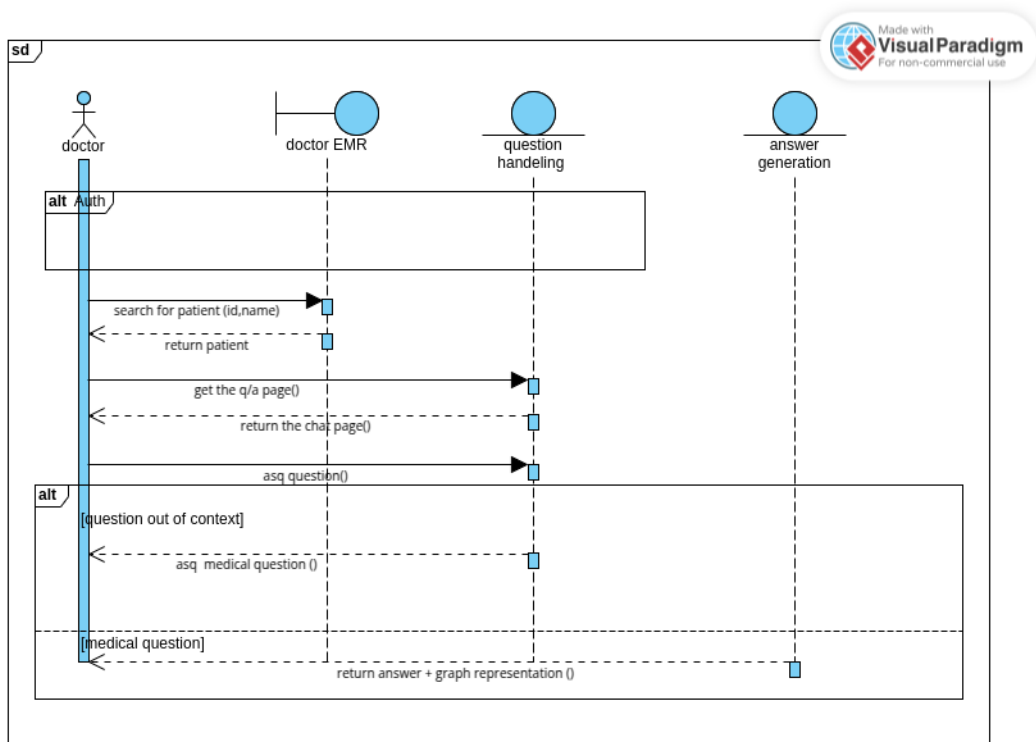


Figure 10.5: Sequence diagram of the doctor’s use of the Q/A system for diagnosis

10.3.5 Patient Vital Signs Recording and Monitoring

The diagram below illustrates how the patient records their vital signs and sends them to the doctor through the system. This process allows the patient to stay informed about their health status while enabling the doctor to monitor the data and take quick action when necessary.

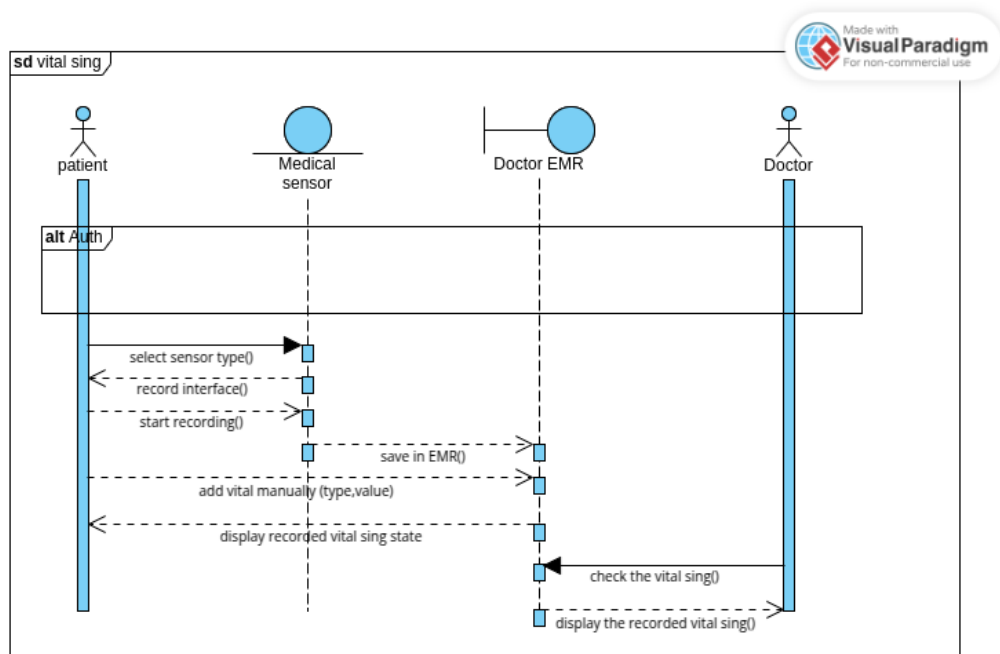


Figure 10.6: Sequence diagram of patient vital signs recording and monitoring

10.4 Security Considerations

10.4.1 Authentication and Authorization

The platform employs JSON Web Tokens (JWT) for secure authorization when accessing resources. Each authenticated user is assigned a token, which is used to verify their identity and ensure that only authorized requests can access protected endpoints. In addition, a Role-Based Access Control (RBAC) mechanism is implemented to define user permissions according to their roles within the system. This ensures that each user can only access the functionalities and resources relevant to their role, thereby enhancing both security and data privacy.

10.4.2 User Input Validation and Sanitization

The platform enforces strict input validation on all user-supplied data before processing. This protects against attacks such as SQL injection by checking type, length, and format, rejecting malformed input, and using parameterized queries for database access.

10.4.3 API Security

The system ensures that all APIs are protected through authentication and authorization mechanisms. Each API request from the user is validated using a security token, which verifies the user's identity and access rights before granting access to the requested resource.

10.5 Conclusion

This chapter outlined the system's requirements and design. It provided a structured view of the main components, their interactions, and security aspects, forming the basis for the system's implementation.

CHAPTER 11

IMPLEMENTATION

11.1 Introduction

This chapter outlines the implementation of the proposed system, covering the technologies, tools, system architecture, and user interface developed for both web and mobile platforms.

11.2 Hardware Overview

11.2.1 Core Hardware Platforms

The core of the IoT system consists of two main hardware platforms: the MySignals HW board and the Arduino Uno R3. These boards serve complementary roles in collecting, processing, and transmitting patient vital signs data.

- **MySignals HW:**

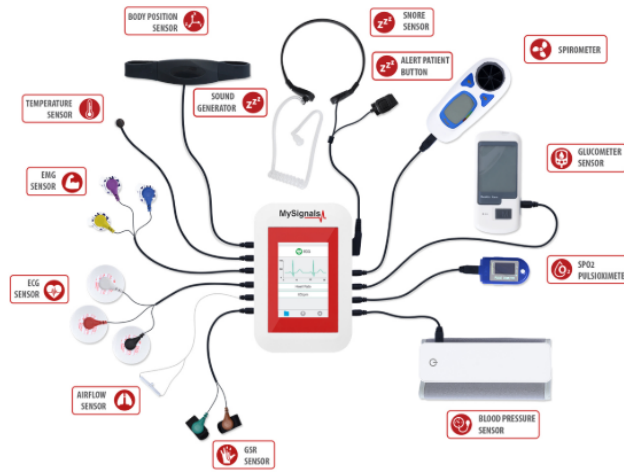


Figure 11.1: MySignals HW

MySignals HW is a biomedical IoT platform developed by Libelium for rapid prototyping of e-health applications. It integrates multiple medical sensors, including ECG, blood pressure, SpO₂, temperature, and airflow sensors, allowing vital signs monitoring and wireless transmission of data to cloud services or mobile applications [47].

In this project, MySignals HW serves as the primary interface for sensor data acquisition. It processes raw sensor signals and transmits them in real-time for remote monitoring. The platform also provides a high-level function library and example applications, enabling easy configuration and control of various modules through the Arduino IDE.

– **Arduino Uno R3:**



Figure 11.2: Arduino Uno R3

Arduino, also known under the name Genuino, is a family of open-source electronic boards built around microcontrollers such as the Atmel AVR (e.g., ATmega328P) and ARM (e.g., Cortex-M3 in the Arduino Due). The hardware schematics of these boards are published under an open-source license, although certain components, like the microcontroller itself, are proprietary.

The Arduino Uno R3 is one of the most widely used boards in this family. It is based on the ATmega328P microcontroller and provides digital and analog input/output pins, as well as USB connectivity. Thanks to its simplicity and large ecosystem of libraries, it has become a standard tool for prototyping in embedded systems, robotics, home automation, and IoT applications [11].

In this project, the Arduino Uno R3 is used for additional signal processing and interfacing tasks, such as filtering sensor readings or handling auxiliary sensors not directly supported by the MySignals HW platform. It communicates with MySignals HW and other modules via USB or serial communication, extending the flexibility of the overall IoT system.

11.2.2 Medical Sensor Integration and Specifications

This section presents the connected device used in our system, which is composed of the MySignals HW platform by Libelium connected to an Arduino board. The device has been implemented for patient health monitoring as part of our e-Health solution, SoinConnect.

In this part, we provide an overview of the hardware components and describe the set of sensors integrated with MySignals HW. While the platform is capable of supporting up to 16 medical sensors, in our project we selected and used four sensors that are most relevant to the objectives of patient monitoring.

- **Temperature sensor:** The Exacon D-S18JK temperature sensor is used to monitor body temperature, a key vital sign that reflects internal health status. The sensor measures in the range of 0–50°C, with normal adult values typically between 36.5°C and 37.5°C. To operate, the sensor must be connected to the dedicated Temperature port on the MySignals hardware board. It uses a direct connector power supply and ensures a proper fit through a one-way connection design. For accurate readings, the metallic part of the sensor must make contact with the skin.

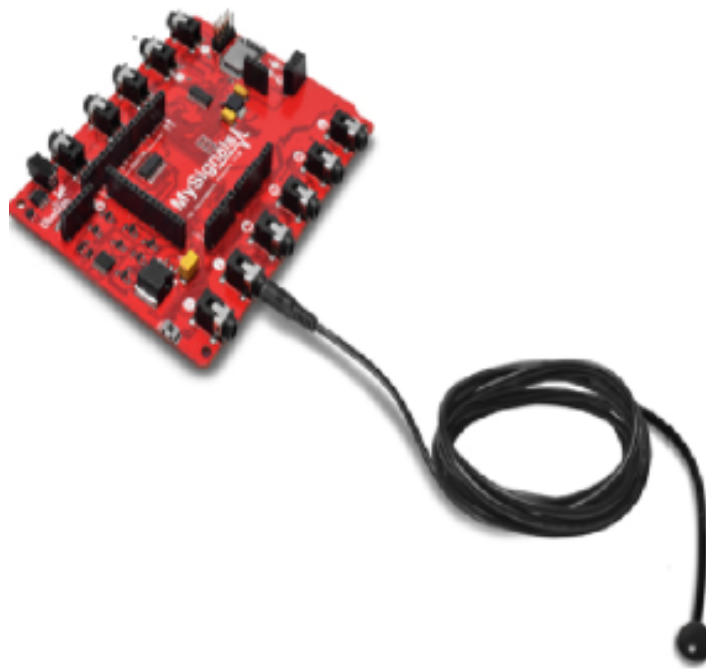


Figure 11.3: Exacon D-S18JK temperature sensor

- **Blood pressure sensor:** The blood pressure sensor measures arterial pressure by monitoring both systolic (heart contraction) and diastolic (heart relaxation) values, as well as pulse rate. It uses an oscillometric method and connects to the dedicated blood pressure port on the MySignals board. Powered by an internal rechargeable battery, the sensor can also store up to 80 measurements with corresponding time and date, making it valuable for detecting hypertension or hypotension.



Figure 11.4: blood pressure sensor

- **ECG sensor:** The ECG (Electrocardiogram) sensor is a diagnostic tool used to evaluate the electrical and muscular activity of the heart. It enables continuous telemetry monitoring through three disposable pre-gelled electrodes connected to the dedicated ECG port on the MySignals board. This sensor is essential for detecting heart rhythm abnormalities and potential cardiac conditions such as infarction, ischemia, or arrhythmias. It can also provide information on heart orientation, hypertrophy, blood flow reduction, and abnormal electrical activity, offering real-time insights into cardiac health.



Figure 11.5: ECG sensor

- **SpO₂:** The SpO₂ sensor is a noninvasive optical device used to measure arterial oxygen saturation and pulse rate, two key indicators of respiratory and cardiovascular function. It operates by emitting light at two wavelengths (red at 660 nm and infrared at 940 nm) to distinguish between oxygenated hemoglobin (HbO₂) and deoxygenated hemoglobin (Hb), with a photodetector calculating oxygen levels based on light absorption differences. The sensor connects to the dedicated

SpO₂ port on the MySignals board and provides real-time monitoring, making it valuable for applications such as intensive care, emergency medicine, and remote patient monitoring.

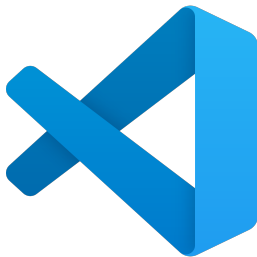


Figure 11.6: SpO₂ sensor

11.3 Technologies and Tools Used

11.3.1 Development Environment

Visual Studio Code



Visual Studio Code (VS Code) is a free, cross-platform code editor by Microsoft, offering syntax highlighting, IntelliSense, debugging, Git integration, and a large extension marketplace, making it one of the most popular development tools today [26]

Google Colab



Google Colab, short for Google Colaboratory, is a free, cloud-based version of Jupyter Notebooks provided by Google. It allows you to write and run Python code in your browser without installing anything, and gives access to resources like GPUs and TPUs, making it ideal for data science, machine learning, teaching, and collaboration [27].

Arduino IDE



Arduino IDE is a free, open-source integrated development environment used to write, compile, and upload code to Arduino boards. It supports the C and C++ languages with a simple editor, includes a library manager, and connects directly to Arduino hardware for programming and debugging, making it beginner-friendly and widely used in electronics and IoT projects [37].

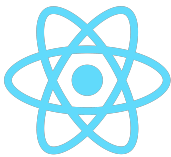
11.3.2 Frontend Technologies

Next.js



An open-source React framework by Vercel that supports server-side rendering (SSR), static site generation (SSG), and client-side rendering (CSR) [1]. Features include file-based routing, API routes, automatic code splitting, image optimization, Incremental Static Regeneration, and built-in middleware support [2]

React Native



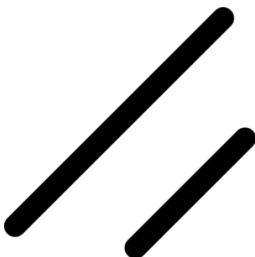
Tailwind CSS

An open-source framework by Meta (Facebook) for building native mobile apps using JavaScript and React—not WebView—rendering genuine native components



A utility-first CSS framework that provides low-level classes (like flex, p-4, text-center) to build custom designs directly in HTML [1]. Unlike Bootstrap, it doesn't include pre-styled components—it gives you granular styling options .

ShadCN



ShadCN is a UI component library that aims to streamline the development process by offering pre-built, accessible, and customizable components. By leveraging Radix UI for accessibility and Tailwind CSS for styling, ShadCN provides developers with a solid foundation to build modern and responsive web interfaces.

11.3.3 Backend Technologies

Nest Js



NestJS is a progressive, modular Node.js framework designed for building efficient, scalable, and maintainable server-side applications using TypeScript (though JavaScript is also supported). Inspired by Angular, it adopts a structured, opinionated approach with features like: File-based organization, Dependency Injection, Support for Express, Fastify and Built-in features (REST, GraphQL APIs, microservices, middleware, guards.)

Python



is a high-level, interpreted, general-purpose programming language whose design philosophy emphasizes code readability through its use of significant indentation and clear syntax. [11]

PyKEEN



is a Python package designed for training and evaluating knowledge graph embedding models. It offers a flexible framework that supports various interaction models, training approaches, loss functions, and includes features such as automatic memory optimization and hyperparameter tuning. [15]

Torch



is an open-source scientific computing framework and machine learning library, originally developed in Lua. It provides a wide range of machine learning algorithms and is optimized for GPU operations, making it suitable for large-scale numerical computations. Torch was widely adopted in early deep learning research and laid the groundwork for later frameworks such as PyTorch. [16]

Torch Geometric



Torch Geometric (sometimes called PyTorch Geometric) is a geometric deep learning library built to extend the Torch ecosystem. It is designed for learning on irregular and graph-structured data such as social networks, knowledge graphs, and molecular graphs. The library provides efficient GPU implementations of graph operations and includes a wide collection of Graph Neural Network (GNN) models, such as Graph Convolutional Networks (GCN), Graph Attention Networks (GAT), and GraphSAGE [17]

11.3.4 Database and Storage

Mongo DB



MongoDB is an open-source, document-oriented NoSQL database that stores structured, semi-structured, and unstructured data as JSON-like documents (in BSON format) instead of rows and columns

Neo4j

 is a graph database management system that stores data as nodes, relationships, and properties, optimized for highly connected data. It uses the Cypher query language to retrieve and manipulate graph structures. [18]

11.4 API & Models Integrations

11.4.1 APIs Used

The platform leverages several external APIs to enhance its functionality and provide seamless integration with different services.

11.4.1.1 Cloudinary API

Cloudinary API is a set of cloud-based, RESTful and URL-based APIs that allow developers to upload, manage, transform, optimize, and deliver images and videos at scale. It provides programmatic access for handling media workflows—covering upload, storage, real-time manipulation, AI-based enhancements, and global CDN delivery. [19]

11.4.1.2 OpenRouter

OpenRouter is a unified “router” API for Large Language Models that exposes a single, OpenAI-compatible endpoint to access hundreds of models across many providers, while transparently handling provider selection, fallbacks, and routing based on price, latency, and availability. By normalizing request/response schemas and centralizing auth, it lets applications integrate multiple LLMs through one interface with operational controls for reliability and cost. [43]

In our case, the OpenRoute API is used to interact with large language models (LLMs) for two main purposes:

1. To understand and classify the questions asked by the doctor.
2. To generate informative answers for the doctor based on both the question and the relevant information.

11.4.2 Pre-Trained Models Utilized

To enhance the intelligent features of the platform, pretrained models are integrated into the system., without the need to train complex models from scratch. Leveraging pre-trained models ensures efficiency, reliability, and the use of well-established techniques in the research community. The main models utilized in this work are described below:

11.4.2.1 spaCy

1. **Definition:** spaCy is an open-source library for advanced Natural Language Processing (NLP) in Python, providing tools for tokenization, part-of-speech tagging, dependency parsing, named entity recognition, and more [44].
2. **Model used:** *en_core_web_sm* (Small English model). In our case, it is used for extracting noun phrases and performing initial text preprocessing before semantic similarity analysis.

11.4.2.2 SentenceTransformer

- **Definition:**The SentenceTransformers library provides easy-to-use pre-trained models for generating semantically meaningful embeddings of sentences and phrases. [45]
- **Model used:***all-MiniLM-L6-v2*a small and efficient transformer model fine-tuned for semantic similarity tasks in our cas it’s used to encode doctors’ questions and medical terms into vector embeddings for semantic matching, classification, and answer generation.

11.4.2.3 SapBERT

- **Definition:**A biomedical language representation model built on PubMedBERT and fine-tuned for entity-level similarity and concept linking. [46]
- **Model used:***cambridgeltl/SapBERT-from-PubMedBERT-fulltext* to generates embeddings for biomedical concepts (e.g., symptoms, diseases) to improve entity extraction and semantic matching in the medical domain.

11.5 User Interface Preview

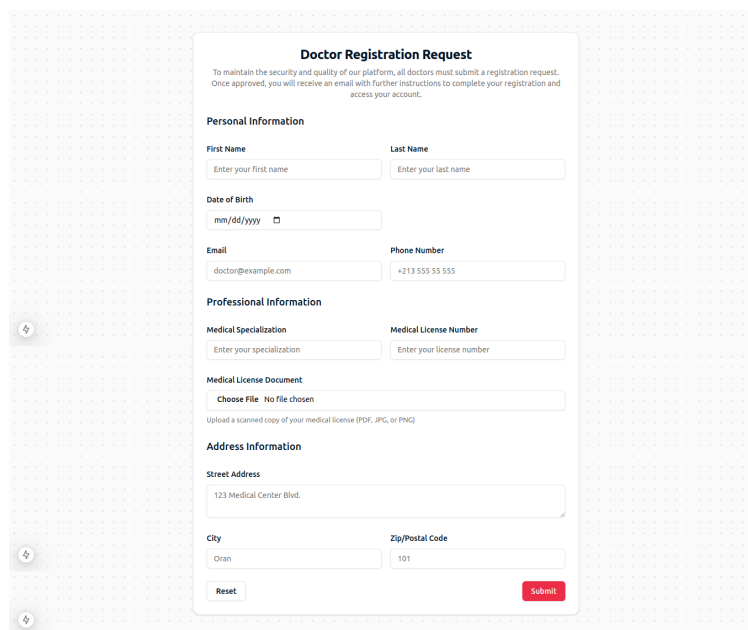
This section presents a visual overview of the user interfaces for both the web-based and mobile applications developed as part of the system

11.5.1 Web Application Screens

The following screens illustrate the interface of the web application designed for doctors and healthcare staff. Each page is presented with detailed explanations, including the different interface states and their functionalities.

11.5.1.1 Registration Request Page

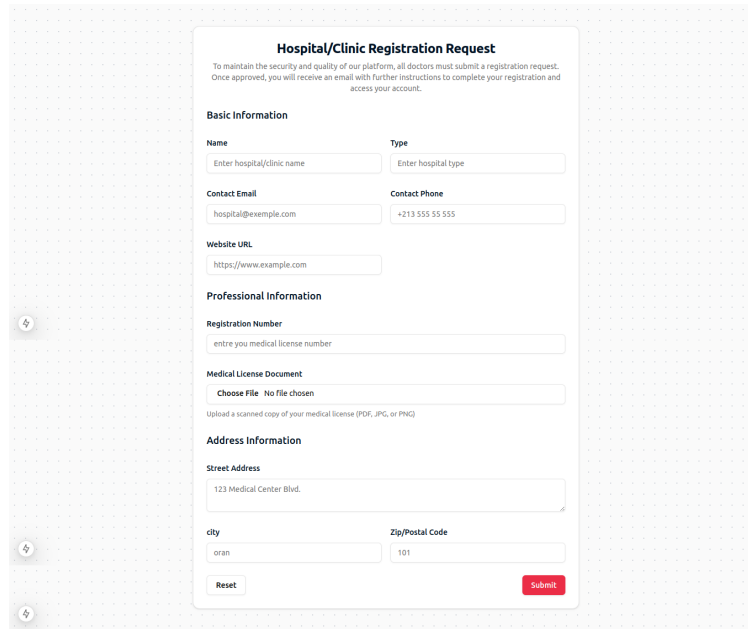
- **Accessibility:** Available to all users.
- **Functionality:** The registration process is divided into two separate screens one for hospitals and one for private doctors who wish to create an account and join the platform. Each screen provides input fields to capture important information relevant to the respective applicant (hospital or private doctor). This data is essential for the verification and onboarding process. Both forms include built-in validation mechanisms to ensure correct input formats and to prevent incomplete or invalid submissions. Users receive immediate feedback on any errors, which enhances data accuracy and improves the overall registration experience
- **Doctor Registration Request Screen**



The screenshot displays a web form titled "Doctor Registration Request". At the top, a disclaimer states: "To maintain the security and quality of our platform, all doctors must submit a registration request. Once approved, you will receive an email with further instructions to complete your registration and access your account." The form is organized into four main sections: "Personal Information", "Professional Information", "Medical License Document", and "Address Information". The "Personal Information" section includes fields for "First Name", "Last Name", "Date of Birth" (with a calendar icon), "Email" (pre-filled with "doctor@example.com"), and "Phone Number" (pre-filled with "+213 555 55 555"). The "Professional Information" section contains "Medical Specialization" and "Medical License Number" fields. The "Medical License Document" section features a file upload button labeled "Choose File" and a note: "No file chosen. Upload a scanned copy of your medical license (PDF, JPG, or PNG)". The "Address Information" section includes "Street Address" (pre-filled with "123 Medical Center Blvd."), "City" (pre-filled with "Oran"), and "Zip/Postal Code" (pre-filled with "101"). At the bottom left is a "Reset" button, and at the bottom right is a red "Submit" button.

Figure 11.7: Doctor registration request screen. Used by private doctors to apply for access to the platform.

- **Hospital Registration Request Screen**



Hospital/Clinic Registration Request

To maintain the security and quality of our platform, all doctors must submit a registration request. Once approved, you will receive an email with further instructions to complete your registration and access your account.

Basic Information

Name: Type:

Contact Email: Contact Phone:

Website URL:

Professional Information

Registration Number:

Medical License Document

Choose File:

Upload a scanned copy of your medical license (PDF, JPG, or PNG)

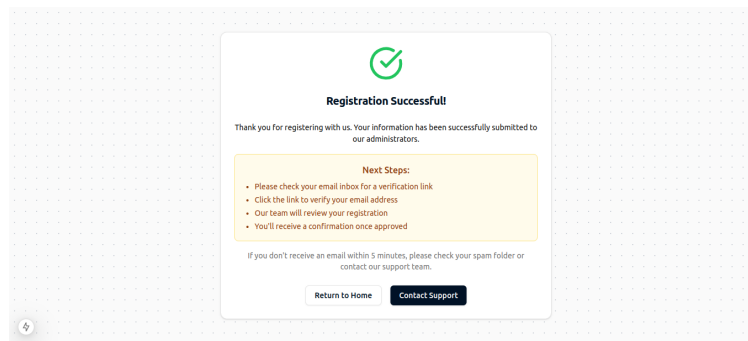
Address Information

Street Address:

City: Zip/Postal Code:

Figure 11.8: Hospital registration request screen. Used by hospitals to to apply for access to the platform.

- **Registration Submission Confirmation Screen:**After completing the registration request form, users see this confirmation screen indicating that their information has been successfully submitted. It provides clear instructions for the next steps, including email verification, and informs users that they will receive a confirmation once their registration has been reviewed and approved.





Registration Successful!

Thank you for registering with us. Your information has been successfully submitted to our administrators.

Next Steps:

- Please check your email inbox for a verification link
- Click the link to verify your email address
- Our team will review your registration
- You'll receive a confirmation once approved

If you don't receive an email within 5 minutes, please check your spam folder or contact our support team.

Figure 11.9: Registration submission confirmation screen. Guides users through the next steps after submitting their registration request.

11.5.1.2 Account Activation Page

- **Accessibility:** Provides authorized access for doctors and hospital administrative staff.
- **Functionality:** Serves as the primary interface for creating and activating user accounts. The page includes clearly labeled input fields for entering doctor information, hospital details, and secure password creation, ensuring a smooth and safe registration process.
- **State:**

Activate Your Doctor Account

Welcome, Doctor. Please verify your account to access your dashboard and start providing care.

First Name: John Last Name: Doe

Specialty: e.g. Cardiology License Number: e.g. MD12345

Email: john.doe@example.com

Password: Password Confirm Password: Password

Must be at least 8 characters with uppercase, lowercase, and numbers.

Activate Your Account

Figure 11.10: Screenshot of the Account Activation Page

11.5.1.3 Login Page

- **Accessibility:**Doctors and cardiology administrative staff in the hospital.
- **Functionality:**Serves as the entry point for validated users to access the platform's features. The interface consists of clearly labeled input fields for email and password, a “Forgot Password” recovery option, and visual feedback for authentication errors. For users who have forgotten their password, a reset link is provided to facilitate account recovery.
- **State:**

Sign In

Enter your credentials to continue

Email Address: doctor@hospital.com

Password: Enter your password [Forgot password?](#)

Sign In Securely

Secure Access
Your login is protected with enterprise-grade security and HIPAA compliance.

Don't have an account? [Request Access](#)

Figure 11.11: Login page. Doctors and cardiology administrative staff use this screen to access the hospital platform.

11.5.1.4 Doctor Dashboard

11.5.1.5 EMR Filtering and Search Page

- **Accessibility:**Connected Doctors

- **Functionality:** This main screen serves as the central hub for the integrated Electronic Medical Record (EMR) system. It provides a summarized view of all active patients under a specific doctor or across the hospital, with quick-access tools for key actions:
 - * **Add New Patient:** Create a new EMR for a patient.
 - * **Schedule Appointments:** Plan and manage upcoming visits.
 - * **View Detailed EMRs:** Access a patient’s full medical history, including diagnoses, medications, and medical records.
 - * **Manage Patient Care:** Update treatment plans, add new medications, record test results, and register new diagnoses.
- **State:**
 - * **Patient list screen:**

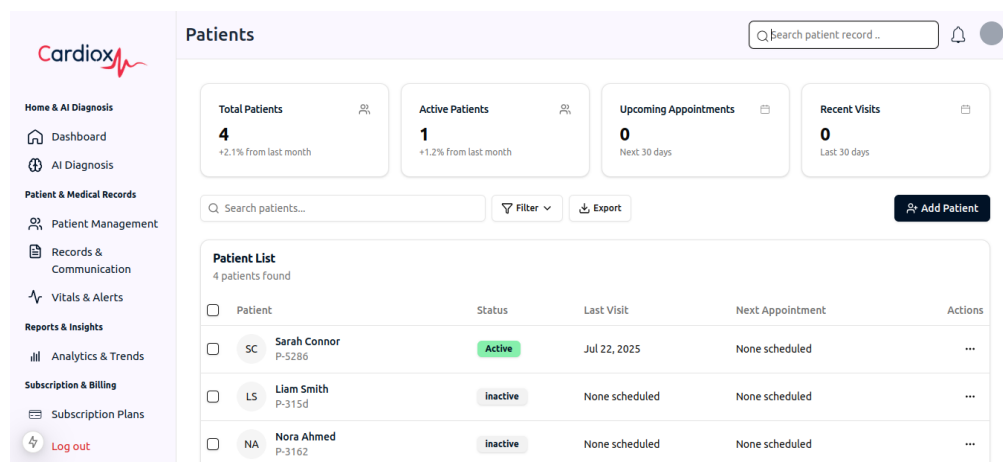


Figure 11.12: Screenshot of the Patient List Screen displaying patient details, search, and filter options.

- * **Add new Patient screen:**

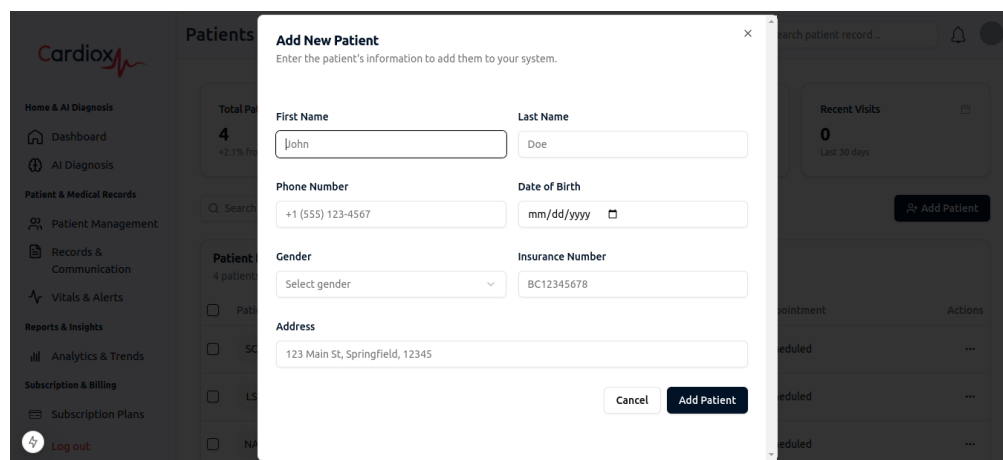


Figure 11.13: Screenshot of the Add New Patient Screen showing input fields for patient details

- * **Schedule Appointments screen**
- * **Patient Detail screen:**

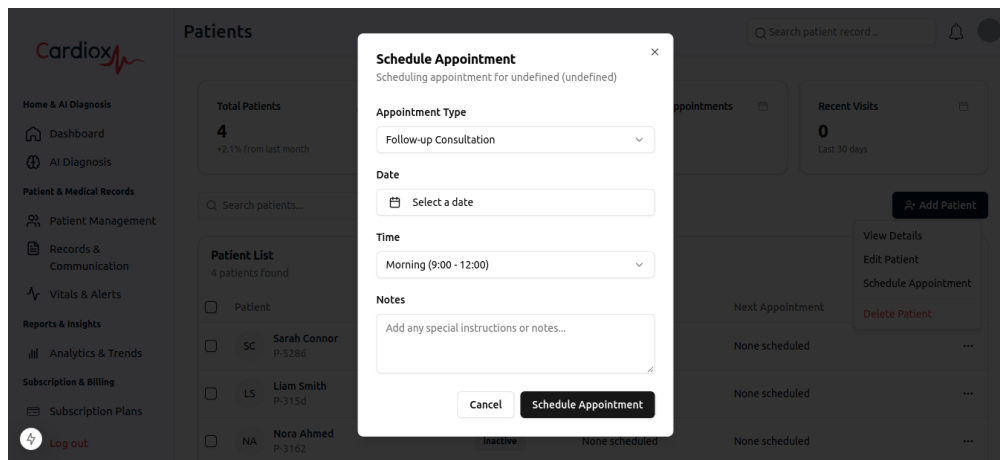


Figure 11.14: Screenshot of the Schedule Appointments Screen

11.5.1.6 Q/A Consultation Page

- **Accessibility:** Connected Doctors
- **Functionality:** This screen serves as the main interface for interacting with the Q/A system. Doctors can ask multiple questions related to their patients, including medication, diagnostic tests, and treatment options. The interface also supports the analysis of medical images, providing a comprehensive consultation tool.
- **State:**
 - * Q& A screen:

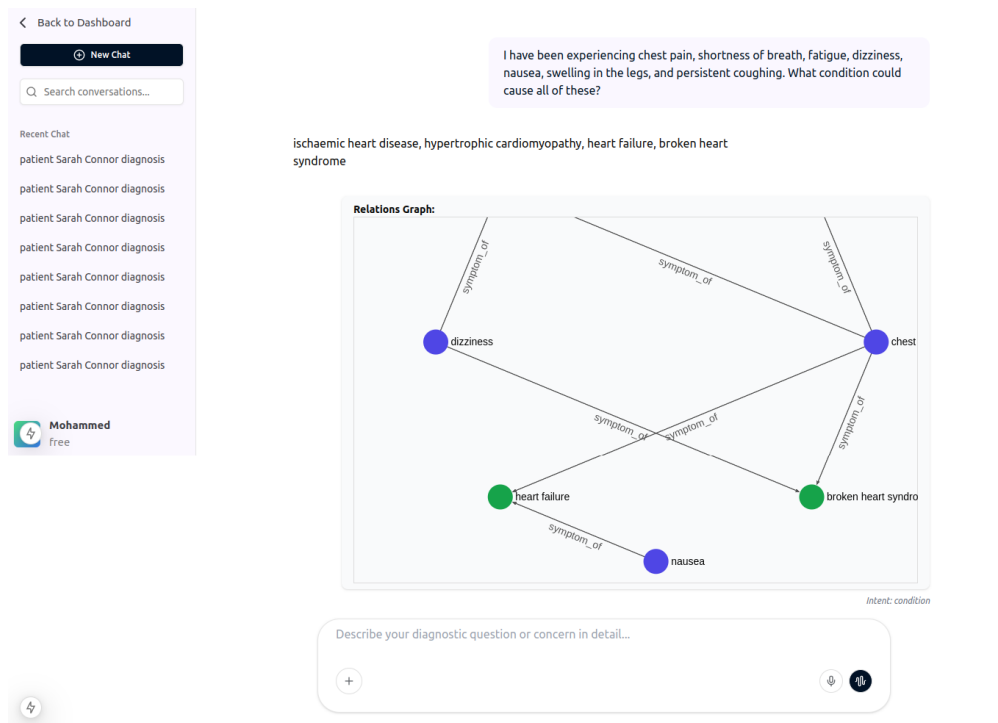


Figure 11.16: Screenshot of the Q&A Screen where doctors can ask questions

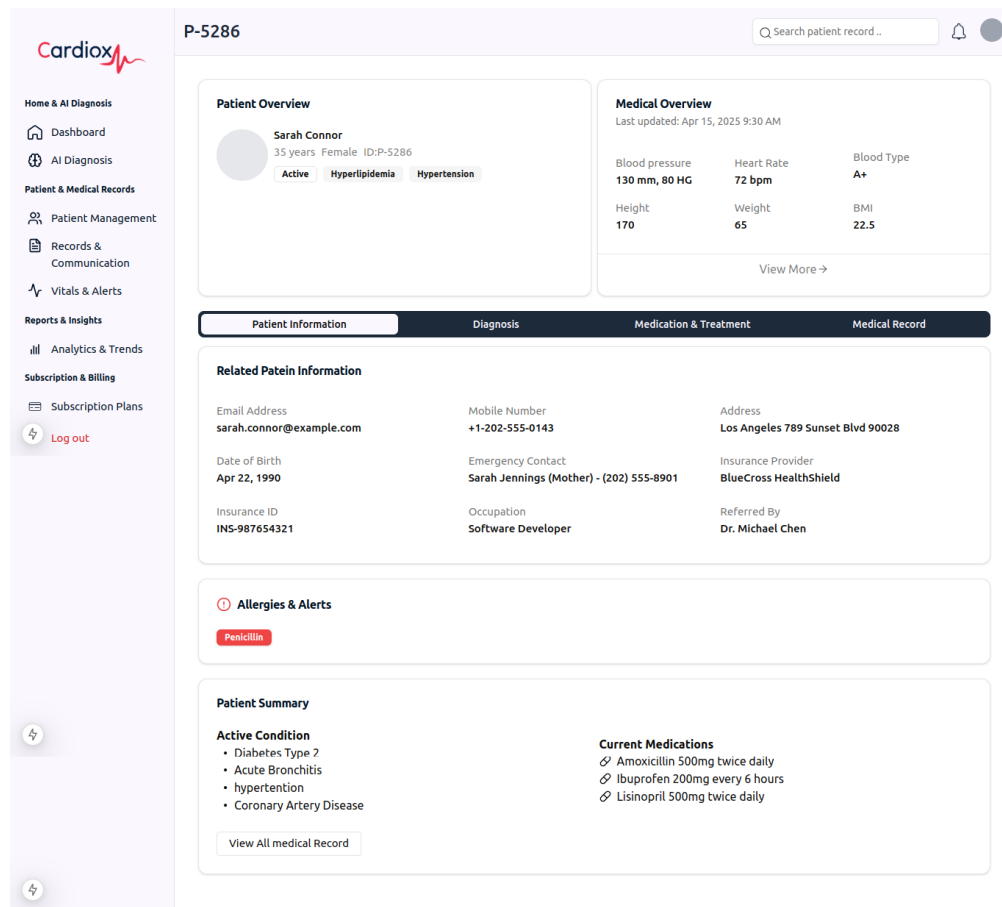


Figure 11.15: Screenshot of the Patient Detail Screen showing detailed patient information, medical history, and contact details.

* Analyse Medical images:

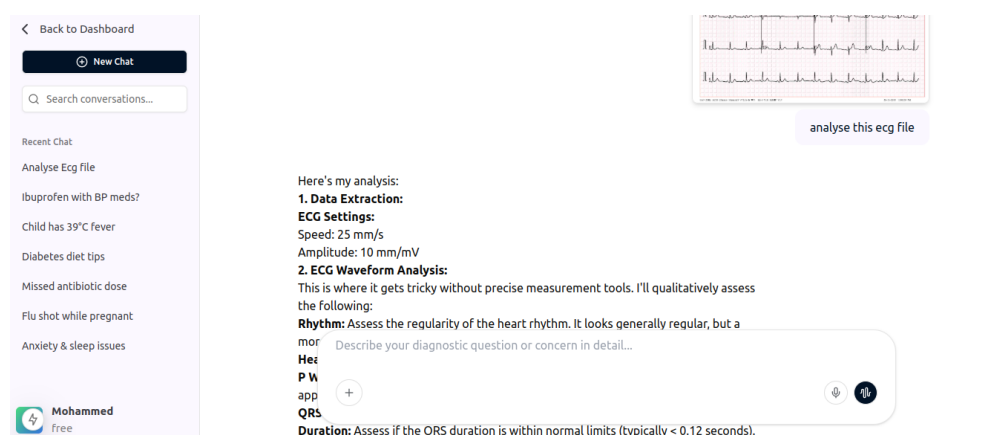


Figure 11.17: Screenshot of the Analyse Medical Images screen where doctors can upload and interpret patient scans for diagnostic support.

11.5.1.7 Vital Sign Tracking Page

- **Accessibility:** Connected Doctors
- **Functionality:** This screen displays the most recent vital sign records for each

patient, collected either through IoT sensors or entered manually by the patient. It includes all necessary recording details and warning levels, enabling doctors to respond quickly when needed. Features include:

- * **Filtering:** Doctors can filter by patient to view all of that patient's vital sign records.
- * **History view:** The full history of previous records is available for review.
- * **Actions:** For each patient, doctors can send notifications, schedule appointments, or take other appropriate actions

– **State:**

- * **Vital Sing monotoring screen:**

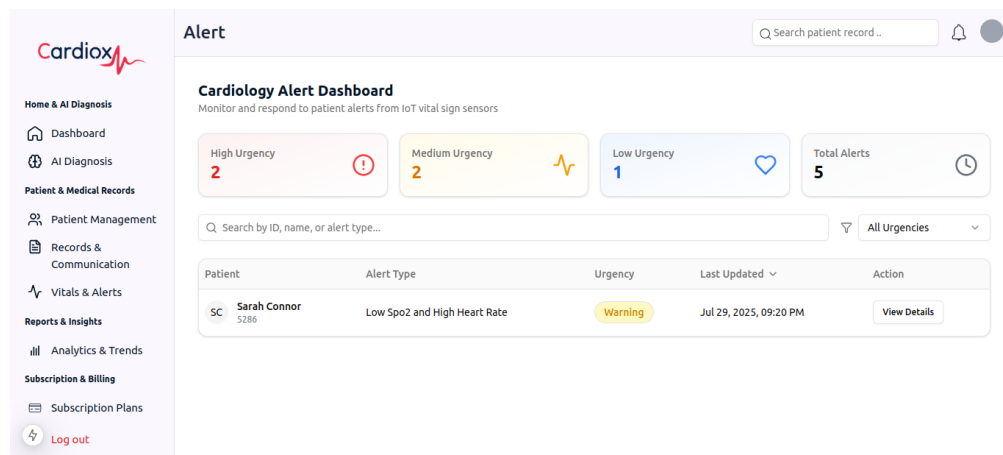


Figure 11.18: Screenshot of the Vital Sign Monitoring Screen showing real-time patient vitals

- * **Patient Detail recording**

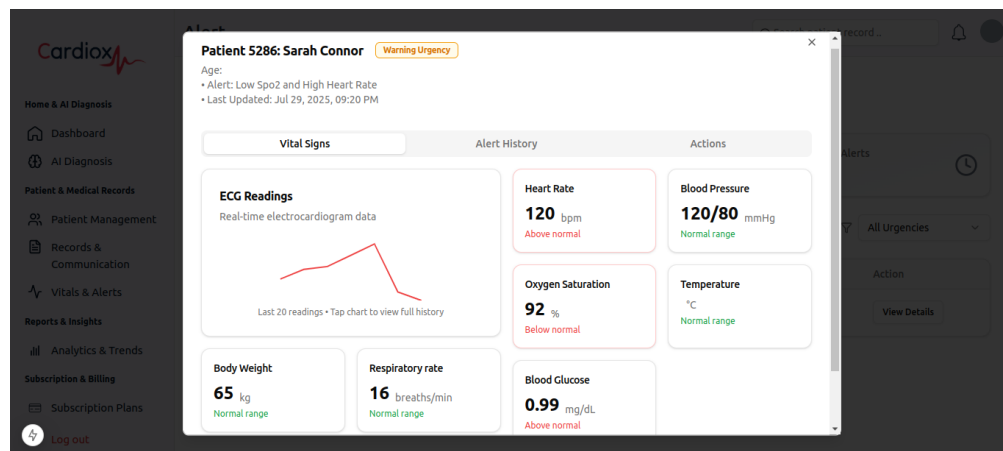


Figure 11.19: Screenshot of the Patient Detail Recording screen showing fields for medical history, symptoms, and treatment notes.

11.5.1.8 Doctor Communication Page

- **Accessibility:** Available to all connected doctors within the platform.

- **Functionality:** Acts as the central hub for doctor-patient communication. The page is divided into two main sections:
 - * **My Notifications:** Displays all notifications sent by the doctor to patients. It includes a side panel that allows the doctor to select specific patients and send them messages or updates.
 - * **Patient Notifications:** Shows messages sent by patients to the doctor. The doctor can filter these messages, review patient inquiries, and respond to medical questions directly through the platform.
- **State:**
 - * **Doctor Notification:**

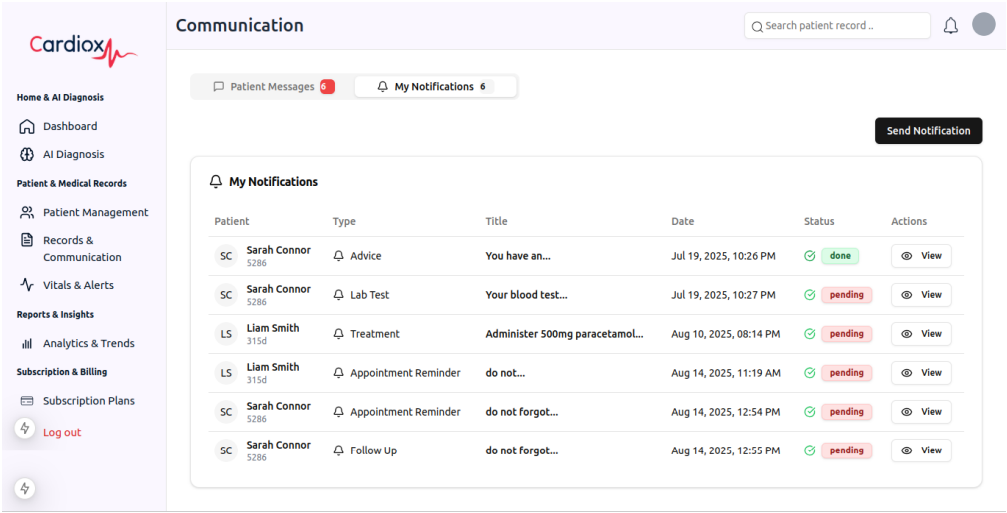


Figure 11.20: Screenshot of the Doctor Notification screen showing system alerts and patient updates for doctors.

- * **Send Notification to patient:**

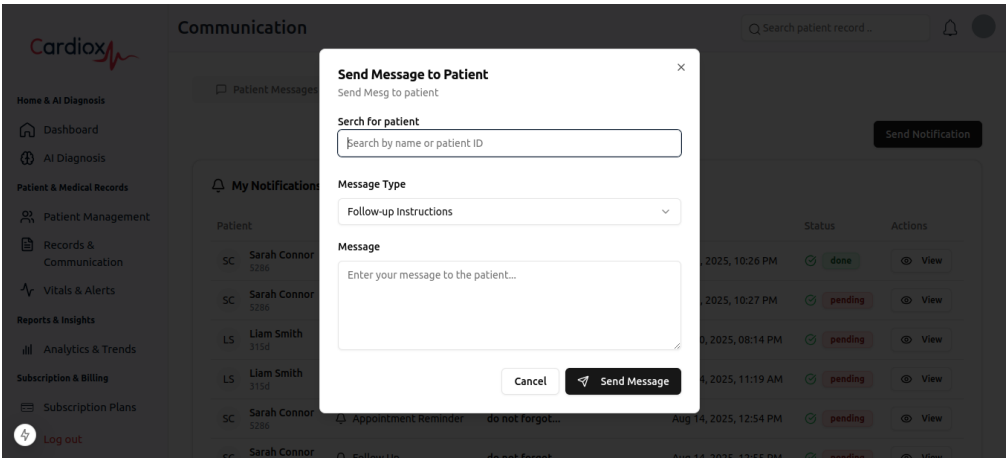


Figure 11.21: Screenshot of the Send Notification to Patient screen with message input and delivery options.

- * **Patient Notification:**

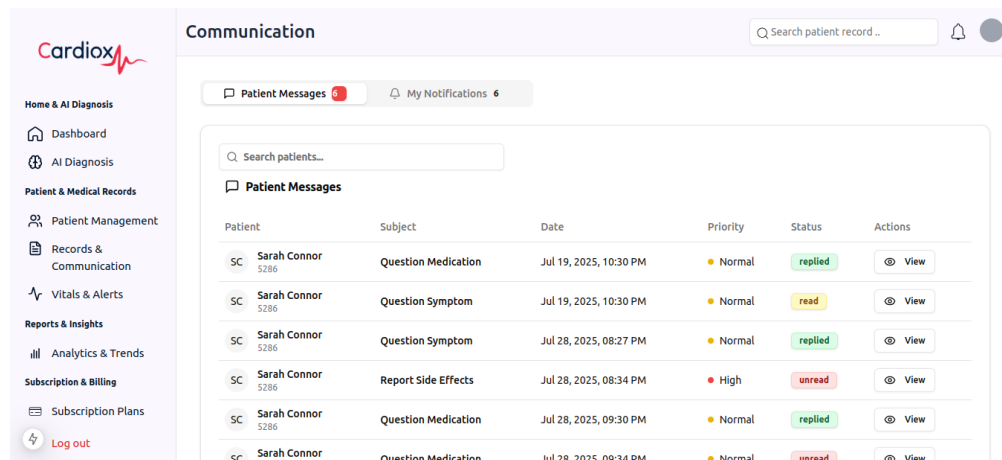


Figure 11.22: Screenshot of the Patient Notification screen where patients receive updates and alerts from doctors or the hospital.

11.5.1.9 Hospital Dashboard

- **Accessibility:** Cardiology administrative staff in the hospital.
- **Functionality:** This dashboard gives cardiology administrative staff a centralized view of all doctors in the hospital and their activity within the system. It streamlines management tasks by allowing administrators to:
 - * **Oversee Doctors:** View all cardiology doctors in the hospital and monitor their activity in the system.
 - * **Manage Doctors:** Send invitations and add new doctors to the platform quickly
 - * **Handle Administration:** Manage payments, subscriptions, and hospital information
- **State:**
 - * **Hospital Dashboard:**

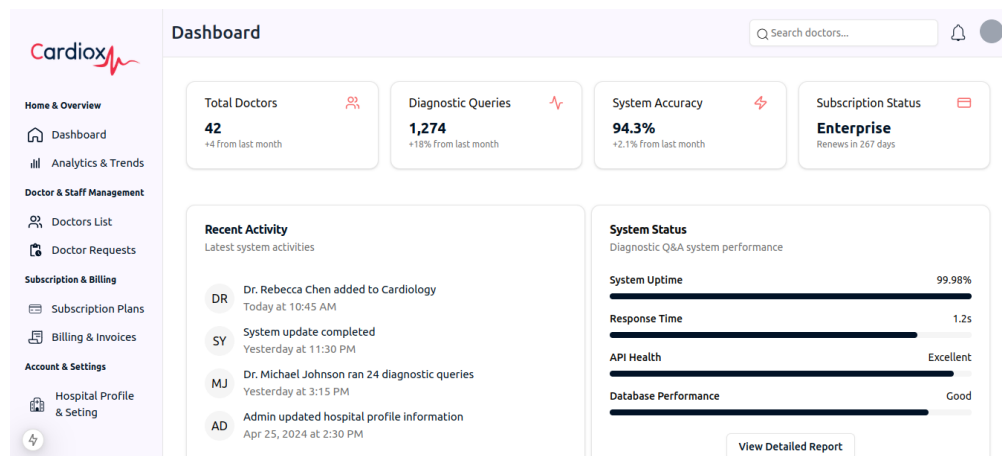


Figure 11.23: Screenshot of the Hospital Dashboard showing overall hospital statistics, active doctors, and patient activity.

* Doctor List:

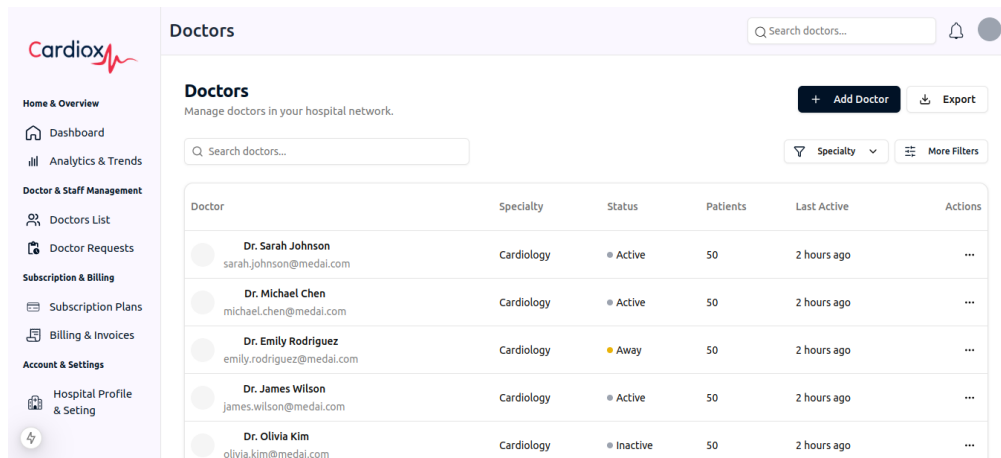


Figure 11.24: Screenshot of the Doctor List screen displaying registered doctors with search and filter options.

* Add Doctor Screen:

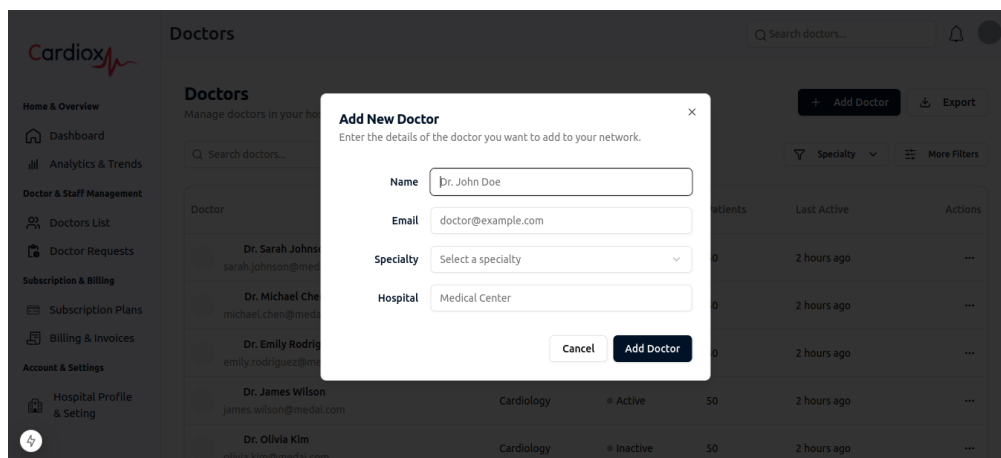


Figure 11.25: Screenshot of the Add Doctor Screen with input fields for doctor details, specialization, and hospital assignment.

11.5.2 Mobile Application Screens

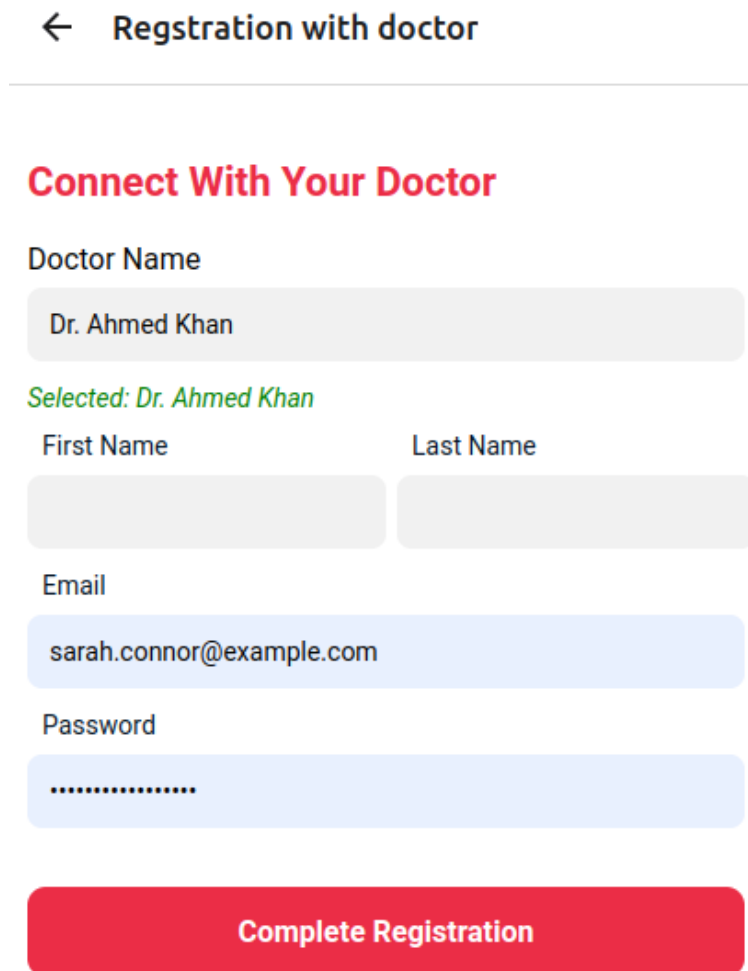
The following screens depict the interface of the mobile application developed for patients. Each page is accompanied by detailed descriptions, covering various states and interactions to demonstrate how patients engage with the application.

11.5.2.1 Patient Registration Screen

- **Accessibility:** Patients who are already included in their doctor's or hospital's EMR system.
- **Function:** This screen serves as the account activation interface for patients. It allows them to create their credentials and gain access to the mobile application,

enabling them to use its features and communicate directly with their cardiology doctor.

- **State:**



The image shows a mobile application screen for patient registration. At the top, there is a back arrow and the title "Registration with doctor". Below this is a red header "Connect With Your Doctor". The form contains several input fields: "Doctor Name" with a dropdown menu showing "Dr. Ahmed Khan" and a green confirmation text "Selected: Dr. Ahmed Khan"; "First Name" and "Last Name" as separate input fields; "Email" with the text "sarah.connor@example.com"; and "Password" with masked characters. At the bottom is a large red button labeled "Complete Registration".

Figure 11.26: Patient registration screen. Used by patients already registered in the EMR system to activate their account and connect with their doctor through the application.

11.5.2.2 Patient Login Screen

- **Accessibility:** Patients visiting the cardiology department.
- **Functionality:** Serves as the entry point for patients to stay connected with their cardiology doctor. The interface consists of clearly labeled input fields for email and password, a “Forgot Password” recovery option, and visual feedback for authentication errors,
- **State:**

 Login

Sign In

Welcome again to **CardioX**. Monitor your heart and track your health—anytime, anywhere.

Email

Email

Password

.....

Forgot Password?

Sign In

Don't have an account? [Sign Up](#)

Figure 11.27: Patient login screen. Provides patients secure access to the platform to stay connected with their cardiology doctor.

11.5.2.3 Patient Home Screen

- **Accessibility:** Available to connected patients.
- **Functionality: Function:** Serves as the main screen after login, providing a summary of the patient's most recent vital signs recorded either by medical sensors or manually by the patient, along with health indicators reflecting the patient's current state. It includes notifications about important events or updates, as well as a button allowing patients to manually register their vital signs by selecting the available vital type and entering the corresponding value.

– State:

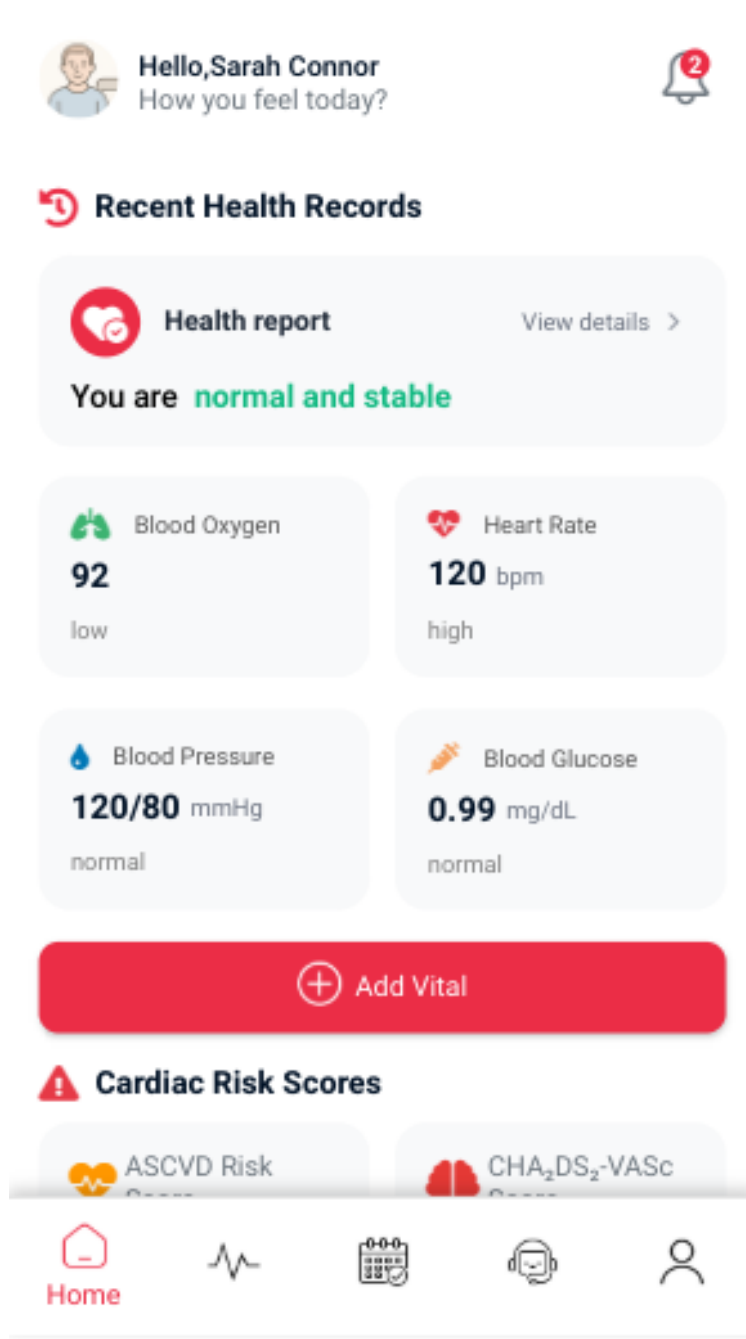


Figure 11.28: Patient home screen showing recent vital signs, health indicators, and notifications.

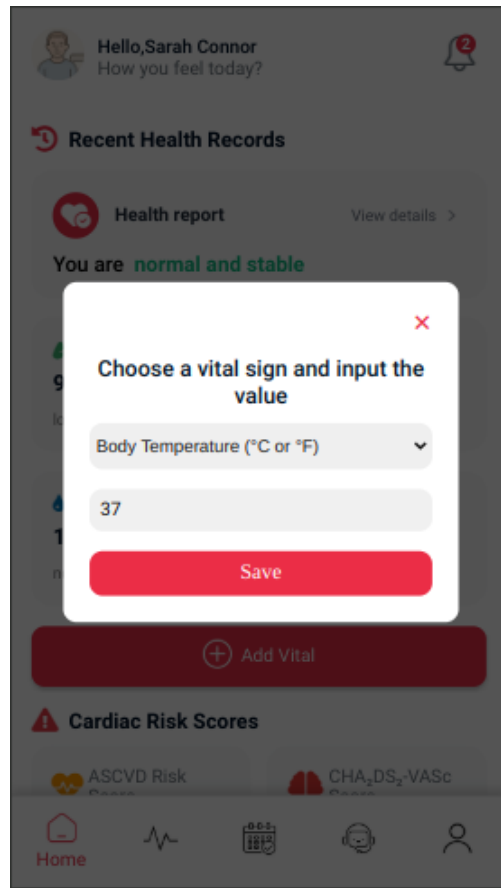


Figure 11.29: Add vital signs screen. Allows manual entry of vital data such as heart rate, temperature, and blood pressure.

11.5.2.4 Inbox Screen

- **Accessibility:** Available to connected patients.
- **Functionality:** This screen displays the most recent incoming appointments and messages sent by the doctor. Each item is presented in a simple and informative card format, containing essential information such as the appointment details or message content. These cards are interactive patients can click to confirm or decline appointments, as well as mark messages as read and update their status. Additionally, the screen provides links that allow the patient to view the full history of their appointments and message exchanges, including messages sent by themselves or their doctor. It also enables the patient to check whether the doctor has responded to their previous messages.
- **State:**

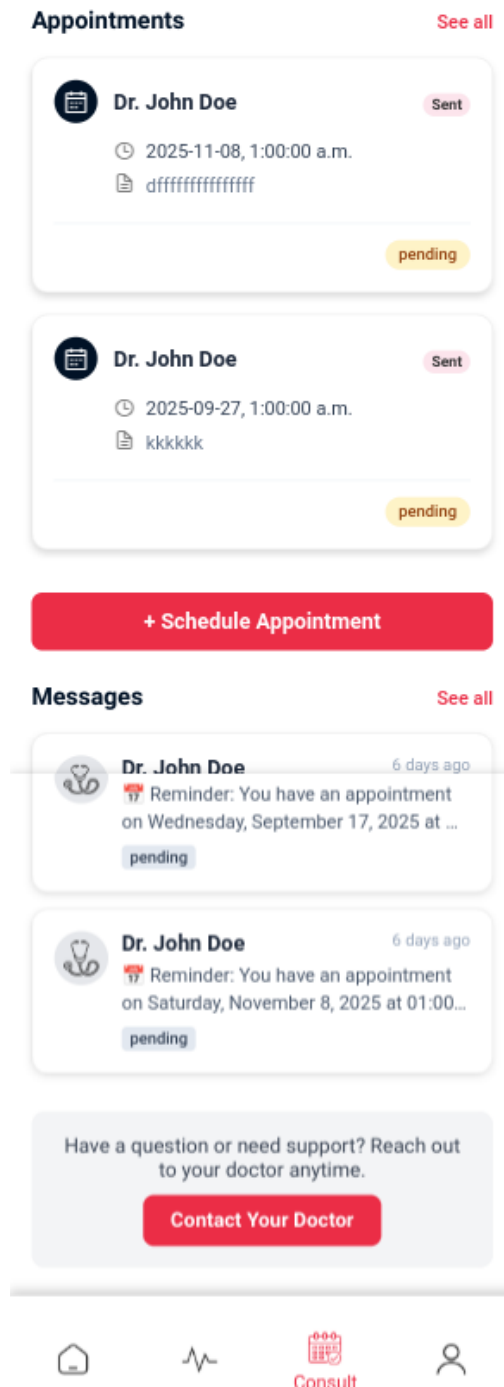


Figure 11.30: Inbox screen. Patients can view recent messages and appointments, interact with them, and access their full communication history with the doctor.

11.5.2.5 Schedule Appointment Screen

- **Accessibility:** Available to connected patients.
- **Functionality:** This screen allows patients to request an appointment by selecting a suitable date, time, and type of appointment. The interface displays the doctor's availability and provides a clear agenda input field to help the patient specify the purpose of the appointment. This streamlines scheduling and helps doctors prepare in advance.

– **State:**

← Schedule Appointment

September 2025

Sun	Mon	Tue	Wed	Thu	Fri	Sat
31	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30	1	2	3	4

Available Time

09:00

09:30

10:00

10:30

11:00

11:30

13:00

13:30

14:00

14:30

15:00

15:30

16:00

16:30

Appointment Options

consultation

follow-up

surgery

examination

vaccination

therapy

other

Book Appointment

Figure 11.31: Schedule Appointment screen. Patients can request an appointment by selecting a date, time, type, and specifying the agenda based on the doctor’s availability.

11.5.2.6 Contact Doctor Screen

- **Accessibility:** Available to connected patients.
- **Functionality:** Acts as a direct communication channel between the patient and their assigned doctor. The interface provides a simple and intuitive form where the patient can select the type of message, write their query or concern, and optionally attach files or images to support their message.
- **State:**

←

Contact Doctor



Dr. Sarah Johnson

Cardiology

 Response time: Usually responds within 24 hours

 Next available: Tomorrow at 2:00 PM

What is your question about?*

Medication Question

Your Message*

Can this medication interact with any over-the-counter supplements I'm taking?

79/1000

Attachments (optional)

You can upload test results, photos, or documents

 Add File

 Take Photo

> Send Message

For urgent medical concerns, please call emergency services or visit the nearest hospital.

Figure 11.32: Contact Doctor screen. Patients can select a message type, compose a message, and attach files or images to communicate with their doctor.

11.5.2.7 Patient Profile Screen

- **Accessibility:** Accessible to all connected and authorized patients.
- **Functionality:** This screen displays the patient’s medical information, including diagnoses, treatments, and personal details. It also provides options for account management, such as logging out or deactivating the account.
- **State:**

Profile

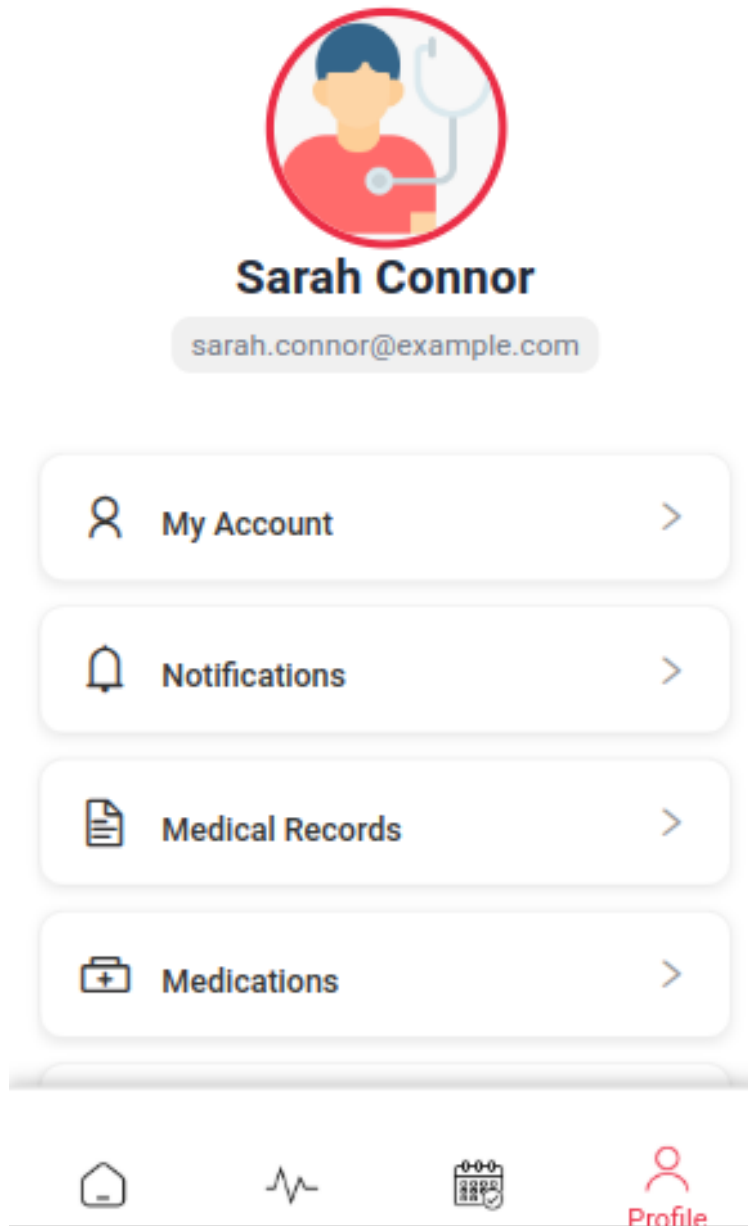


Figure 11.33: Patient Profile screen. Patients can view their medical information, including diagnoses, treatments, and personal details, as well as manage their account settings such as logging out or deactivating the account.

11.6 Conclusion

This chapter described the full implementation of the healthcare platform, including system architecture, APIs, pre-trained model integration, and user interface design across web and mobile environments. The implementation confirms the feasibility of the proposed design and provides a functional system that integrates medical data management, knowledge graph reasoning, and Q/A support into a unified solution.

Part VI

Conclusion

12.1 Conclusion

This project was carried out in the context of my final year thesis and focused on developing a medical diagnostic platform integrated with a Q/A system powered by Knowledge Graphs (KGs). The platform was designed to work alongside doctors' Electronic Medical Records (EMRs), enabling them to ask natural language questions and receive personalized diagnostic support tailored to patient-specific information.

The work followed several key steps:

- **Literature Review:** We began by reviewing existing research on Q/A systems based on KGs in the medical domain, exploring different techniques for KG construction and the role of embedding models in inferring relevant knowledge.
- **Domain Selection & KG Development:** The cardiovascular domain was chosen as the main use case. A domain-specific KG was built, and multiple embedding models were trained and compared to evaluate their ability to support link prediction and knowledge inference.
- **Q/A System Integration:** The selected model was integrated into a multilingual Q/A system capable of handling both in-context and out-of-context queries. The system was enhanced with medical entity recognition and intent detection to ensure accurate extraction of relevant information from the KG.
- **Platform Implementation:** Finally, the solution was embedded into a broader platform that functions as both an EMR system for doctors and an interactive tool for patients. For doctors, it supports patient management, appointment scheduling, and payment handling. For patients, the platform enables direct communication with their doctor through notifications and provides real-time sharing of vital signs using medical sensors. This ensures continuous monitoring and a stronger doctor-patient connection.

Overall, the project demonstrates how combining KGs, embedding models, and Q/A systems can significantly enhance clinical decision support while also empowering pa-

tients through continuous monitoring and personalized interaction with their healthcare providers.

12.2 Perspectives

With the contributions of this project, several perspectives can be considered for future work to further enhance the platform’s functionality and impact:

- **Integration with Real EMR Data:** Incorporate real-world Electronic Medical Records (EMRs) to construct a more comprehensive and clinically valid Knowledge Graph.
- **Patient-Oriented QA System:** Build a dedicated medical QA application for patients, similar to a symptom checker, that provides personalized advice and prevention strategies in simple, clear, and understandable language. This would make the system accessible to non-experts and empower patients in managing their own health.
- **Domain Expansion:** Extend the system beyond cardiovascular diseases to cover other medical specialties such as oncology, neurology, or pediatrics.
- **Doctor-Oriented Interactive Graph Exploration:** Enable doctors to interact with the medical knowledge graph directly by exploring, refining, and drilling down into subgraphs. Natural language interactions would be automatically translated into SPARQL queries behind the scenes, allowing clinicians to navigate and query the KG intuitively while maintaining the precision of structured queries.
- **Patient-Centric Features:** Develop smarter monitoring tools with predictive analytics on patient vital signs to enable early warning systems and proactive medical interventions.
- **Privacy and Security Enhancements:** Strengthen data protection mechanisms by integrating privacy-preserving machine learning techniques such as federated learning and differential privacy.

Part VII

Bibliography

CHAPTER 13

BIBLIOGRAPHY

- [1] Zhe Chen, Yuehan Wang, Bin Zhao, Jing Cheng, Xin Zhao, Zongtao Duan. “Knowledge Graph Completion: A Review”.In:IEEE (2020)
URL :<https://ieeexplore.ieee.org/abstract/document/9220143>
- [2] Zifang Tang,Tong Li ,Di Wu,Junrui Liu,Zhen Yang. “A Systematic Literature Review of Reinforcement Learning-based Knowledge Graph Research”.In:Science Direct (2024)
URL :<https://www.sciencedirect.com/science/article/pii/S0957417423023825>
- [3] “Top 10 Open Source Graph Databases in 2025“ URL: <https://www.geeksforgeeks.org/open-source-graph-databases/>
- [4] “Evaluation Metrics“ URL: <https://deepai.org/machine-learning-glossary-and-terms/evaluation-metrics>
- [5]M. Uschold, M. Gruninger, “Ontologies: Principles, Methods and Applications “ Knowledge Engineering Review; Vol. 11 N°2, 1996
- [6] Yuanfei Dai ,Shiping Wang ,Neal N. Xiong ,Wenzhong Guo “A Survey on Knowledge Graph Embedding: Approaches, Applications and Benchmarks“.In:MDPI (2020) URL: <https://www.mdpi.com/2079-9292/9/5/750>
- [7] MALKI Mimoun (2023, November 22). Chapitre 2 – Ingénierie ontologique. Département d’Informatique, 3ASC Ingénieur: SIW & IASD, Semestre S5.
- [8] MALKI Mimoun. (2023). Chapitre 3 – Modèles GC. Master 3ASC, Semestre S5
- [9] <https://store.arduino.cc/products/arduino-uno-rev3>
- [10] Lino Murali and G. Gopakumar and Daleesha M. Viswanathan and Prema Nedun-
gadi “Towards electronic health record-based medical knowledge graph construction,
completion, and applications: A literature study“.In:Journal of Biomedical Informatics

- [11] <https://www.python.org/>
- [12] “Comparison of Knowledge Graph Embeddings (KGE) models“.In:Medium (2023)
<https://medium.com/@shreyash.pandey.katni/comparison-of-knowledge-graph-embeddings-kge-models-2bcd52ca3f76>
- [13] “What is question answering?“.In:IBM (2025)
[https://www.ibm.com/think/topics/question-answering#:~:text=Question%20answering%20\(QA\)%20is%20a,natural%20language%20with%20natural%20language.](https://www.ibm.com/think/topics/question-answering#:~:text=Question%20answering%20(QA)%20is%20a,natural%20language%20with%20natural%20language.)
- [14] Bolanle Ojokoh,Emmanuel Adebisi “A Review of Question Answering Systems“.In:Researchgate (2019)
https://www.researchgate.net/publication/333627091_A_Review_of_Question_Answering_Systems
- [15]PyKEEN <https://pykeen.readthedocs.io/en/stable/>
- [16] <http://torch.ch/>
- [17] <https://pytorch-geometric.readthedocs.io/en/latest/>
- [18] <https://neo4j.com/product/>
- [19] https://cloudinary.com/documentation/cloudinary_references
- [20] Busra Oguzoglu “Creating Embeddings from Knowledge Graphs Using Graph Neural Networks“.In: medium (2024)
<https://medium.com/@busra.oguzoglu/creating-embeddings-from-knowledge-graphs-using-graph-neural-networks-ffc6cc62275c>
- [21]“What are Graph Neural Networks? “.In: Geeksforgeeks (2025)
<https://www.geeksforgeeks.org/what-are-graph-neural-networks/>
- [22]“Introduction to Graph Neural Networks (GNNs)“.In: medium (2023)
<https://medium.com/@tejpal.abhyuday/introduction-to-graph-neural-networks-gnns-f79ca974a57d>
- [23]“Graph convolutional neural networks“.In: mbernste (2023)
<https://mbernste.github.io/posts/gcn/>
- [24]“Graph Neural Networks Part 2. Graph Attention Networks vs. GCNs“.In: Towards Data Science (2024)
<https://towardsdatascience.com/graph-neural-networks-part-2-graph-attention-networks-vs-gcns-029efd7a1d92/>

- [25]“Knowledge Embedding Based Graph Convolutional Network“ Michael Schlichtkrull, Thomas N. Kipf, Peter Bloem, Rianne van den Berg, Ivan Titov, Max Welling .In:arXiv (2017)
<https://towardsdatascience.com/graph-neural-networks-part-2-graph-attention-networks-vs-gcns-029efd7a1d92/>
- [26] “What is Visual Studio (VS) Code?“ Dominykas J. Sep 12, 2025
<https://www.hostinger.com/tutorials/what-is-vs-code>
- [27] Google Colaboratory
<https://colab.google/>
- [28]“Capsule Networks (CapsNet)“In: Ultralytics
<https://www.ultralytics.com/glossary/capsule-networks-capsnet>
- [29]“Capsule Networks: A Quick Primer “In: digitalocean (2024)
<https://www.digitalocean.com/community/tutorials/capsule-networks>
- [30]“Knowledge Graph Embedding“ (2024)
https://medium.com/@manjua_75748/knowledge-graph-embedding-bdcd487d2fff
- [31]Shivani Choudhary, Tarun Luthra, Ashima Mittal, Rajat Singh “A Survey of Knowledge Graph Embedding and Their Applications“ (2021)
<https://arxiv.org/abs/2107.07842>
- [32] “Using Embeddings to Predict New Links in Knowledge Graphs“ (2021)
<https://www.andrearossi.io/link-prediction-primer/>
- [33] Gediminas Adomavicius and Alexander Tuzhilin. “Toward the next generation of recommender systems: a survey of the state-of-the-art and possible extensions“In:IEEE transactions on knowledge and data engineering (2005)
<https://ieeexplore.ieee.org/document/1423975>
- [34] Q. Guo, F. Zhuang, C. Qin, H. Zhu, X. Xie, H. Xiong, and Q. He “A Survey on Knowledge Graph-Based Recommender Systems“ In:IEEE Transactions on Knowledge and Data Engineering (Volume: 34, Issue: 8, 01 August 2022)
<https://ieeexplore.ieee.org/document/9216015>
- [35] Mahimai Raja J “Build Knowledge Graph From TextData using LangChain | Under 2min“ (2024)
<https://mahimairaja.medium.com/build-knowledge-graph-from-textdata-using-langchain-under-2min-ce0d0d0e44e8>
- [36] “Graph Database Query Languages You Should Try“(2022)
<https://memgraph.com/blog/graph-database-query-languages-you-should-try>
- [37] “Arduino“ https://en.wikipedia.org/wiki/Arduino#Legacy_IDE

- [38] Yonghong Chen, Hao Li, Han Li, Wenhao Liu, Yirui Wu, Qian Huang, Shaohua Wan “An Overview of Knowledge Graph Reasoning: Key Technologies and Applications“ In:Journal of Sensor and Actuator Networks 2022. DOI:10.3390/jsan11040078 <https://doi.org/10.3390/jsan11040078>
- [39] Wang, P.-W.Stepanova, D. Domokos, C. Kolter, J.Z. “Differentiable learning of numerical rules in knowledge graphs.“ In:Proceedings of the 8th International Conference on Learning Representations (ICLR), Addis Ababa, Ethiopia, 26–30 April 2020.
- [40] Guy Korland “How to Build a Knowledge Graph: A Step-by-Step Guide“ 2024 <https://www.falkordb.com/blog/how-to-build-a-knowledge-graph/>
- [41] “Named Entity Recognition“ (2025) <https://www.geeksforgeeks.org/nlp/named-entity-recognition/>
- [42] “Qu’est-ce que le grand modèle de langage (LLM)?“ <https://aws.amazon.com/what-is/large-language-model/>
- [43] <https://openrouter.ai/docs/quickstart>
- [44] <https://spacy.io/usage/models>
- [45] https://www.sbert.net/docs/sentence_transformer/pretrained_models.html [46] <https://huggingface.co/cambridgeltl/SapBERT-from-PubMedBERT-fulltext>
- [47] <https://www.libelium.com/libeliumworld/success-stories/mysignals-brings-physical-rehabilitation-to-people-with-disabilities-in-mexico-partnering-with-kinnov/>
- [48] Nilesh Chakraborty, Denis Lukovnikov, Gaurav Maheshwari, Priyansh Trivedi, Jens Lehmann, Asja Fischer “Introduction to Neural Network based Approaches for Question Answering over Knowledge Graphs“ IN:2019 URL:<https://doi.org/10.48550/arXiv.1907.09361>
- [49] “What is a GNN?“ URL:https://ignnition.org/doc/what_is_a_gnn.html
- [50] “Question-Answering-System-based-on-Tourism-Knowledge-Graph-using-state-of-the-art-BERT-embeddings“ URL:<https://github.com/ChristinaManoli/Question-Answering-System-based-on-Tourism-Knowledge-Graph-using-state-of-the-art-BERT-embeddings?tab=readme-ov-file>
- [51] Xuqing Chai “Diagnosis Method of Thyroid Disease Combining Knowledge Graph and Deep Learning“.In:IEEE Access, vol. 8, pp. 149787-149795 2020. DOI:10.1109/ACCESS.2020.3016676 <https://ieeexplore.ieee.org/document/9167227>

[52] Joana Vilela, Muhammad Asif, Ana Rita Marques, João Xavier Santos, Célia Rasga, Astrid Vicente, Hugo Martiniano “Biomedical knowledge graph embeddings for personalized medicine: Predicting disease-gene associations“ IN:Expert Systems . 2022. DOI:10.1111/exsy.13181
<https://doi.org/10.1111/exsy.13181>

[53] Yang R, Ye Q, Cheng C, Zhang S, Lan Y, Zou J. “Decision-Making System for the Diagnosis of Syndrome Based on Traditional Chinese Medicine Knowledge Graph.“ IN:Evid Based Complement Alternat Med 2022. DOI:10.1155/2022/8693937
<https://pubmed.ncbi.nlm.nih.gov/35186106/>

[54] Huang, Y., Chen, Q., Lin, Y., Yuan, W “A Knowledge Graph Based Question Answering Method for Medical Domain.“In:PeerJ Computer Science 7:e667 2021 DOI:10.7717/peerj-cs.667
<https://peerj.com/articles/cs-667/#fig-2>

[55] Zhixue Jiang, Chengying Chi, Yunyun Zhan “Research on Medical Question Answering System Based on Knowledge Graph“.In:IEEE Access, vol. 9, pp. 21094-21101, 2021. DOI:10.1109/ACCESS.2021.3055371.
<https://ieeexplore.ieee.org/document/9339902>

[56] Duan Y, Zhou Q, Li Y, Qin C, Wang Z, Kan H, Hu J. “Research on a traditional Chinese medicine case-based question-answering system integrating large language models and knowledge graph“ IN:Front Med (Lausanne). 2025. DOI:10.3389/fmed.2024.151232
URL:<https://pubmed.ncbi.nlm.nih.gov/39839612/>

[57] Yin Y, Zhang L, Wang Y, Wang M, Zhang Q, Li GZ. “Question Answering System Based on Knowledge Graph in Traditional Chinese Medicine Diagnosis and Treatment of Viral Hepatitis B“ IN:Biomed Res Int. 2022.DOI: 10.1155/2022/7139904
URL:<https://pubmed.ncbi.nlm.nih.gov/35198638/>

[58]Pham Van Duong, Tien-Dat Trinh, Hai Van Pham, Tran Manh Tuan, Le Hoang Son, Huy-The Vu, Minh-Tien Nguyen, Pham Minh Chuan. “A New Approach of Question Answering based on Knowledge Graph in Traditional Medicine” in pp. 285–288, ACSIS, Vol. 33, 2022. DOI: 10.15439/2022R44.
URL:https://annals-csis.org/Volume_33/drp/pdf/44.pdf

[59] Qin Z., Wu D., Zang Z., Chen X., Zhang H., Wong C.U.I. “Building an intelligent diabetes Q&A system with knowledge graphs and large language models“ In: Public Health, 13:1540946 (2025). DOI: 10.3389/fpubh.2025.1540946
URL:<https://www.frontiersin.org/journals/public-health/articles/10.3389/fpubh.2025.1540946/full>

[60] “What is the Internet of Things (IoT)?“
URL:<https://www.ibm.com/think/topics/internet-of-things>

[61] “What is the Internet of Things (IoT)?“
URL:<https://www.techtarget.com/iotagenda/definition/IoMT-Internet-of-Medical-Things>

[62] “10 INTERNET OF THINGS (IoT) HEALTHCARE EXAMPLES“

URL:<https://ordr.net/article/iot-healthcare-examples>

[63] “How IoT Devices Transform Healthcare with Real-Time Data Collection“

URL:<https://www.iotforall.com/how-iot-devices-transform-healthcare-with-real-time-data-collection>

[64] “How IoT is Transforming the Healthcare Industry“

URL:<https://embeddedcomputing.com/application/healthcare/telehealth-healthcare-iot/how-iot-is-transforming-the-healthcare-industry>